

AMITY UNIVERSITY MADHYA PRADESH, GWALIOR

SESSION: 2025-2026

LAB FILE

NEURAL NETWORKS AND DEEP LEARNING LAB

CSE421



BATCH: 2024-2028

SUBMITTED TO:

Mr. Praveen Gupta
Assistant Professor,
ASET, AUMP

SUBMITTED BY:

Vinayak Pawar
B. Tech CSE – IV ‘B’
A60205224241

Department of Computer Science and Engineering,
Amity School of Engineering and Technology,
Amity University Madhya Pradesh

INDEX

S.NO.	LIST OF PROGRAMS	DATE	SIGNATURE
1	Introduction of Deep Learning with ANN and CNN architecture		
2	Brief Description of Libraries –		
3	Implementation of Logic Gates		
4	CNN Architecture of Alzheimer dataset		
5	MNIST – Deep Neural Network with Keras		
6	Stock Market Prediction using LSTM		
7	Gated Recurrent Unit (GRU) Implementation		
8	U-Net Architecture for Brain Tumor Segmentation (BraTS)		
9	R-CNN (Region-Based Convolutional Neural Network) Implementation		
10	YOLO Object Detection		
11	VGG16 — From Scratch & Transfer Learning		
12	Faster R-CNN Object Detection		

Introduction of Deep Learning with ANN and CNN architecture

Deep Learning: Deep Learning is a specialized subset of Machine Learning that uses artificial neural networks with multiple hidden layers to automatically learn complex patterns and hierarchical representations from large amounts of raw data. The term “**deep**” refers to the presence of two or more hidden layers between the input and output layers. Unlike traditional machine learning methods that require manual feature extraction, deep learning models can automatically identify important features such as edges, shapes, textures, speech patterns, or textual relationships. These models are inspired by the structure and functioning of the human brain and are capable of solving highly complex tasks with high accuracy. Deep Learning is widely used in applications such as image recognition, natural language processing, self-driving cars, medical diagnosis, recommendation systems, and voice assistants.

Types of Deep Learning Models

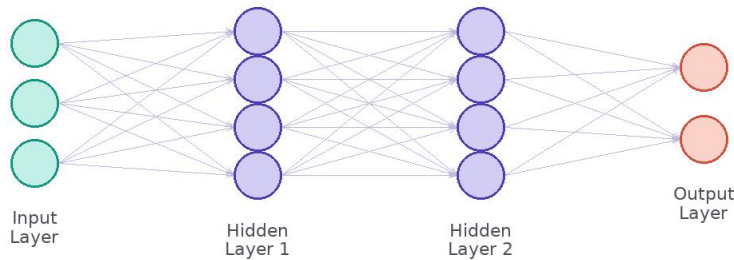
1. **ANN (Artificial Neural Network)** – A basic deep learning model made of interconnected neurons used for pattern recognition and prediction tasks.
2. **CNN (Convolutional Neural Network)** – A deep learning model mainly used for image processing and computer vision tasks.
3. **RNN (Recurrent Neural Network)** – A neural network designed for sequential data such as text, speech, and time-series analysis.
4. **LSTM (Long Short-Term Memory)** – An advanced type of RNN that can remember long-term dependencies in sequential data.
5. **GRU (Gated Recurrent Unit)** – A simplified version of LSTM used for sequence prediction with faster computation.
6. **Autoencoder** – A neural network used for data compression, feature learning, and noise reduction.
7. **GAN (Generative Adversarial Network)** – A model consisting of two networks that generate realistic images, videos, or data.
8. **Transformer Network** – A deep learning architecture widely used in natural language processing and large language models.
9. **DBN (Deep Belief Network)** – A probabilistic deep learning model used for feature extraction and unsupervised learning.
10. **SOM (Self-Organizing Map)** – A neural network used for clustering and visualization of high-dimensional data

① **ANN — Artificial Neural Network** is a deep learning model inspired by the structure of the human brain. It consists of interconnected neurons arranged in input, hidden, and output layers. ANN learns patterns from data by adjusting weights during training using backpropagation.

Forward pass: Input → Hidden layers (weighted sum + activation) → Output

ANN

Artificial Neural Network



Key Equations

Weighted sum:
 $z = \sum(w_i \cdot x_i) + b$
Activation:
 $a = f(z)$
ReLU:
 $f(z) = \max(0, z)$
Sigmoid:
 $f(z) = \frac{1}{1 + e^{-z}}$
Loss (MSE):
 $L = (1/n) \sum (y - \hat{y})^2$

Working of ANN:

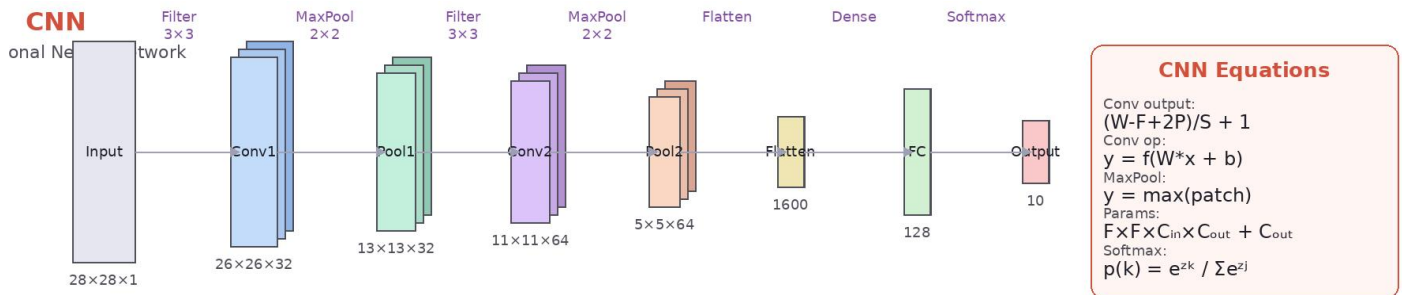
1. Input data is provided to the input layer.
2. Weighted sums are calculated using neurons.
3. Activation functions introduce non-linearity.
4. Output is generated and compared with actual values.
5. Error is minimized using backpropagation and gradient descent.

Types of Artificial Neural Network (ANN)

1. **Feed Forward Neural Network (FFNN)** – Information moves only in one direction from input to output layer.
2. **Single Layer Perceptron** – The simplest ANN having only one layer used for linear classification.
3. **Multi-Layer Perceptron (MLP)** – An ANN with multiple hidden layers used for complex pattern recognition.
4. **Radial Basis Function Network (RBFN)** – Uses radial basis functions for classification and function approximation.
5. **Recurrent Neural Network (RNN)** – An ANN where outputs are fed back as inputs for sequence-based tasks.
6. **Convolutional Neural Network (CNN)** – A specialized ANN mainly used for image and video processing.
7. **Modular Neural Network** – Consists of multiple independent neural networks working together on complex problems.
8. **Self-Organizing Map (SOM)** – An unsupervised ANN used for clustering and data visualization.
9. **Hopfield Network** – A recurrent ANN mainly used for memory storage and optimization problems.
10. **Boltzmann Machine** – A stochastic neural network used for learning probability distributions.

② **CNN — Convolutional Neural Network** is a specialized deep learning architecture mainly used for image processing and computer vision tasks. CNN automatically extracts features such as edges, shapes, and textures using convolution filters.

Conv filters detect local patterns; Pooling reduces spatial size; FC layers classify



Working of CNN:

1. Convolution layers apply filters to detect patterns.
2. Activation functions introduce non-linearity.
3. Pooling layers reduce dimensions and computation.
4. Fully connected layers classify the extracted features.
5. Model training occurs through backpropagation.

Types of Convolutional Neural Network (CNN)

1. **LeNet** – One of the earliest CNN architectures mainly used for handwritten digit recognition.
2. **AlexNet** – A deep CNN that significantly improved image classification accuracy using GPUs and ReLU activation.
3. **VGGNet** – A CNN architecture using very small convolution filters with increased depth for better accuracy.
4. **GoogLeNet (Inception Network)** – Uses inception modules to reduce computation while improving performance.
5. **ResNet (Residual Network)** – Introduces skip connections to solve the vanishing gradient problem in deep networks.
6. **DenseNet** – A CNN where each layer is connected to every other layer for efficient feature reuse.
7. **MobileNet** – A lightweight CNN designed for mobile and embedded devices.
8. **EfficientNet** – A CNN architecture that balances depth, width, and resolution for better efficiency.
9. **U-Net** – A CNN mainly used for image segmentation in medical imaging applications.
10. **RCNN (Region-Based CNN)** – A CNN used for object detection and localization in images.

③ Quick Reference

Equations	ANN vs CNN	Training Steps
$z = \sum(w_i x_i) + b$ [weighted sum]	ANN: fully connected every layer	1. Forward pass — compute $\hat{y}$
$a = f(z)$ [activation]	CNN: sparse local connections	2. Loss $L = (1/n)\sum(y - \hat{y})^2$
ReLU: $f(z) = \max(0, z)$	CNN output size: $(W-F+2P)/S+1$	3. Backprop — compute $\partial L / \partial w$
Sigmoid: $f(z) = 1/(1+e^{-z})$	CNN params: $F \times F \times C_{in} \times C_{out} + C_{out}$	4. Update: $w \leftarrow w - \alpha \cdot \partial L / \partial w$
Softmax: $p(k) = e^{z_k} / \sum e^{z_j}$	Pooling: $y = \max$ or $\text{avg}(\text{patch})$	5. Repeat until convergence

Brief Description of Libraries –

- **TensorFlow** – Open-source deep learning framework developed by Google.
- **Keras** – High-level API for building and training neural networks.
- **PyTorch (torch)** – Deep learning framework focused on flexibility and dynamic computation graphs.
- **TorchVision (torchvision)** – PyTorch package providing datasets, transforms, and pre-trained vision models.
- **Sequential** – Creates a linear stack of neural network layers.
- **Conv2D** – Performs 2D convolution for feature extraction from images.
- **MaxPooling2D** – Reduces feature map dimensions while preserving important features.
- **Flatten** – Converts multidimensional tensors into a 1D vector.
- **Dense** – Fully connected neural network layer.
- **Dropout** – Randomly deactivates neurons during training to reduce overfitting.
- **BatchNormalization** – Normalizes activations to improve training stability.
- **Activation** – Applies activation functions such as ReLU and Softmax.
- **SpatialDropout2D** – Drops entire feature maps during CNN training.
- **LSTM** – Long Short-Term Memory layer for sequential and time-series data.
- **GRU** – Gated Recurrent Unit layer, a simpler alternative to LSTM.
- **Bidirectional** – Processes sequence data in both forward and backward directions.
- **Adam** – Adaptive learning rate optimizer widely used in deep learning.
- **RMSprop** – Optimizer designed for recurrent neural networks.
- **regularizers** – Applies penalties to weights to reduce overfitting.
- **EarlyStopping** – Stops training when validation performance stops improving.
- **ReduceLROnPlateau** – Reduces learning rate when model performance stagnates.
- **ModelCheckpoint** – Saves the best model during training.
- **NumPy (numpy)** – Numerical computing library for arrays and matrix operations.
- **Pandas (pandas)** – Data analysis and manipulation library using DataFrames.
- **ImageDataGenerator** – Performs image preprocessing and data augmentation.
- **Dataset** – Base class for creating custom datasets in PyTorch.
- **DataLoader** – Loads datasets efficiently in batches during training.
- **MinMaxScaler** – Scales features to a specified range, usually [0,1].
- **LabelEncoder** – Converts categorical labels into numerical form.
- **StandardScaler** – Standardizes features to zero mean and unit variance.
- **train_test_split** – Splits data into training and testing sets.
- **Matplotlib (matplotlib.pyplot)** – Creates plots, charts, and visualizations.
- **Seaborn (seaborn)** – Statistical data visualization library built on Matplotlib.
- **Patches (matplotlib.patches)** – Draws bounding boxes and geometric shapes.
- **cm (matplotlib.cm)** – Provides color maps for visualizations.
- **scikit-learn (sklearn)** – Machine learning library for classification, regression, and preprocessing.
- **LinearSVC** – Linear Support Vector Machine classifier.
- **Ridge** – Linear regression with L2 regularization.
- **Pipeline** – Chains preprocessing and machine learning steps.
- **mean_squared_error (MSE)** – Measures average squared prediction error.
- **mean_absolute_error (MAE)** – Measures average absolute prediction error.
- **r2_score** – Calculates coefficient of determination (R^2).
- **OpenCV (cv2)** – Computer vision library for image and video processing.
- **Ultralytics YOLO** – Implementation of YOLOv8 for real-time object detection.
- **YOLO** – Object detection model that predicts bounding boxes and classes in one pass.
- **ResNet50** – Deep residual network commonly used as a feature extractor.
- **VGG16** – Deep CNN architecture consisting of 16 learnable layers.
- **R-CNN** – Region-based Convolutional Neural Network for object detection.

Program 1 – Implementation of Logic Gates

Library Used – Nil

CODE –

```
# AND Gate Implementation
```

```
def AND_gate(a, b):
```

```
    return a and b
```

```
print("AND Gate Truth Table")
```

```
print("A B Output")
```

```
for a in [0, 1]:
```

```
    for b in [0, 1]:
```

```
        print(a, b, AND_gate(a, b))
```

```
# OR Gate Implementation
```

```
def OR_gate(a, b):
```

```
    return a or b
```

```
print("OR Gate Truth Table")
```

```
print("A B Output")
```

```
for a in [0, 1]:
```

```
    for b in [0, 1]:
```

```
        print(a, b, OR_gate(a, b))
```

```
# XOR Gate Implementation
```

```
def XOR_gate(a, b):
```

```
    return a ^ b
```

```
print("XOR Gate Truth Table")
```

```
print("A B Output")
```

```
for a in [0, 1]:
```

```
    for b in [0, 1]:
```

```
        print(a, b, XOR_gate(a, b))
```

Outputs –

AND Gate Truth Table

A B Output

0 0 0

0 1 0

1 0 0

1 1 1

OR Gate Truth Table

A B Output

0 0 0

0 1 1

1 0 1

1 1 1

XOR Gate Truth Table

A B Output

0 0 0

0 1 1

1 0 1

1 1 0

Program 2 - CNN Architecture of Alzheimer dataset

Library used - from tensorflow.keras.preprocessing.image import ImageDataGenerator

```
import warnings
from keras.models import Sequential
from keras.layers import Conv2D
from keras.layers import Flatten
from keras.layers import MaxPooling2D
from keras.layers import Dense
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras import regularizers
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization, Activation, SpatialDropout2D)
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam
```

CODE –

```
warnings.filterwarnings("ignore")
```

```
train_dir = '/kaggle/input/datasets/vineetm1204/alzheimer-dataset/alzheimer dataset/train'
```

```
test_dir = '/kaggle/input/datasets/vineetm1204/alzheimer-dataset/alzheimer dataset/test'
```

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.4,
    zoom_range=0.2,
    width_shift_range=0.2,
    height_shift_range=0.2,
    rotation_range=90,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

```
test_datagen = ImageDataGenerator(rescale=1./255)
```

Vinayak Pawar (A60205224241)

```
train_generator = train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(128, 128),  
    color_mode='grayscale',  
    class_mode='categorical',  
)
```

```
test_generator = test_datagen.flow_from_directory(  
    test_dir,  
    target_size=(128, 128),  
    color_mode='grayscale',  
    class_mode='categorical',  
)
```

— Baseline CNN —————

```
CNN1 = Sequential()  
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=(128, 128, 1)))  
CNN1.add(MaxPooling2D(pool_size=(2, 2)))  
CNN1.add(Conv2D(64, (3, 3), activation='relu'))  
CNN1.add(MaxPooling2D(pool_size=(2, 2)))  
CNN1.add(Conv2D(256, (3, 3), activation='relu'))  
CNN1.add(MaxPooling2D(pool_size=(2, 2)))  
CNN1.add(Conv2D(128, (3, 3), activation='relu'))  
CNN1.add(MaxPooling2D(pool_size=(2, 2)))  
CNN1.add(Conv2D(256, (3, 3), activation='relu'))  
CNN1.add(MaxPooling2D(pool_size=(2, 2)))  
CNN1.add(Flatten())  
CNN1.add(Dense(128, activation='relu'))  
CNN1.add(Dropout(0.2))  
CNN1.add(Dense(64, activation='relu'))  
CNN1.add(Dense(4, activation='softmax'))  
CNN1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])  
CNN1.summary()  
CNN1.fit(train_generator, epochs=50)  
loss, accuracy1 = CNN1.evaluate(test_generator)  
print(f'Test Accuracy of CNN1: {accuracy1}')
```

— CNN + Batch Normalization —————

```

CNN1 = Sequential()
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=(128, 128, 1)))
CNN1.add(BatchNormalization())
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(64, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(128, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Flatten())
CNN1.add(Dense(128, activation='relu'))
CNN1.add(BatchNormalization())
CNN1.add(Dropout(0.2))
CNN1.add(Dense(64, activation='relu'))
CNN1.add(BatchNormalization())
CNN1.add(Dense(4, activation='softmax'))
CNN1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
CNN1.summary()
loss, accuracy1 = CNN1.evaluate(test_generator)
print(f'Test Accuracy of CNN1 with batch normalization: {accuracy1}')
```

— CNN + L1 Regularization —————

```

CNN1 = Sequential()
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=(128, 128, 1),
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(64, (3, 3), activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(256, (3, 3), activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
```

```

CNN1.add(Conv2D(128, (3, 3), activation='relu',
               kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Conv2D(256, (3, 3), activation='relu',
               kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(Flatten())
CNN1.add(Dense(128, activation='relu',
               kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(Dropout(0.2))
CNN1.add(Dense(64, activation='relu',
               kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(Dense(4, activation='softmax'))
CNN1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
CNN1.summary()
CNN1.fit(train_generator, epochs=50)
loss, accuracy1 = CNN1.evaluate(test_generator)
print(f'Test Accuracy of CNN1 with L1 Regularization: {accuracy1}')

```

```

# — CNN + BN + L2 + SpatialDropout —————
CNN1 = Sequential()
CNN1.add(Conv2D(256, (3, 3), input_shape=(128, 128, 1),
               kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))
CNN1.add(Conv2D(64, (3, 3),
               kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))
CNN1.add(Conv2D(256, (3, 3),
               kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))

```



```

CNN1.add(Conv2D(128, (3, 3),
               kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))
CNN1.add(Conv2D(256, (3, 3),
               kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))
CNN1.add(Flatten())
CNN1.add(Dense(128, kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(Dropout(0.5))
CNN1.add(Dense(64, kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(Dropout(0.5))
CNN1.add(Dense(4, activation='softmax'))
CNN1.compile(optimizer=Adam(learning_rate=0.0004),
             loss='categorical_crossentropy',
             metrics=['accuracy'])
CNN1.summary()
CNN1.fit(train_generator, epochs=50)
loss, accuracy1 = CNN1.evaluate(test_generator)
print(f'Test Accuracy of CNN1 with L2 Regularization: {accuracy1}')

```

Output –

Found 4647 images belonging to 4 classes.

Found 1232 images belonging to 4 classes.

Epoch 1/50

146/146 ————— 46s 239ms/step - accuracy: 0.5607 - loss: 1.0492

Epoch 2/50

146/146 ————— 15s 103ms/step - accuracy: 0.5131 - loss: 1.1229

...

39/39 ————— 11s 223ms/step - accuracy: 0.2596 - loss: 1.8337

Test Accuracy of CNN1: 0.2589285671710968

39/39 ————— 3s 67ms/step - accuracy: 0.9555 - loss: 0.1255

Test Accuracy of CNN1 with batch normalization: 0.9504064917564392

Epoch 1/50

146/146 ————— 23s 102ms/step - accuracy: 0.4489 - loss: 1.3857

...

39/39 ————— 5s 105ms/step - accuracy: 0.2803 - loss: 2.6809

Test Accuracy of CNN1 with L1 Regularization: 0.27516233921051025

Epoch 1/50

146/146 ————— 34s 153ms/step - accuracy: 0.2550 - loss: 2.7401

...

39/39 ————— 7s 146ms/step - accuracy: 0.3936 - loss: 1.2786

Test Accuracy of CNN1 with L2 Regularization: 0.38474026322364807

Program 3: MNIST - Deep Neural Network with Keras

In [1]:

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load in

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

import matplotlib.pyplot as plt # plotting library
%matplotlib inline

from keras.models import Sequential
from keras.layers import Dense, Activation, Dropout
from keras.optimizers import Adam, RMSprop
from keras import backend as K

# Input data files are available in the "../input/" directory.
# For example, running this (by clicking run or pressing Shift+Enter) will list the files in the input directory

from subprocess import check_output
print(check_output(["ls", "../input"]).decode("utf8"))

# Any results you write to the current directory are saved as output.
```

Using TensorFlow backend.

digit-recognizer

In [2]:

```
# import dataset
from keras.datasets import mnist

# load dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# count the number of unique train labels
unique, counts = np.unique(y_train, return_counts=True)
print("Train labels: ", dict(zip(unique, counts)))

# count the number of unique test labels
unique, counts = np.unique(y_test, return_counts=True)
print("\nTest labels: ", dict(zip(unique, counts)))
```

Downloading data from <https://s3.amazonaws.com/img-datasets/mnist.npz>

11493376/11490434 [=====] - 2s 0us/step

Train labels: {0: 5923, 1: 6742, 2: 5958, 3: 6131, 4: 5842, 5: 5421, 6: 5918, 7: 6265, 8: 5851, 9: 5949}

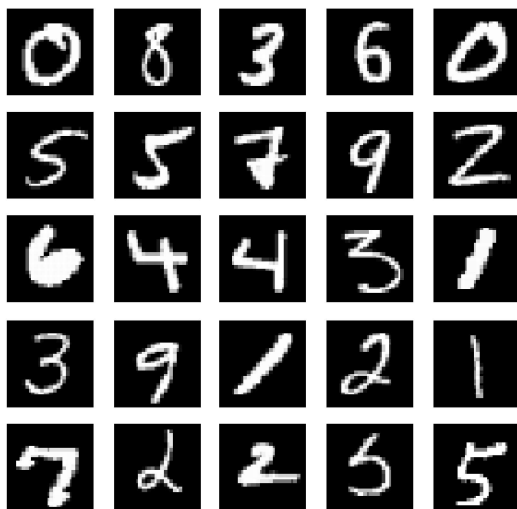
Test labels: {0: 980, 1: 1135, 2: 1354, 3: 1079, 4: 1414, 5: 892, 6: 958, 7: 1028, 8: 974, 9: 1009}

In [3]:

```
# sample 25 mnist digits from train dataset
indexes = np.random.randint(0, x_train.shape[0], size=25)
images = x_train[indexes]
labels = y_train[indexes]

# plot the 25 mnist digits
plt.figure(figsize=(5,5))
for i in range(len(indexes)):
    plt.subplot(5, 5, i + 1)
    image = images[i]
    plt.imshow(image, cmap='gray')
    plt.axis('off')

plt.show()
plt.savefig("mnist-samples.png")
plt.close('all')
```



In [4]:

```
from keras.models import Sequential
from keras.layers import Dense, Activation, Dropout
from keras.utils import to_categorical, plot_model
```

In [5]:

```
# compute the number of labels
num_labels = len(np.unique(y_train))
```

In [6]:

```
# convert to one-hot vector
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

In [7]:

```
# image dimensions (assumed square)
image_size = x_train.shape[1]
input_size = image_size * image_size
input_size
```

Out[7]:

784

In [8]:

```
# resize and normalize
```

```
x_train = np.reshape(x_train, [-1, input_size])
x_train = x_train.astype('float32') / 255
x_test = np.reshape(x_test, [-1, input_size])
x_test = x_test.astype('float32') / 255
```

In [9]:

```
# network parameters
batch_size = 128
hidden_units = 256
dropout = 0.45
```

In [10]:

```
# model is a 3-layer MLP with ReLU and dropout after each layer
model = Sequential()
model.add(Dense(hidden_units, input_dim=input_size))
model.add(Activation('relu'))
model.add(Dropout(dropout))
model.add(Dense(hidden_units))
model.add(Activation('relu'))
model.add(Dropout(dropout))
model.add(Dense(num_labels))
model.add(Activation('softmax'))
```

In [11]:

```
model.summary()
```

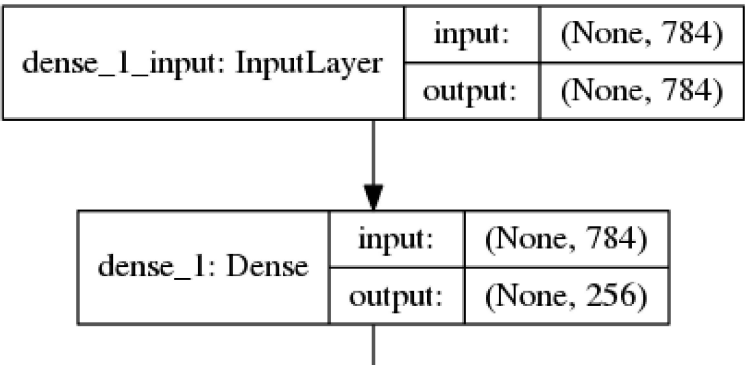
Model: "sequential_1"

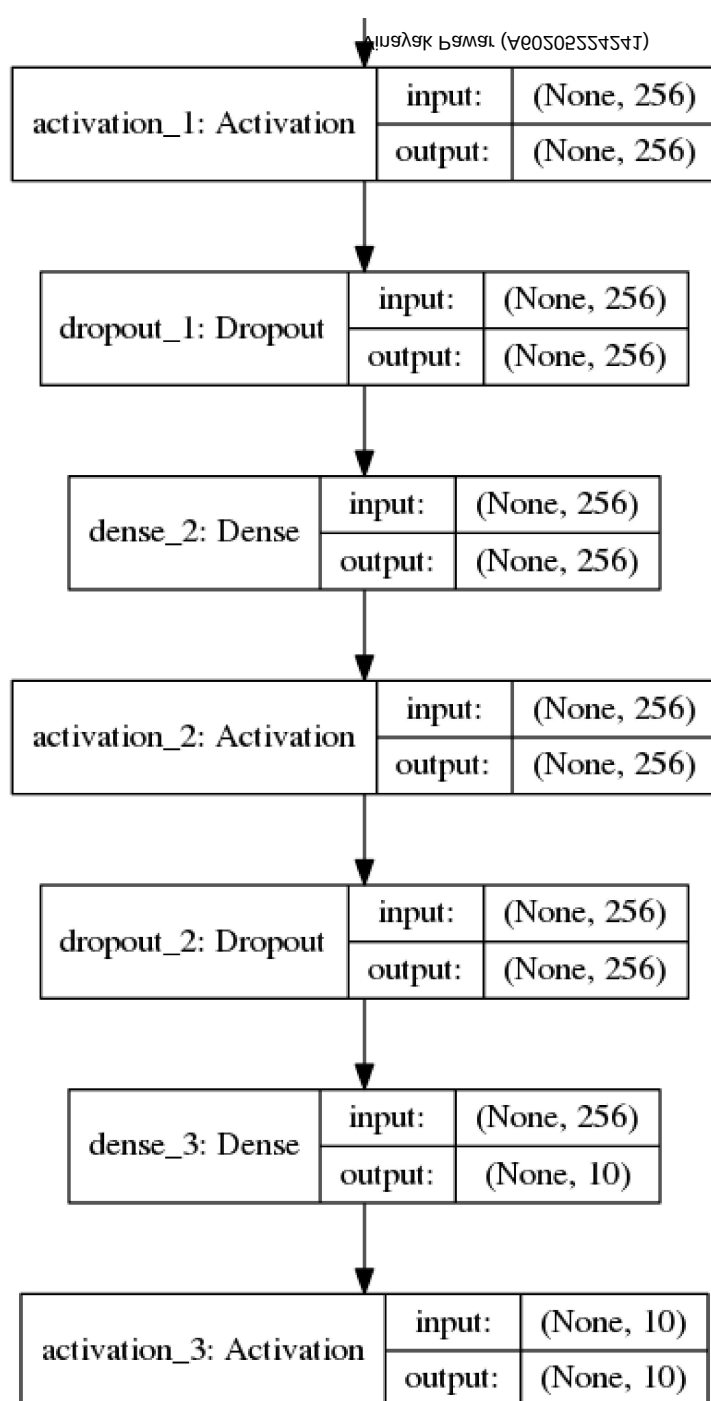
Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 256)	200960
activation_1 (Activation)	(None, 256)	0
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 256)	65792
activation_2 (Activation)	(None, 256)	0
dropout_2 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 10)	2570
activation_3 (Activation)	(None, 10)	0
Total params: 269,322		
Trainable params: 269,322		
Non-trainable params: 0		

In [12]:

```
plot_model(model, to_file='mlp-mnist.png', show_shapes=True)
```

Out[12]:





In [13]:

```
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

In []:

```
model.fit(x_train, y_train, epochs=20, batch_size=batch_size)
```

In [15]:

```
loss, acc = model.evaluate(x_test, y_test, batch_size=batch_size)
print("\nTest accuracy: %.1f%%" % (100.0 * acc))
```

10000/10000 [=====] - 0s 22us/step

Test accuracy: 98.2%

In [16]:

```
from keras.regularizers import l2
model.add(Dense(hidden_units,
                 kernel_regularizer=l2(0.001),
```

```
input_dim=input_size))
```

Program 4: Stock Market Prediction using LSTM

In []:

```
%pip install yfinance

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf # For fetching stock data
from datetime import datetime, timedelta
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout, GRU, Bidirectional
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import math
import warnings

# Suppress warnings
warnings.filterwarnings('ignore')

# Set style for plots
sns.set_style('whitegrid')
plt.style.use("fivethirtyeight")

# Check versions
print(f"TensorFlow version: {tf.__version__}")
print(f"Keras version: {keras.__version__}")
print(f"Pandas version: {pd.__version__}")
print(f"NumPy version: {np.__version__}")
```

In [4]:

```
# Define the stock tickers and time period
tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN']
start_date = '2018-01-01'
end_date = datetime.now().strftime('%Y-%m-%d')

# Function to fetch stock data
def get_stock_data(ticker, start, end):
    try:
        data = yf.download(ticker, start=start, end=end)
        data.reset_index(inplace=True)
        return data
    except Exception as e:
        print(f"Error fetching data for {ticker}: {e}")
        # Create synthetic data if fetch fails
        return generate_synthetic_stock_data(ticker, start, end)

# Function to generate synthetic stock data if API fetch fails
def generate_synthetic_stock_data(ticker, start_date, end_date):
    # Convert dates to datetime
    start = pd.to_datetime(start_date)
    end = pd.to_datetime(end_date)

    # Generate date range
    date_range = pd.date_range(start=start, end=end, freq='B') # 'B' for business days

    # Create random starting price based on ticker
    if ticker == 'AAPL':
        baseprice = 150
    elif ticker == 'MSFT':
```



```

    base_price = 250
elif ticker == 'GOOGL':
    base_price = 1200
elif ticker == 'AMZN':
    base_price = 1800
else:
    base_price = 100

# Generate random walk for prices with upward trend
n_days = len(date_range)
noise = np.random.normal(0, 1, n_days)
trend = np.linspace(0, 30, n_days) # Upward trend
seasonality = 10 * np.sin(np.linspace(0, 15 * np.pi, n_days)) # Seasonal pattern

# Combine components
prices = base_price + trend + seasonality + np.cumsum(noise)

# Create volume data
volume = np.random.randint(1000000, 10000000, n_days)

# Create DataFrame
df = pd.DataFrame({
    'Date': date_range,
    'Open': prices * np.random.uniform(0.99, 1.01, n_days),
    'High': prices * np.random.uniform(1.01, 1.03, n_days),
    'Low': prices * np.random.uniform(0.97, 0.99, n_days),
    'Close': prices,
    'Adj Close': prices,
    'Volume': volume
})

print(f"Generated synthetic data for {ticker}")
return df

# Fetch data for each ticker
stock_data = {}
for ticker in tickers:
    stock_data[ticker] = get_stock_data(ticker, start_date, end_date)
    print(f"Retrieved {len(stock_data[ticker])} days of data for {ticker}")

# Select one stock for detailed analysis (Apple)
selected_stock = 'AAPL'
df = stock_data[selected_stock].copy()

# Display the first few rows
df.head()

```

YF.download() has changed argument auto_adjust default to True

```
[*****100%*****] 1 of 1 completed
```

Retrieved 1854 days of data for AAPL

```
[*****100%*****] 1 of 1 completed
```

Retrieved 1854 days of data for MSFT

```
[*****100%*****] 1 of 1 completed
```

Retrieved 1854 days of data for GOOGL

```
[*****100%*****] 1 of 1 completed
```

Retrieved 1854 days of data for AMZN

Out[4]:

Price	Date	Close	High	Low	Open	Volume
Ticker		AAPL	AAPL	AAPL	AAPL	AAPL
0	2018-01-02	40.426826	40.436216	39.722772	39.933990	102223600
1	2018-01-03	40.419792	40.964263	40.356430	40.490198	118071600

Price	Date	Close	High	Low	Open	Volume
2	2018-01-04	40.607540	40.710802	40.384590	40.492543	89738400
Ticker		AAPL	AAPL	AAPL	AAPL	AAPL
3	2018-01-05	41.069859	41.156691	40.612224	40.703751	94640000
4	2018-01-08	40.917324	41.213026	40.818753	40.917324	82271200

In [5]:

```

# Check basic statistics
print(f"\nStatistics for {selected_stock}:")
print(df.describe())

# Check for missing values
print(f"\nMissing values in {selected_stock} data:")
print(df.isnull().sum())

# Plot the stock's closing price history
plt.figure(figsize=(16, 8))
plt.title(f'{selected_stock} Stock Price History')
plt.plot(df['Date'], df['Close'])
plt.xlabel('Date', fontsize=14)
plt.ylabel('Close Price (USD)', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Statistics for AAPL:

Price	Date	Close	High	Low \
Ticker		AAPL	AAPL	AAPL
count	1854	1854.000000	1854.000000	1854.000000
mean	2021-09-07 07:34:22.135922	127.071460	128.370795	125.623331
min	2018-01-02 00:00:00	33.870842	34.711717	33.825582
25%	2019-11-04 06:00:00	58.870005	60.070647	58.003616
50%	2021-09-07 12:00:00	137.429787	139.645323	135.496434
75%	2023-07-12 18:00:00	172.366409	173.933534	170.980190
max	2025-05-16 00:00:00	258.396667	259.474086	257.010028
std	NaN	61.769196	62.322379	61.115449

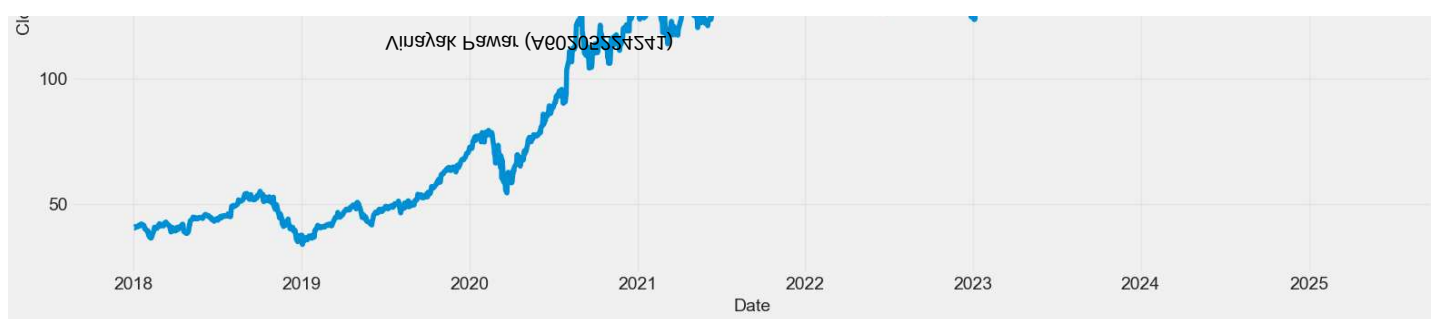
Price	Open	Volume
Ticker	AAPL	AAPL
count	1854.000000	1.854000e+03
mean	126.938500	9.807716e+07
min	34.297233	2.323470e+07
25%	58.957534	6.014090e+07
50%	137.397325	8.470835e+07
75%	172.219155	1.189786e+08
max	257.568678	4.265100e+08
std	61.692889	5.489307e+07

Missing values in AAPL data:

Price	Ticker
Date	0
Close	AAPL 0
High	AAPL 0
Low	AAPL 0
Open	AAPL 0
Volume	AAPL 0

dtype: int64





In [6]:

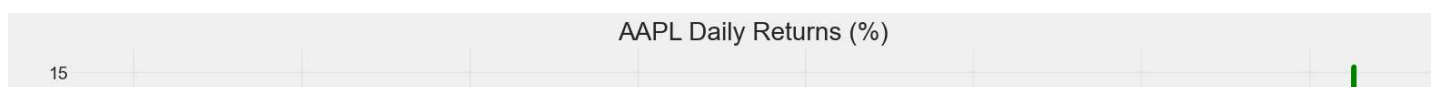
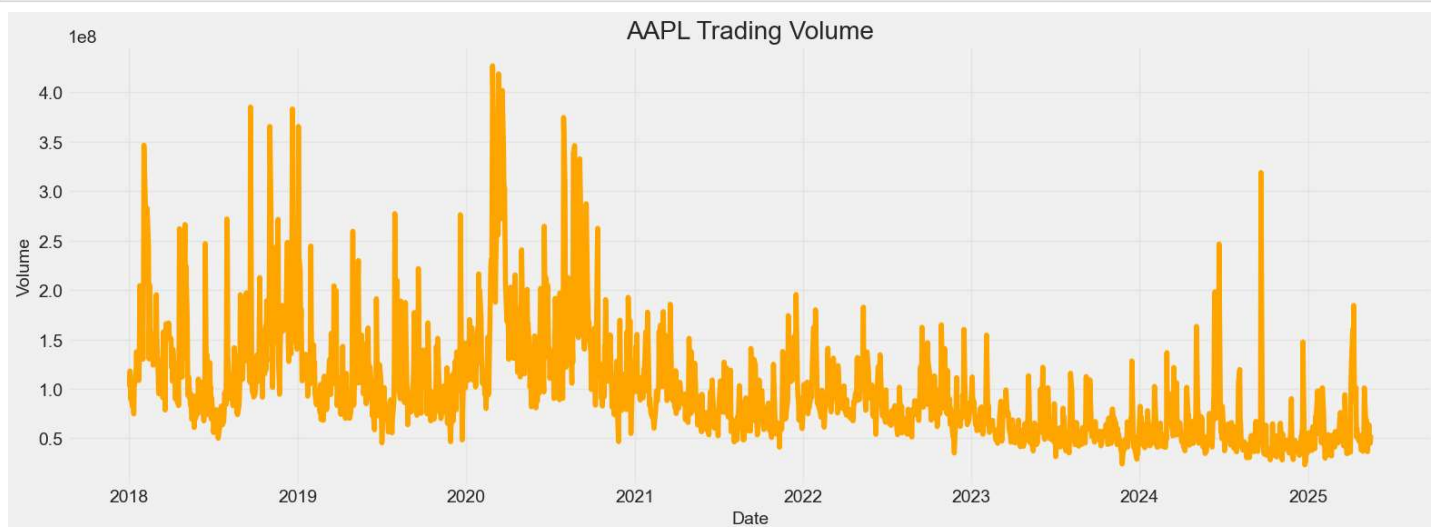
```
# Create additional plots to understand the data better

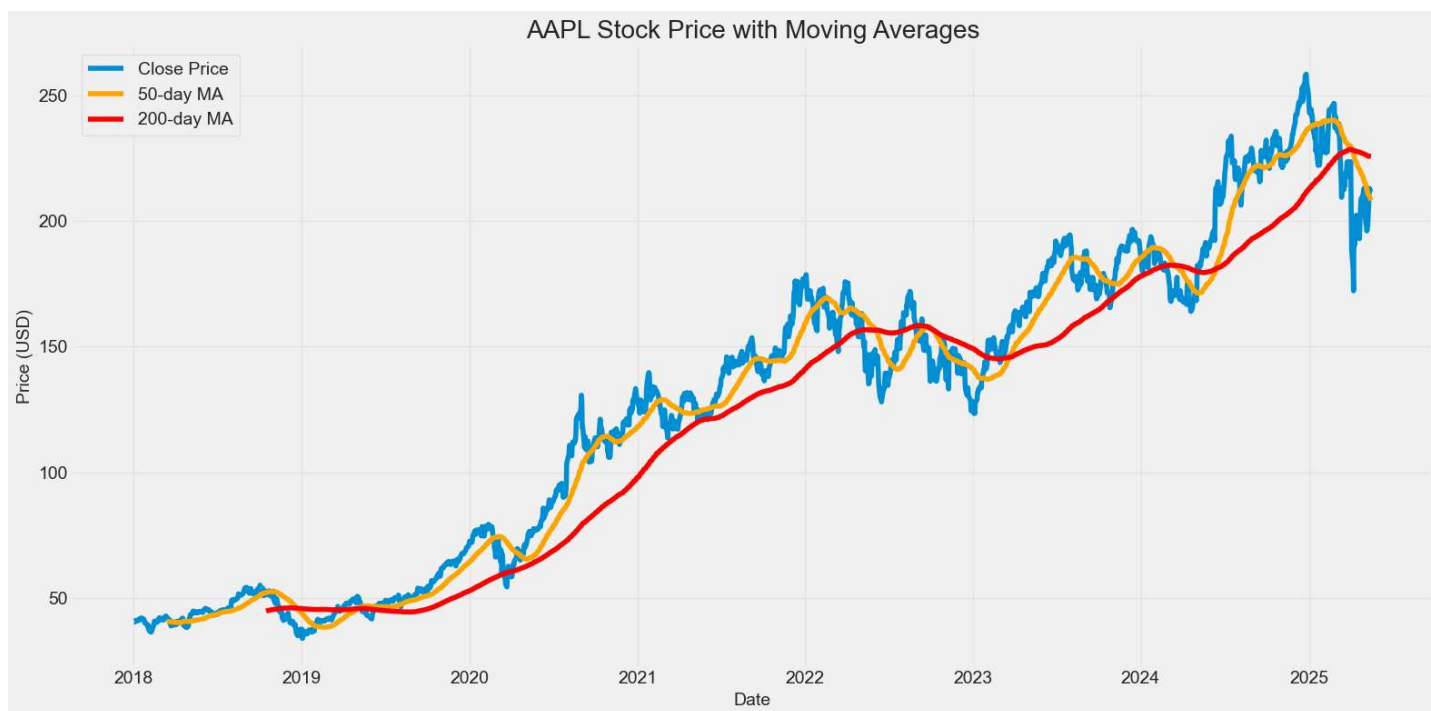
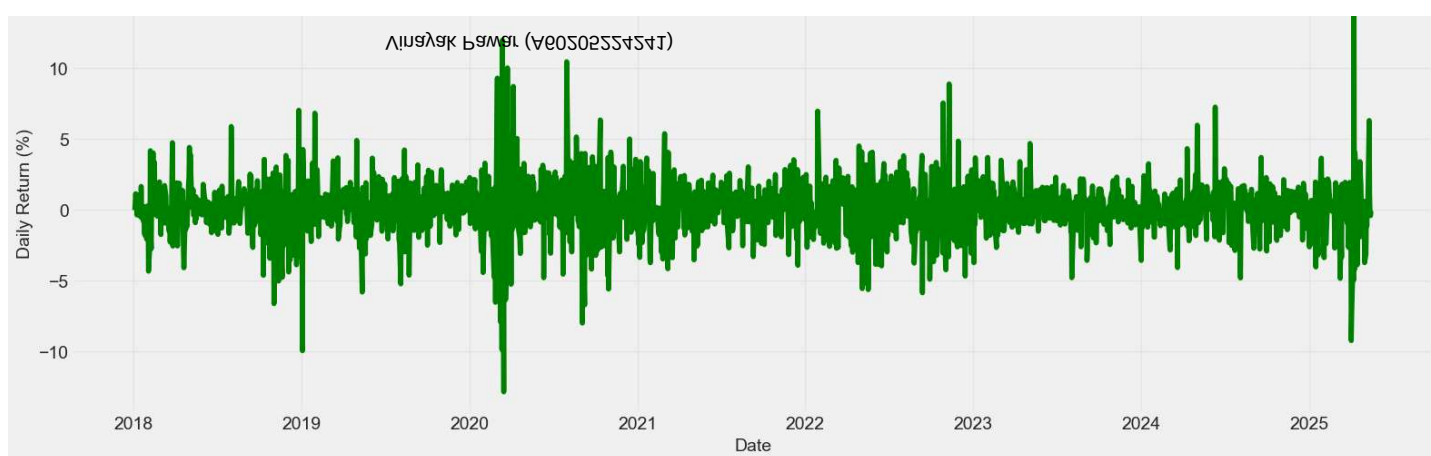
# Plot volume over time
plt.figure(figsize=(16, 6))
plt.plot(df['Date'], df['Volume'], color='orange')
plt.title(f'{selected_stock} Trading Volume')
plt.xlabel('Date', fontsize=14)
plt.ylabel('Volume', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Plot daily price change
df['Daily_Return'] = df['Close'].pct_change() * 100
plt.figure(figsize=(16, 6))
plt.plot(df['Date'], df['Daily_Return'], color='green')
plt.title(f'{selected_stock} Daily Returns (%)')
plt.xlabel('Date', fontsize=14)
plt.ylabel('Daily Return (%)', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Plot moving averages
df['MA50'] = df['Close'].rolling(window=50).mean()
df['MA200'] = df['Close'].rolling(window=200).mean()

plt.figure(figsize=(16, 8))
plt.plot(df['Date'], df['Close'], label='Close Price')
plt.plot(df['Date'], df['MA50'], label='50-day MA', color='orange')
plt.plot(df['Date'], df['MA200'], label='200-day MA', color='red')
plt.title(f'{selected_stock} Stock Price with Moving Averages')
plt.xlabel('Date', fontsize=14)
plt.ylabel('Price (USD)', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



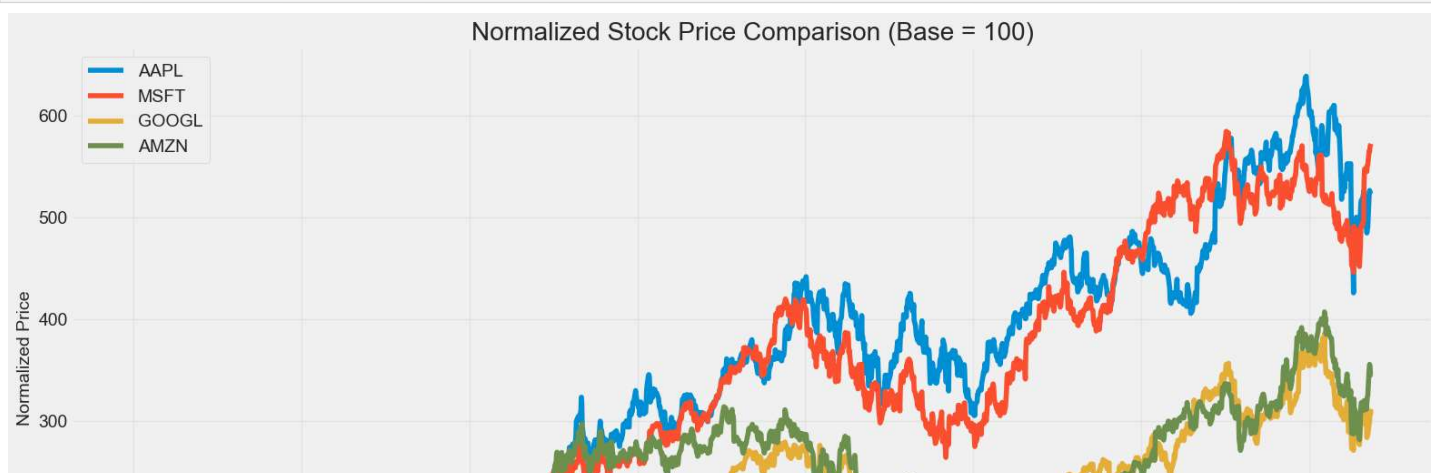


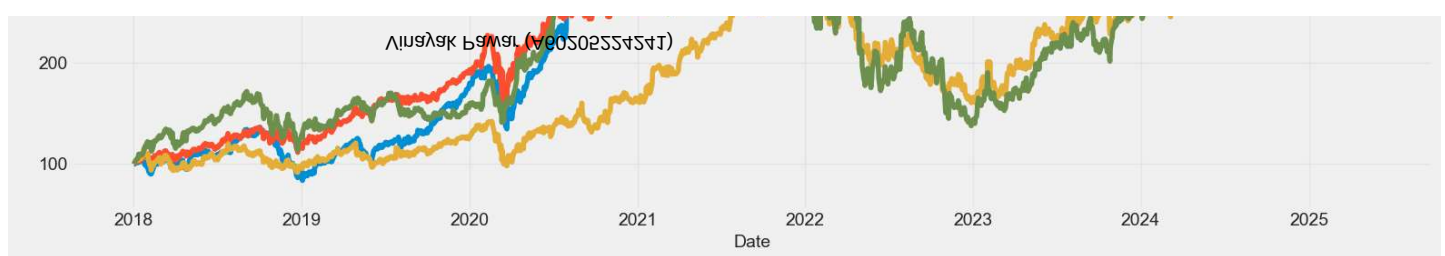
In [7]:

```
# Compare all stocks on the same chart
plt.figure(figsize=(16, 8))

for ticker in tickers:
    # Normalize to the starting price for fair comparison
    normalized = stock_data[ticker]['Close'] / stock_data[ticker]['Close'].iloc[0] * 100
    plt.plot(stock_data[ticker]['Date'], normalized, label=ticker)

plt.title('Normalized Stock Price Comparison (Base = 100)')
plt.xlabel('Date', fontsize=14)
plt.ylabel('Normalized Price', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```





In [8]:

```
# Prepare data for LSTM model
def prepare_lstm_data(data, target_col='Close', sequence_length=60, train_split=0.8):
    """Prepare data for LSTM model with proper scaling and sequence creation"""
    # Extract target column data
    target_data = data[target_col].values.reshape(-1, 1)

    # Scale the data
    scaler = MinMaxScaler(feature_range=(0, 1))
    scaled_data = scaler.fit_transform(target_data)

    # Determine training data length
    train_size = int(len(scaled_data) * train_split)
    train_data = scaled_data[:train_size]
    test_data = scaled_data[train_size - sequence_length:]

    # Create sequences for training
    X_train, y_train = [], []
    for i in range(sequence_length, len(train_data)):
        X_train.append(train_data[i - sequence_length:i, 0])
        y_train.append(train_data[i, 0])

    # Create sequences for testing
    X_test, y_test = [], []
    for i in range(sequence_length, len(test_data)):
        X_test.append(test_data[i - sequence_length:i, 0])
        y_test.append(test_data[i, 0])

    # Convert to numpy arrays
    X_train, y_train = np.array(X_train), np.array(y_train)
    X_test, y_test = np.array(X_test), np.array(y_test)

    # Reshape for LSTM [samples, time steps, features]
    X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
    X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

    # Return processed data and scaler for inverse transformation
    return X_train, y_train, X_test, y_test, scaler, train_size

# Prepare data for the selected stock
sequence_length = 60 # 60 days of data to predict the next day
X_train, y_train, X_test, y_test, scaler, train_size = prepare_lstm_data(
    df, target_col='Close', sequence_length=sequence_length, train_split=0.8
)

print(f"Training data shape: {X_train.shape}")
print(f"Test data shape: {X_test.shape}")

# Show date ranges for training and testing
train_dates = df['Date'][:train_size]
test_dates = df['Date'][train_size:]
print(f"Training date range: {train_dates.iloc[0]} to {train_dates.iloc[-1]}")
print(f"Testing date range: {test_dates.iloc[0]} to {test_dates.iloc[-1]}")
```

```
Training data shape: (1423, 60, 1)
Test data shape: (371, 60, 1)
Training date range: 2018-01-02 00:00:00 to 2023-11-21 00:00:00
Testing date range: 2023-11-22 00:00:00 to 2025-05-16 00:00:00
```

In [9]:

```
# Simple LSTM model
```

```

def create_simple_lstm_model(sequence_length):
    model = Sequential([
        LSTM(50, return_sequences=True, input_shape=(sequence_length, 1)),
        Dropout(0.2),
        LSTM(50, return_sequences=False),
        Dropout(0.2),
        Dense(25),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model

# Stacked LSTM model
def create_stacked_lstm_model(sequence_length):
    model = Sequential([
        LSTM(100, return_sequences=True, input_shape=(sequence_length, 1)),
        Dropout(0.2),
        LSTM(100, return_sequences=True),
        Dropout(0.2),
        LSTM(100, return_sequences=False),
        Dropout(0.2),
        Dense(50),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model

# Bidirectional LSTM model
def create_bidirectional_lstm_model(sequence_length):
    model = Sequential([
        Bidirectional(LSTM(50, return_sequences=True), input_shape=(sequence_length, 1))
        ,
        Dropout(0.2),
        Bidirectional(LSTM(50, return_sequences=False)),
        Dropout(0.2),
        Dense(25),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model

# Create models
simple_lstm_model = create_simple_lstm_model(sequence_length)
stacked_lstm_model = create_stacked_lstm_model(sequence_length)
bidirectional_lstm_model = create_bidirectional_lstm_model(sequence_length)

# Display model architecture
simple_lstm_model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 25)	1,275
dense_1 (Dense)	(None, 1)	26

Total params: 31,901 (124.61 KB)

Trainable params: 31,901 (124.61 KB)

Non-trainable params: 0 (0.00 B)

In []:

```
# Define callbacks for training
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, min_lr=1e-5)
]

# Train Simple LSTM
print("Training Simple LSTM Model...")
simple_history = simple_lstm_model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    callbacks=callbacks,
    verbose=1
)

# Train Stacked LSTM
print("\nTraining Stacked LSTM Model...")
stacked_history = stacked_lstm_model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    callbacks=callbacks,
    verbose=1
)

# Train Bidirectional LSTM
print("\nTraining Bidirectional LSTM Model...")
bidirectional_history = bidirectional_lstm_model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    callbacks=callbacks,
    verbose=1
)
```

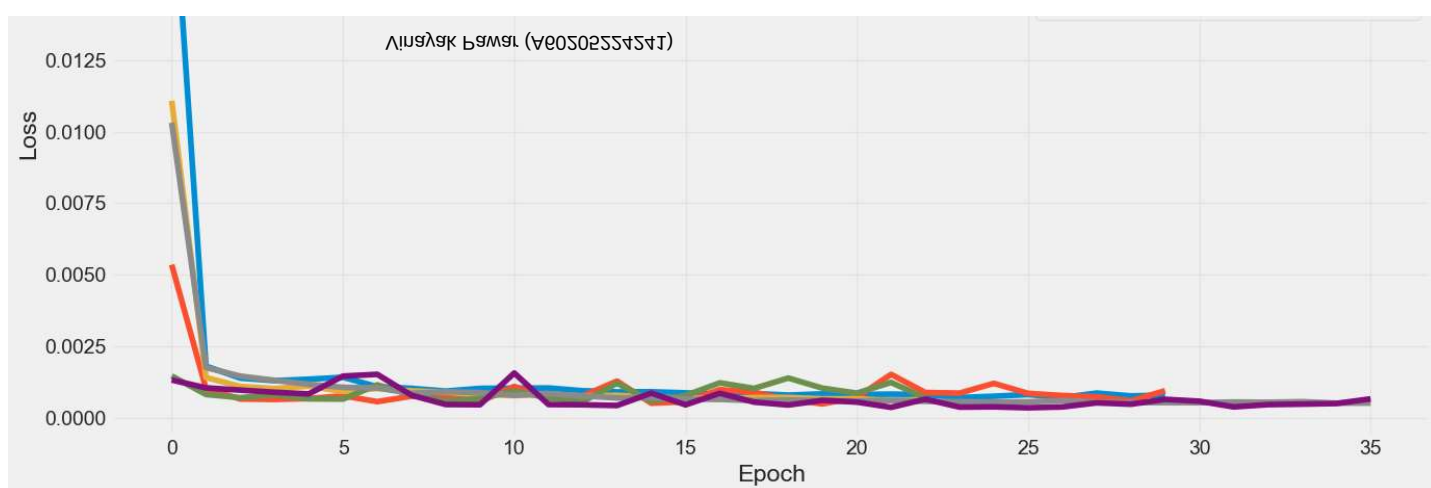
In [11]:

```
# Plot training history for all models
plt.figure(figsize=(14, 7))

plt.plot(simple_history.history['loss'], label='Simple LSTM Training Loss')
plt.plot(simple_history.history['val_loss'], label='Simple LSTM Validation Loss')
plt.plot(stacked_history.history['loss'], label='Stacked LSTM Training Loss')
plt.plot(stacked_history.history['val_loss'], label='Stacked LSTM Validation Loss')
plt.plot(bidirectional_history.history['loss'], label='Bidirectional LSTM Training Loss')
plt.plot(bidirectional_history.history['val_loss'], label='Bidirectional LSTM Validation Loss')

plt.title('Model Loss Comparison')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```





In [12]:

```
# Function to evaluate model performance
def evaluate_model(model, X_test, y_test, scaler, model_name):
    """Evaluate a model and return predictions and metrics"""
    # Make predictions
    predictions = model.predict(X_test)

    # Inverse transform to get actual price values
    y_test_inv = scaler.inverse_transform(y_test.reshape(-1, 1)).flatten()
    predictions_inv = scaler.inverse_transform(predictions).flatten()

    # Calculate error metrics
    mse = mean_squared_error(y_test_inv, predictions_inv)
    rmse = math.sqrt(mse)
    mae = mean_absolute_error(y_test_inv, predictions_inv)
    r2 = r2_score(y_test_inv, predictions_inv)

    print(f"\n{model_name} Performance Metrics:")
    print(f"MSE: {mse:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"MAE: {mae:.2f}")
    print(f"R² Score: {r2:.4f}")

    return predictions_inv, y_test_inv, {
        'mse': mse,
        'rmse': rmse,
        'mae': mae,
        'r2': r2
    }

# Evaluate all models
simple_pred, y_test_inv, simple_metrics = evaluate_model(simple_lstm_model, X_test, y_test, scaler, "Simple LSTM")
stacked_pred, _, stacked_metrics = evaluate_model(stacked_lstm_model, X_test, y_test, scaler, "Stacked LSTM")
bidirectional_pred, _, bidirectional_metrics = evaluate_model(bidirectional_lstm_model, X_test, y_test, scaler, "Bidirectional LSTM")
```

12/12 ————— 1s 48ms/step

Simple LSTM Performance Metrics:

MSE: 77.65

RMSE: 8.81

MAE: 6.84

R² Score: 0.8666

12/12 ————— 1s 75ms/step

Stacked LSTM Performance Metrics:

MSE: 88.82

RMSE: 9.42

MAE: 7.42

R² Score: 0.8475

12/12 ————— 2s 81ms/step

Bidirectional LSTM Performance Metrics:

Bidirectional LSTM Performance Metrics.

MSE: 54.49

RMSE: 7.38

MAE: 5.69

R² Score: 0.9064

In [13]:

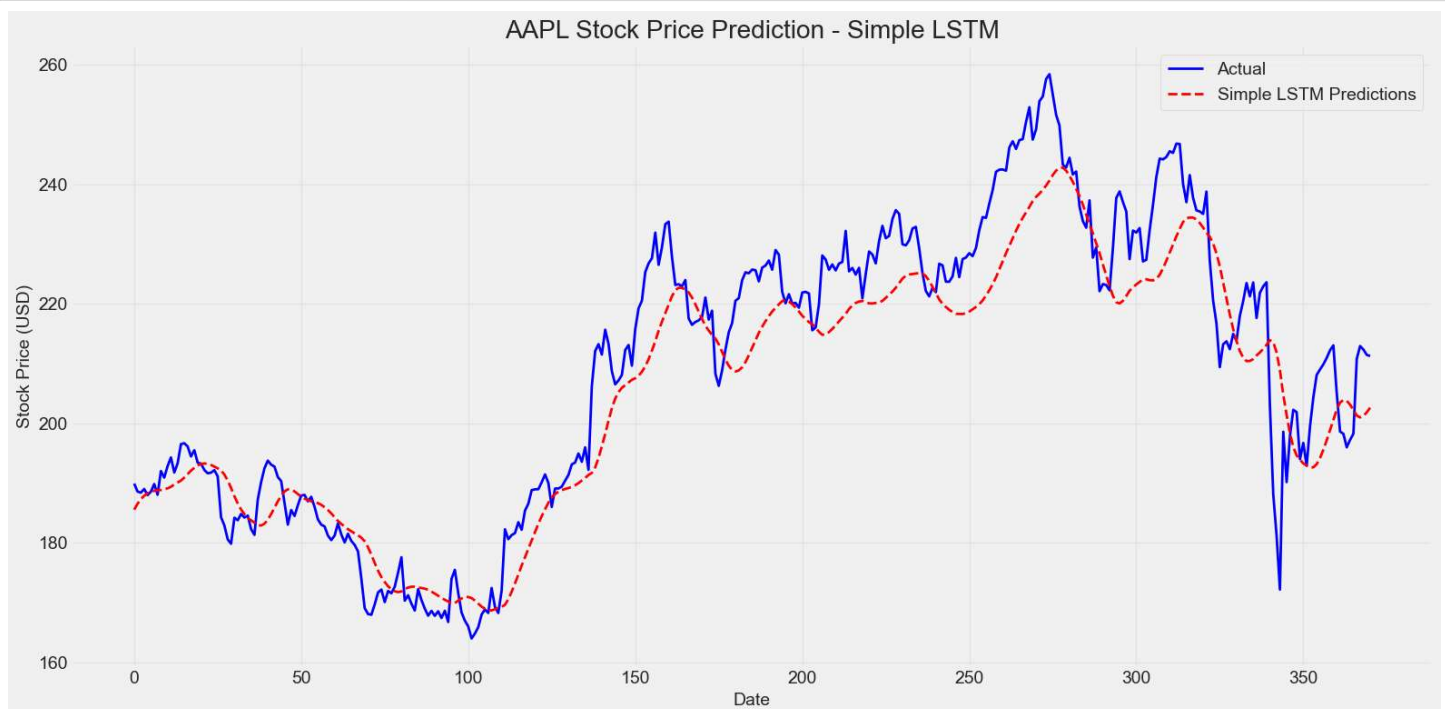
```
# Plot the predictions vs actual
def plot_predictions(predictions, actual, model_name, dates=None):
    plt.figure(figsize=(16, 8))

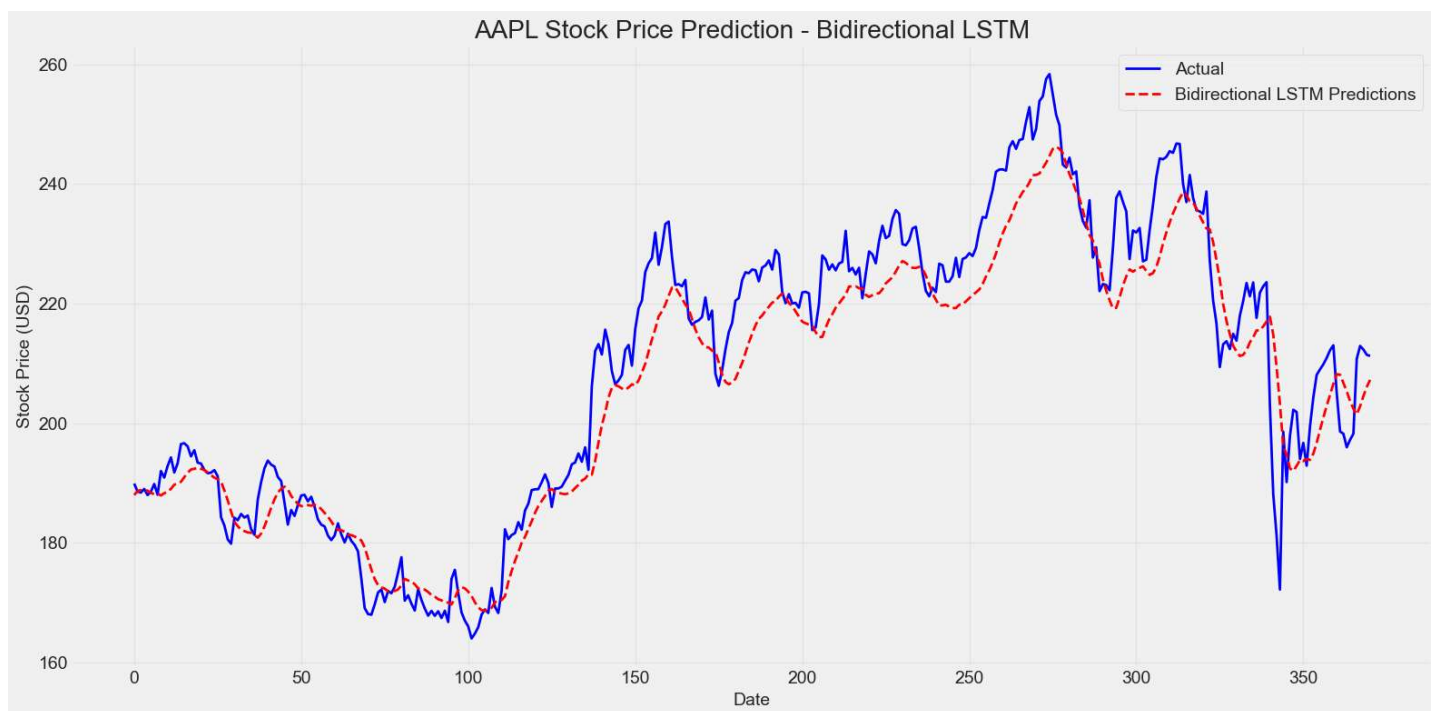
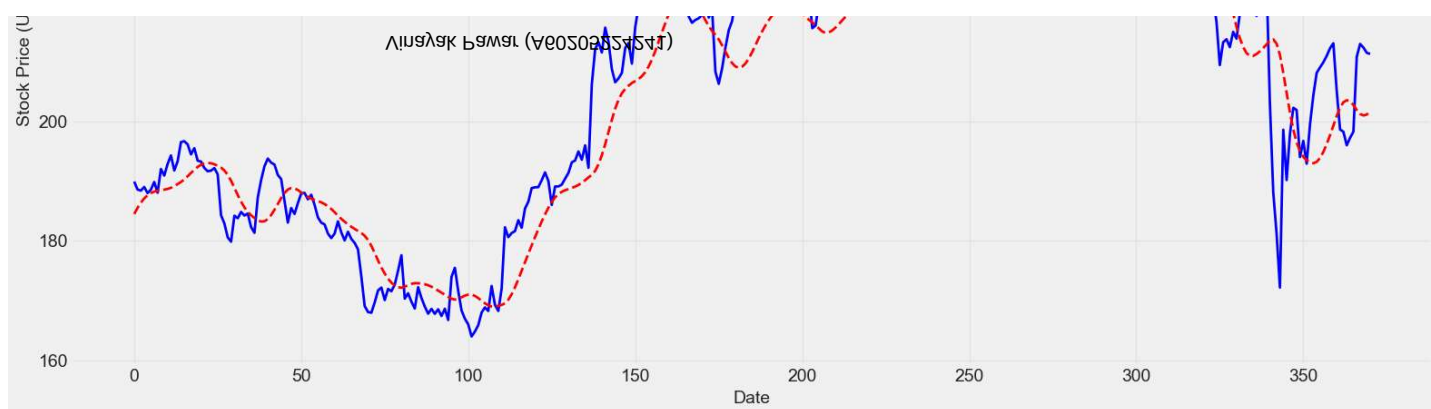
    if dates is not None and len(dates) == len(actual):
        plt.plot(dates, actual, label='Actual', color='blue', linewidth=2)
        plt.plot(dates, predictions, label=f'{model_name} Predictions', color='red', linestyle='--', linewidth=2)
    else:
        plt.plot(actual, label='Actual', color='blue', linewidth=2)
        plt.plot(predictions, label=f'{model_name} Predictions', color='red', linestyle='--', linewidth=2)

    plt.title(f'{selected_stock} Stock Price Prediction - {model_name}')
    plt.xlabel('Date', fontsize=14)
    plt.ylabel('Stock Price (USD)', fontsize=14)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

# Extract dates for test data
test_dates_for_plot = df['Date'].iloc[train_size + sequence_length:train_size + sequence_length + len(y_test_inv)]

# Plot predictions for each model
plot_predictions(simple_pred, y_test_inv, "Simple LSTM", test_dates_for_plot)
plot_predictions(stacked_pred, y_test_inv, "Stacked LSTM", test_dates_for_plot)
plot_predictions(bidirectional_pred, y_test_inv, "Bidirectional LSTM", test_dates_for_plot)
```



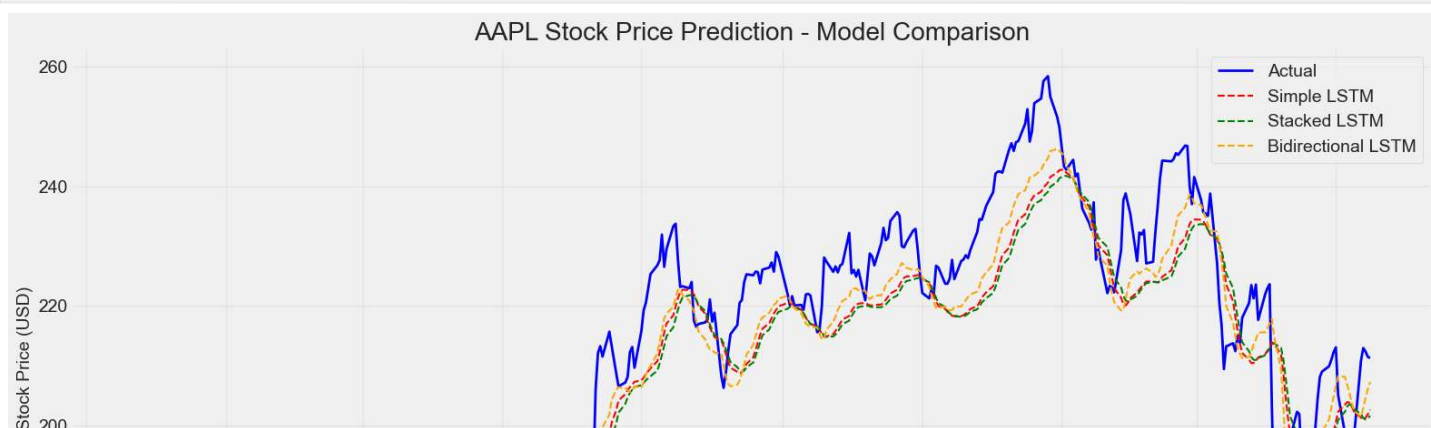


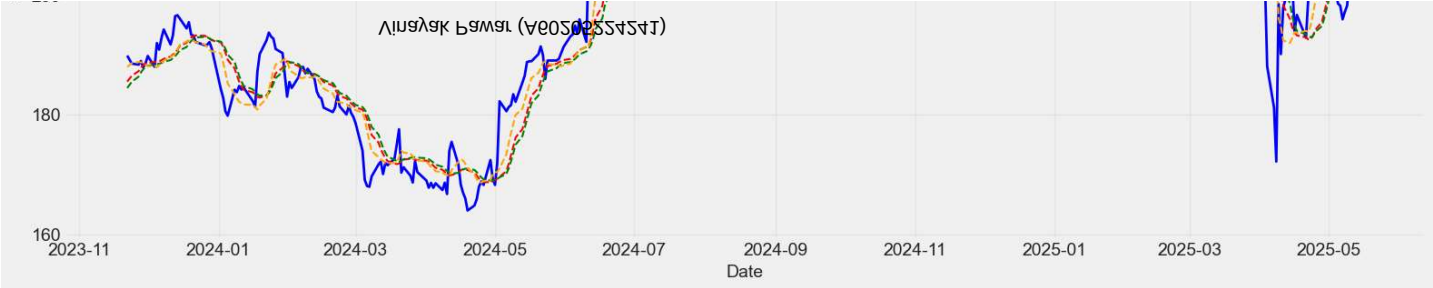
In [15]:

```
# Plot all predictions on the same graph for comparison
plt.figure(figsize=(16, 8))

plt.plot(test_dates, y_test_inv, label='Actual', color='blue', linewidth=2)
plt.plot(test_dates, simple_pred, label='Simple LSTM', color='red', linestyle='--', linewidth=1.5)
plt.plot(test_dates, stacked_pred, label='Stacked LSTM', color='green', linestyle='--', linewidth=1.5)
plt.plot(test_dates, bidirectional_pred, label='Bidirectional LSTM', color='orange', linestyle='--', linewidth=1.5)

plt.title(f'{selected_stock} Stock Price Prediction - Model Comparison')
plt.xlabel('Date', fontsize=14)
plt.ylabel('Stock Price (USD)', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```





In [16]:

```
# Compare error metrics
metrics = pd.DataFrame({
    'Simple LSTM': [simple_metrics['rmse'], simple_metrics['mae'], simple_metrics['r2']]
    ,
    'Stacked LSTM': [stacked_metrics['rmse'], stacked_metrics['mae'], stacked_metrics['r2']],
    'Bidirectional LSTM': [bidirectional_metrics['rmse'], bidirectional_metrics['mae'],
    bidirectional_metrics['r2']]
}, index=['RMSE', 'MAE', 'R² Score'])

# Display the metrics table
print("Model Performance Comparison:")
metrics
```

Model Performance Comparison:

Out[16]:

	Simple LSTM	Stacked LSTM	Bidirectional LSTM
RMSE	8.811947	9.424283	7.381580
MAE	6.841683	7.418596	5.688194
R² Score	0.866637	0.847458	0.906418

In [17]:

```
# Visualize model comparison
metrics_to_plot = ['RMSE', 'MAE']
plt.figure(figsize=(12, 6))

for i, metric in enumerate(metrics_to_plot):
    plt.subplot(1, 2, i+1)
    values = metrics.loc[metric]
    bars = plt.bar(values.index, values.values, color=['red', 'green', 'orange'])
    plt.title(f'Model Comparison: {metric}')
    plt.ylabel(metric)
    plt.xticks(rotation=45)

    # Add value labels on bars
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2., height + 0.1,
            f'{height:.2f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()

# Plot R² scores
plt.figure(figsize=(10, 6))
r2_values = metrics.loc['R² Score']
bars = plt.bar(r2_values.index, r2_values.values, color=['red', 'green', 'orange'])
plt.title('Model Comparison: R² Score')
plt.ylabel('R² Score')
plt.ylim(0, 1)
plt.xticks(rotation=45)

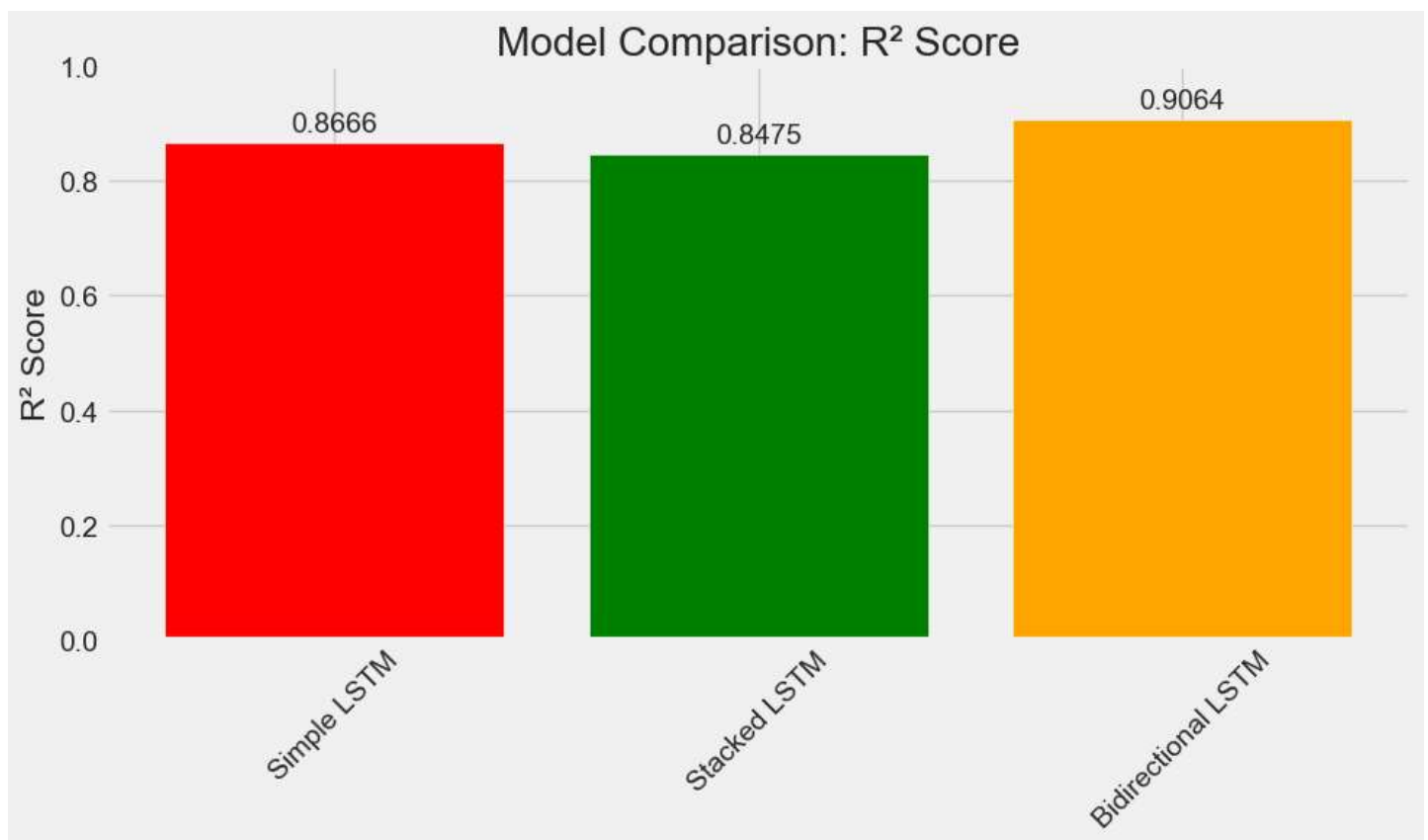
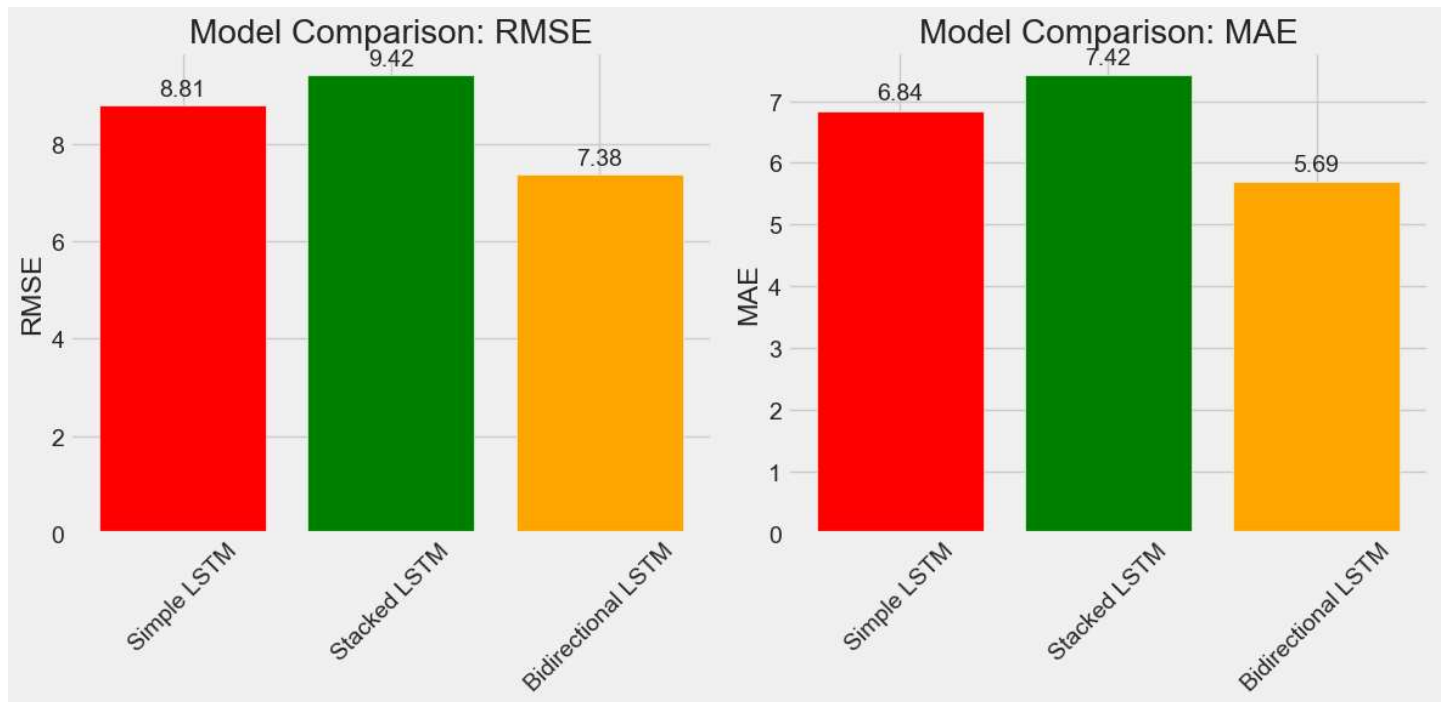
# Add value labels on bars
for bar in bars:
```

```

height = bar.get_height()
plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
         f'{height:.4f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()

```



In []:

```

# Function to predict future stock prices
def predict_future_prices(model, last_sequence, days_to_predict, scaler):
    """Predict future stock prices given the last sequence of data"""
    future_predictions = []
    current_sequence = last_sequence.copy()

    for _ in range(days_to_predict):
        # Reshape for prediction
        current_resaped = current_sequence.reshape(1, current_sequence.shape[0], 1)

        # Predict next price

```

```

next_price = model.predict(current_reshaped)[0][0]
future_predictions.append(next_price)

# Update sequence for next prediction
current_sequence = np.append(current_sequence[1:], next_price)

# Inverse transform to get actual price values
future_predictions = scaler.inverse_transform(
    np.array(future_predictions).reshape(-1, 1)
).flatten()

return future_predictions

# Get the last sequence from our test data
last_sequence = X_test[-1]

# Predict the next 30 days
days_to_predict = 30

# Generate future dates
last_date = test_dates_for_plot.iloc[-1]
future_dates = pd.date_range(start=last_date + timedelta(days=1), periods=days_to_predict, freq='B')

# Predict future prices with all models
simple_future = predict_future_prices(simple_lstm_model, last_sequence, days_to_predict, scaler)
stacked_future = predict_future_prices(stacked_lstm_model, last_sequence, days_to_predict, scaler)
bidirectional_future = predict_future_prices(bidirectional_lstm_model, last_sequence, days_to_predict, scaler)

# Combine historical and future data for visualization
plt.figure(figsize=(16, 8))

# Plot historical actual prices
plt.plot(test_dates_for_plot, y_test_inv[-len(test_dates_for_plot):], label='Historical Prices', color='blue', linewidth=2)

# Plot future predictions
plt.plot(future_dates, simple_future, label='Simple LSTM Forecast', color='red', linestyle='--')
plt.plot(future_dates, stacked_future, label='Stacked LSTM Forecast', color='green', linestyle='--')
plt.plot(future_dates, bidirectional_future, label='Bidirectional LSTM Forecast', color='orange', linestyle='--')

# Add a vertical line to separate historical data from predictions
plt.axvline(x=test_dates_for_plot.iloc[-1], color='gray', linestyle='-.')
plt.text(test_dates_for_plot.iloc[-1], min(y_test_inv), 'Prediction Start', rotation=90, verticalalignment='bottom')

plt.title(f'{selected_stock} Stock Price Forecast - Next {days_to_predict} Trading Days')
plt.xlabel('Date', fontsize=14)
plt.ylabel('Stock Price (USD)', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print the forecasted prices
forecast_df = pd.DataFrame({
    'Date': future_dates,
    'Simple LSTM': simple_future,
    'Stacked LSTM': stacked_future,
    'Bidirectional LSTM': bidirectional_future
})

print("\nForecasted Prices for the Next 30 Trading Days:")
forecast_df.set_index('Date')

```

```

# Add technical indicators to the original dataframe
def add_technical_indicators(df):
    """Add common technical indicators to the dataframe"""
    # Copy of the dataframe
    df_temp = df.copy()

    # Moving Averages
    df_temp['MA5'] = df_temp['Close'].rolling(window=5).mean()
    df_temp['MA10'] = df_temp['Close'].rolling(window=10).mean()
    df_temp['MA20'] = df_temp['Close'].rolling(window=20).mean()
    df_temp['MA50'] = df_temp['Close'].rolling(window=50).mean()

    # Exponential Moving Averages
    df_temp['EMA12'] = df_temp['Close'].ewm(span=12, adjust=False).mean()
    df_temp['EMA26'] = df_temp['Close'].ewm(span=26, adjust=False).mean()

    # MACD (Moving Average Convergence Divergence)
    df_temp['MACD'] = df_temp['EMA12'] - df_temp['EMA26']
    df_temp['MACD_Signal'] = df_temp['MACD'].ewm(span=9, adjust=False).mean()

    # RSI (Relative Strength Index)
    delta = df_temp['Close'].diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.rolling(window=14).mean()
    avg_loss = loss.rolling(window=14).mean()
    rs = avg_gain / avg_loss
    df_temp['RSI'] = 100 - (100 / (1 + rs))

    # Bollinger Bands
    df_temp['MA20_std'] = df_temp['Close'].rolling(window=20).std()
    df_temp['Upper_Band'] = df_temp['MA20'] + (df_temp['MA20_std'] * 2)
    df_temp['Lower_Band'] = df_temp['MA20'] - (df_temp['MA20_std'] * 2)

    # Price Rate of Change
    df_temp['ROC'] = ((df_temp['Close'] / df_temp['Close'].shift(10)) - 1) * 100

    # Drop rows with NaN values resulting from calculations
    df_temp.dropna(inplace=True)

    return df_temp

# Add technical indicators
df_with_indicators = add_technical_indicators(df)

# Display the dataframe with indicators
df_with_indicators.tail()

```

In [23]:

```

# Visualize some technical indicators
def plot_technical_indicators(df, indicators=['MA20', 'Upper_Band', 'Lower_Band', 'RSI', 'MACD']):
    """Plot the stock price along with selected technical indicators"""
    # Create figure with subplots
    fig, axes = plt.subplots(3, 1, figsize=(16, 15), sharex=True)

    # Plot price and Bollinger Bands
    df['Close'].plot(ax=axes[0], color='blue', label='Close Price')
    if 'MA20' in indicators:
        df['MA20'].plot(ax=axes[0], color='orange', label='20-day MA')
    if 'Upper_Band' in indicators and 'Lower_Band' in indicators:
        df['Upper_Band'].plot(ax=axes[0], color='red', linestyle='--', label='Upper Bollinger Band')
        df['Lower_Band'].plot(ax=axes[0], color='green', linestyle='--', label='Lower Bollinger Band')

    axes[0].set_title(f'{selected_stock} Stock Price with Bollinger Bands')
    axes[0].set_ylabel('Price')
    axes[0].legend()

```



```

axes[0].grid(True, alpha=0.3)

# Plot RSI
if 'RSI' in indicators:
    df['RSI'].plot(ax=axes[1], color='purple', label='RSI')
    # Add overbought/oversold levels
    axes[1].axhline(y=70, color='red', linestyle='--', alpha=0.5)
    axes[1].axhline(y=30, color='green', linestyle='--', alpha=0.5)
    axes[1].text(df.index[0], 70, 'Overbought', color='red')
    axes[1].text(df.index[0], 30, 'Oversold', color='green')

axes[1].set_title('Relative Strength Index (RSI)')
axes[1].set_ylabel('RSI')
axes[1].grid(True, alpha=0.3)

# Plot MACD
if 'MACD' in indicators:
    df['MACD'].plot(ax=axes[2], color='blue', label='MACD')
    df['MACD_Signal'].plot(ax=axes[2], color='red', label='Signal Line')

    # Highlight MACD Histogram
    macd_hist = df['MACD'] - df['MACD_Signal']
    axes[2].bar(df.index, macd_hist, color=macd_hist.apply(lambda x: 'green' if x >
0 else 'red'),
                label='MACD Histogram', alpha=0.5, width=2)

axes[2].set_title('Moving Average Convergence Divergence (MACD)')
axes[2].set_ylabel('MACD')
axes[2].axhline(y=0, color='black', linestyle='-', alpha=0.3)
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# First need to recreate the df with indicators since it appears to be empty
df_with_indicators = add_technical_indicators(df)

# Check if we have data
print(f"Technical indicators dataframe shape: {df_with_indicators.shape}")

# Plot technical indicators for the last 200 days
if not df_with_indicators.empty:
    plot_technical_indicators(df_with_indicators.iloc[-200:])
else:
    print("Error: The dataframe with technical indicators is empty.")

```

Technical indicators dataframe shape: (0, 21)
Error: The dataframe with technical indicators is empty.

10. Monte Carlo Simulation for Risk Assessment

In [24]:

```

# Function for Monte Carlo simulation
def monte_carlo_simulation(last_price, days, simulations, volatility):
    """Perform Monte Carlo simulation for stock price predictions"""
    # Calculate daily returns
    returns = np.random.normal(0, volatility, (days, simulations))

    # Create price paths
    price_paths = np.zeros((days + 1, simulations))
    price_paths[0] = last_price

    # Calculate price paths
    for t in range(1, days + 1):
        price_paths[t] = price_paths[t-1] * np.exp(returns[t-1])

    return price_paths

```

```

# Get the last known price
last_price = y_test_inv[-1]

# Calculate historical volatility (standard deviation of returns)
returns = df['Daily_Return'].dropna() / 100 # convert from percentage
volatility = returns.std()

# Monte Carlo parameters
days_to_simulate = 30
num_simulations = 1000

# Run simulation
simulated_paths = monte_carlo_simulation(last_price, days_to_simulate, num_simulations,
volatility)

# Plot simulation results
plt.figure(figsize=(16, 8))

# Plot historical prices
plt.plot(range(-30, 0), y_test_inv[-30:], label='Historical Prices', color='blue', linewidth=2)

# Plot simulation paths (sample 100 to avoid overcrowding)
sample_paths = np.random.choice(num_simulations, 100, replace=False)
for i in sample_paths:
    plt.plot(range(days_to_simulate), simulated_paths[1:, i], color='gray', alpha=0.2)

# Plot expected path (mean)
mean_path = np.mean(simulated_paths, axis=1)
plt.plot(range(days_to_simulate), mean_path[1:], label='Expected Path', color='red', linewidth=2)

# Plot confidence intervals
percentile_5 = np.percentile(simulated_paths, 5, axis=1)
percentile_95 = np.percentile(simulated_paths, 95, axis=1)
plt.fill_between(range(days_to_simulate), percentile_5[1:], percentile_95[1:], color='red', alpha=0.2, label='90% Confidence Interval')

# Add LSTM predictions for comparison
plt.plot(range(days_to_simulate), bidirectional_future, label='Bidirectional LSTM Forecast', color='orange', linestyle='--', linewidth=2)

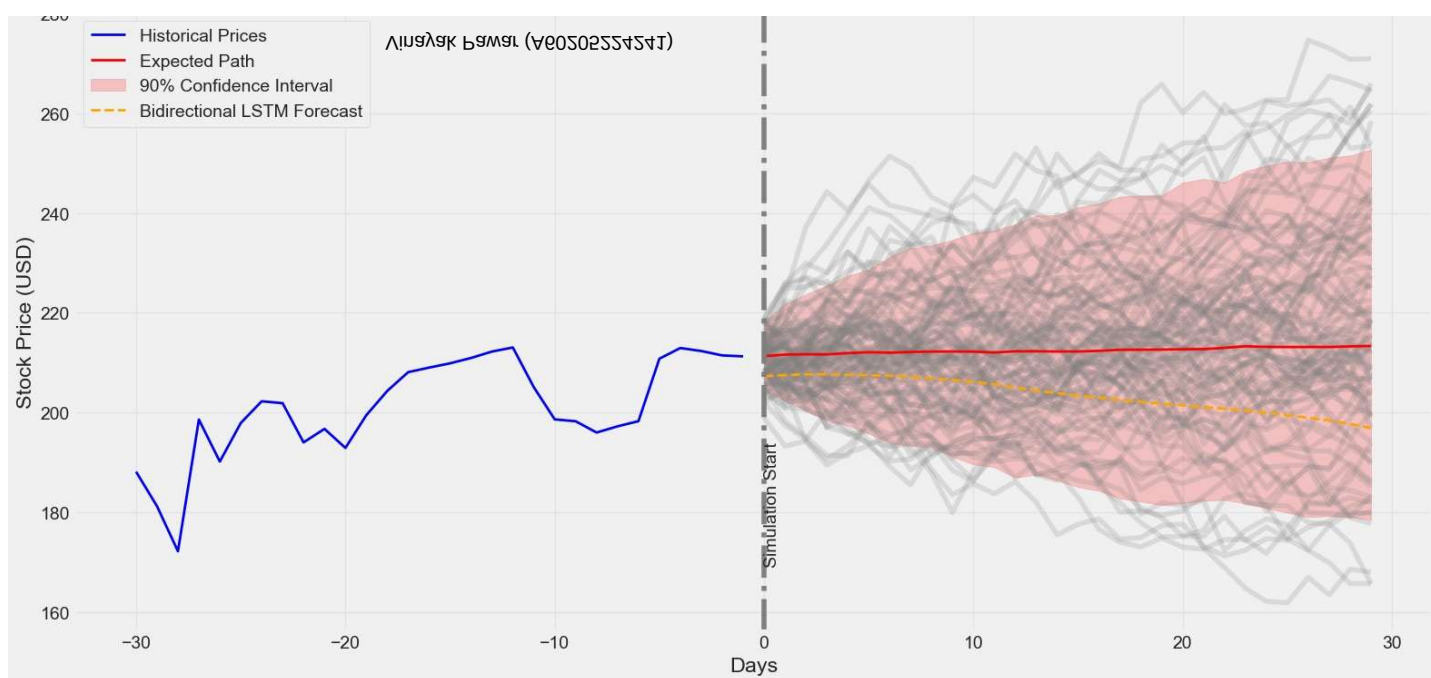
# Add vertical line separating historical from future
plt.axvline(x=0, color='gray', linestyle='-.')
plt.text(0, last_price*0.8, 'Simulation Start', rotation=90, verticalalignment='bottom')

plt.title(f'Monte Carlo Simulation for {selected_stock} (1000 simulations)')
plt.xlabel('Days')
plt.ylabel('Stock Price (USD)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Calculate risk metrics from the simulation
final_prices = simulated_paths[-1]
expected_price = np.mean(final_prices)
var_95 = last_price - np.percentile(final_prices, 5) # 95% Value at Risk
gain_potential = np.percentile(final_prices, 95) - last_price # 95% Upside potential

print(f"\nRisk Assessment for {selected_stock} over {days_to_simulate} days:")
print(f"Current Price: ${last_price:.2f}")
print(f"Expected Price: ${expected_price:.2f} (Change: {(expected_price/last_price-1)*100:.2f}%)")
print(f"95% Value at Risk: ${var_95:.2f} ({var_95/last_price*100:.2f}% of current price)")
print(f"95% Upside Potential: ${gain_potential:.2f} ({gain_potential/last_price*100:.2f}% of current price)")
print(f"90% Confidence Range: ${np.percentile(final_prices, 5):.2f} to ${np.percentile(final_prices, 95):.2f}")

```

Risk Assessment for AAPL over 30 days:

Current Price: \$211.26

Expected Price: \$213.34 (Change: 0.99%)

95% Value at Risk: \$32.76 (15.51% of current price)

95% Upside Potential: \$41.52 (19.65% of current price)

90% Confidence Range: \$178.50 to \$252.78

```
In [2]: class GRUCellNumpy:
        """
        GRU Cell implemented from scratch using NumPy.
        Gates:
        - Update Gate (z): controls how much past info to keep
        - Reset Gate (r): controls how much past info to forget
        - Candidate Hidden State (h_tilde)
        """
        def __init__(self, input_size, hidden_size):
            self.input_size = input_size
            self.hidden_size = hidden_size
            scale = 0.01

            # --- Update Gate weights ---
            self.Wz = np.random.randn(hidden_size, input_size) * scale
            self.Uz = np.random.randn(hidden_size, hidden_size) * scale
            self.bz = np.zeros((hidden_size, 1))

            # --- Reset Gate weights ---
            self.Wr = np.random.randn(hidden_size, input_size) * scale
            self.Ur = np.random.randn(hidden_size, hidden_size) * scale
            self.br = np.zeros((hidden_size, 1))

            # --- Candidate hidden state weights ---
            self.Wh = np.random.randn(hidden_size, input_size) * scale
            self.Uh = np.random.randn(hidden_size, hidden_size) * scale
            self.bh = np.zeros((hidden_size, 1))

        def sigmoid(self, x):
            return 1 / (1 + np.exp(-x))

        def forward(self, x, h_prev):
            """
            x      : input shape (input_size, 1)
            h_prev : previous hidden state shape (hidden_size, 1)
            """
```

```

Returns: h_next (hidden_size, 1)
"""
# Update gate
z = self.sigmoid(self.Wz @ x + self.Uz @ h_prev + self.bz)

# Reset gate
r = self.sigmoid(self.Wr @ x + self.Ur @ h_prev + self.br)

# Candidate hidden state
h_tilde = np.tanh(self.Wh @ x + self.Uh @ (r * h_prev) + self.bh)

# New hidden state
h_next = (1 - z) * h_prev + z * h_tilde

return h_next, z, r, h_tilde

print('GRUCellNumpy class defined!')

```

GRUCellNumpy class defined!

3. Test the NumPy GRU Cell

```

In [3]: np.random.seed(42)

input_size = 3
hidden_size = 5
seq_length = 10

gru_cell = GRUCellNumpy(input_size, hidden_size)

# Random input sequence
X = np.random.randn(seq_length, input_size, 1)

# Initial hidden state
h = np.zeros((hidden_size, 1))

print('Processing sequence through NumPy GRU...')
print(f'Input shape per step : {X[0].shape}')
print(f'Hidden state shape   : {h.shape}\n')

for t in range(seq_length):
    h, z, r, h_tilde = gru_cell.forward(X[t], h)
    print(f'Step {t+1:2d} | Update gate mean: {z.mean():.4f} | Reset gate mean: {r.mean():.4f}')

print(f'\nFinal hidden state:\n{h.flatten()}')

```

Processing sequence through NumPy GRU...

Input shape per step : (3, 1)

Hidden state shape : (5, 1)

Step	Update gate mean	Reset gate mean	h_next mean
1	0.5011	0.4988	0.0018
2	0.4939	0.5021	0.0030
3	0.4982	0.5011	0.0018
4	0.5003	0.5001	0.0015
5	0.4982	0.5014	-0.0003
6	0.5046	0.4978	-0.0002
7	0.5028	0.4986	0.0006
8	0.5042	0.4978	0.0010
9	0.5012	0.4999	-0.0010
10	0.4963	0.5018	-0.0003

Final hidden state:

[-0.00072057 0.00141453 -0.00192145 -0.00299243 0.00249894]

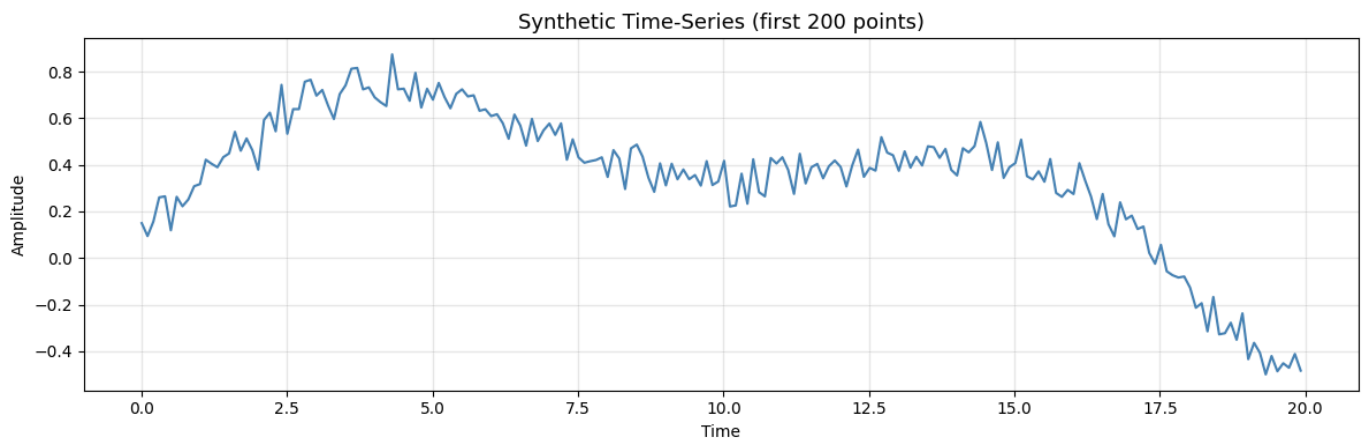
4. Generate Synthetic Time-Series Data

```
In [4]: np.random.seed(0)
torch.manual_seed(0)

# Generate a noisy sine wave
t = np.linspace(0, 100, 1000)
signal = np.sin(0.2 * t) + 0.5 * np.sin(0.5 * t) + 0.1 * np.random.randn(len(t))

# Normalize
scaler = MinMaxScaler(feature_range=(-1, 1))
signal_scaled = scaler.fit_transform(signal.reshape(-1, 1)).flatten()

plt.figure(figsize=(12, 4))
plt.plot(t[:200], signal_scaled[:200], color='steelblue', linewidth=1.5)
plt.title('Synthetic Time-Series (first 200 points)', fontsize=13)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print(f'Total data points: {len(signal_scaled)}')
```



Total data points: 1000

5. Prepare Sequences for PyTorch GRU

```
In [5]: def create_sequences(data, seq_len):
        xs, ys = [], []
        for i in range(len(data) - seq_len):
            xs.append(data[i : i + seq_len])
            ys.append(data[i + seq_len])
        return np.array(xs), np.array(ys)

SEQ_LEN = 30
TRAIN_SPLIT = 0.8

X_all, y_all = create_sequences(signal_scaled, SEQ_LEN)

split = int(len(X_all) * TRAIN_SPLIT)
X_train, X_test = X_all[:split], X_all[split:]
y_train, y_test = y_all[:split], y_all[split:]

# Convert to tensors: shape (batch, seq_len, features)
X_train_t = torch.tensor(X_train, dtype=torch.float32).unsqueeze(-1)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(-1)
X_test_t = torch.tensor(X_test, dtype=torch.float32).unsqueeze(-1)
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(-1)
```

```
print(f'Train samples : {X_train_t.shape}')
print(f'Test samples : {X_test_t.shape}')
```

```
Train samples : torch.Size([776, 30, 1])
```

```
Test samples : torch.Size([194, 30, 1])
```

6. Define PyTorch GRU Model

```
In [6]: class GRUModel(nn.Module):
        def __init__(self, input_size=1, hidden_size=64, num_layers=2, output_size=1, dropout=0.2):
            super(GRUModel, self).__init__()
            self.hidden_size = hidden_size
            self.num_layers = num_layers

            self.gru = nn.GRU(
                input_size = input_size,
                hidden_size = hidden_size,
                num_layers = num_layers,
                batch_first = True,
                dropout = dropout if num_layers > 1 else 0
            )
            self.fc = nn.Linear(hidden_size, output_size)

        def forward(self, x):
            h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
            out, _ = self.gru(x, h0) # out: (batch, seq_len, hidden)
            out = self.fc(out[:, -1, :]) # take last time step
            return out

model = GRUModel()
print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f'\nTotal trainable parameters: {total_params:,}')
```

```
GRUModel(
  (gru): GRU(1, 64, num_layers=2, batch_first=True, dropout=0.2)
  (fc): Linear(in_features=64, out_features=1, bias=True)
)
```

```
Total trainable parameters: 37,889
```

7. Train the GRU Model

```
In [7]: EPOCHS = 100
        LR = 0.001
        BATCH_SIZE = 32

        criterion = nn.MSELoss()
        optimizer = torch.optim.Adam(model.parameters(), lr=LR)
        scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.5)

        train_losses = []

        model.train()
        for epoch in range(1, EPOCHS + 1):
            perm = torch.randperm(X_train_t.size(0))
            epoch_loss = 0.0
            batches = 0

            for i in range(0, X_train_t.size(0), BATCH_SIZE):
                idx = perm[i : i + BATCH_SIZE]
                xb, yb = X_train_t[idx], y_train_t[idx]

                optimizer.zero_grad()
```

```

pred = model(xb)
loss = criterion(pred, yb)
loss.backward()
nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()

epoch_loss += loss.item()
batches    += 1

avg_loss = epoch_loss / batches
train_losses.append(avg_loss)
scheduler.step()

if epoch % 10 == 0:
    print(f'Epoch [{epoch:3d}/{EPOCHS}]  Loss: {avg_loss:.6f}  LR: {scheduler.get_last_lr}')

print('\nTraining complete!')

```

```

Epoch [ 10/100]  Loss: 0.006181  LR: 0.001000
Epoch [ 20/100]  Loss: 0.005370  LR: 0.001000
Epoch [ 30/100]  Loss: 0.005181  LR: 0.000500
Epoch [ 40/100]  Loss: 0.005088  LR: 0.000500
Epoch [ 50/100]  Loss: 0.004898  LR: 0.000500
Epoch [ 60/100]  Loss: 0.005261  LR: 0.000250
Epoch [ 70/100]  Loss: 0.004777  LR: 0.000250
Epoch [ 80/100]  Loss: 0.004707  LR: 0.000250
Epoch [ 90/100]  Loss: 0.004858  LR: 0.000125
Epoch [100/100]  Loss: 0.004602  LR: 0.000125

```

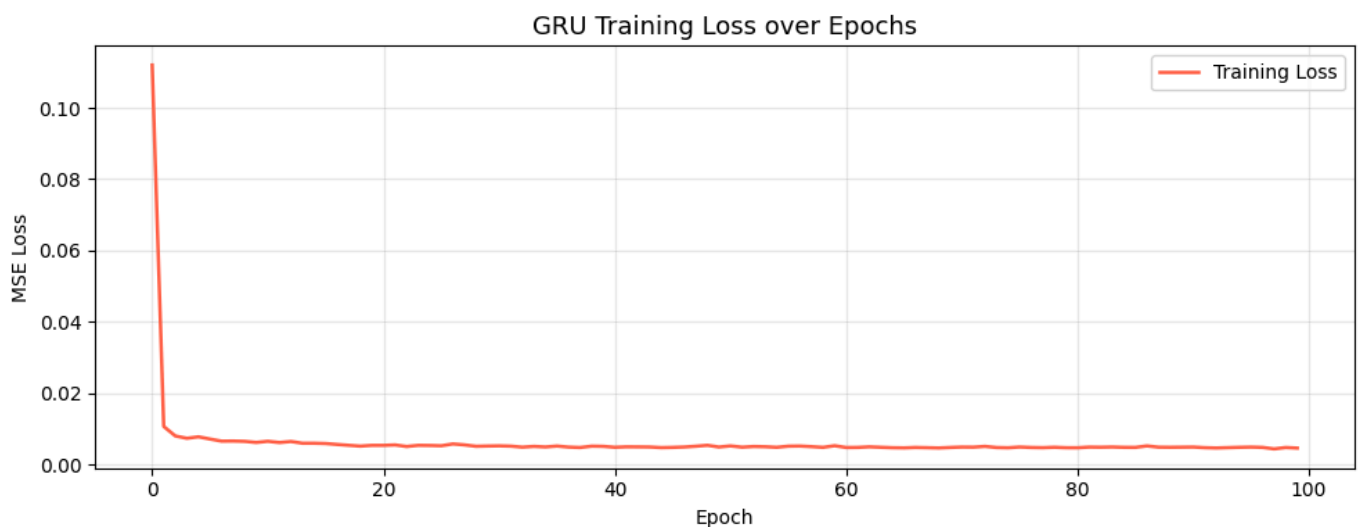
Training complete!

8. Plot Training Loss

```

In [8]: plt.figure(figsize=(10, 4))
plt.plot(train_losses, color='tomato', linewidth=1.8, label='Training Loss')
plt.title('GRU Training Loss over Epochs', fontsize=13)
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



9. Evaluate & Visualize Predictions

```

In [9]: model.eval()
        with torch.no_grad():
            train_pred = model(X_train_t).numpy()
            test_pred = model(X_test_t).numpy()

        # Inverse transform
        train_pred_inv = scaler.inverse_transform(train_pred)
        test_pred_inv = scaler.inverse_transform(test_pred)
        y_train_inv = scaler.inverse_transform(y_train_t.numpy())
        y_test_inv = scaler.inverse_transform(y_test_t.numpy())

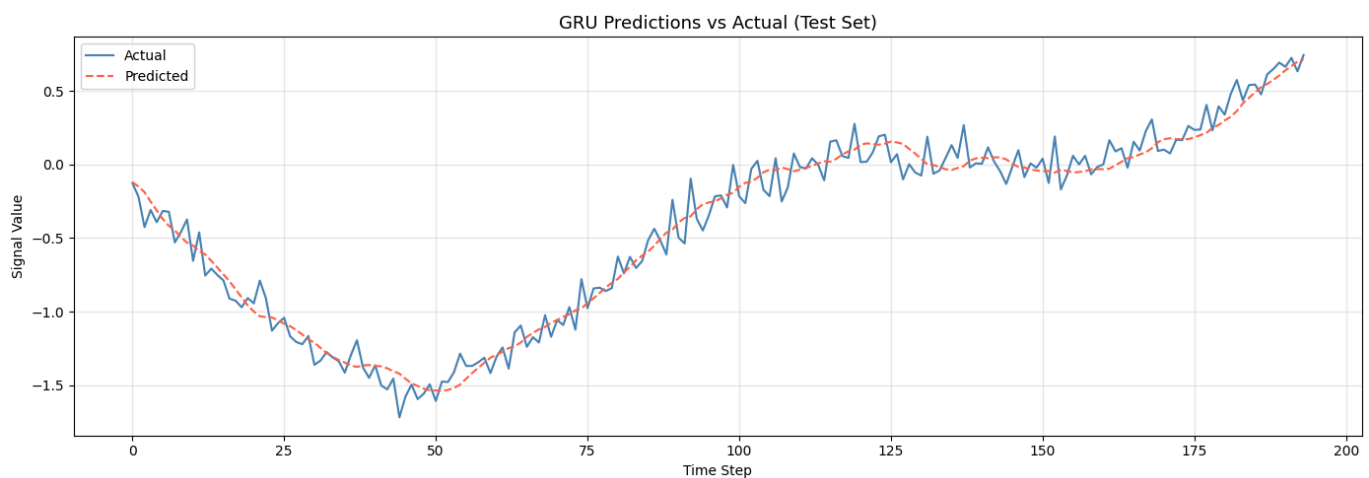
        # MSE on original scale
        train_mse = np.mean((train_pred_inv - y_train_inv) ** 2)
        test_mse = np.mean((test_pred_inv - y_test_inv) ** 2)
        print(f'Train MSE: {train_mse:.6f}')
        print(f'Test MSE: {test_mse:.6f}')

        # Plot
        plt.figure(figsize=(14, 5))
        plt.plot(y_test_inv, label='Actual', color='steelblue', linewidth=1.5)
        plt.plot(test_pred_inv, label='Predicted', color='tomato', linewidth=1.5, linestyle='--')
        plt.title('GRU Predictions vs Actual (Test Set)', fontsize=13)
        plt.xlabel('Time Step')
        plt.ylabel('Signal Value')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.show()

```

Train MSE: 0.012496

Test MSE: 0.011273



10. Visualize GRU Gate Activations

```

In [10]: # Re-run one sample through NumPy GRU and collect gate values
np.random.seed(99)
demo_input = np.sin(np.linspace(0, 4 * np.pi, 50))
gru_viz = GRUCellNumpy(input_size=1, hidden_size=8)

h_viz = np.zeros((8, 1))
update_gates, reset_gates = [], []

for val in demo_input:
    x_in = np.array([[val]])
    h_viz, z, r, _ = gru_viz.forward(x_in, h_viz)
    update_gates.append(z.mean())
    reset_gates.append(r.mean())

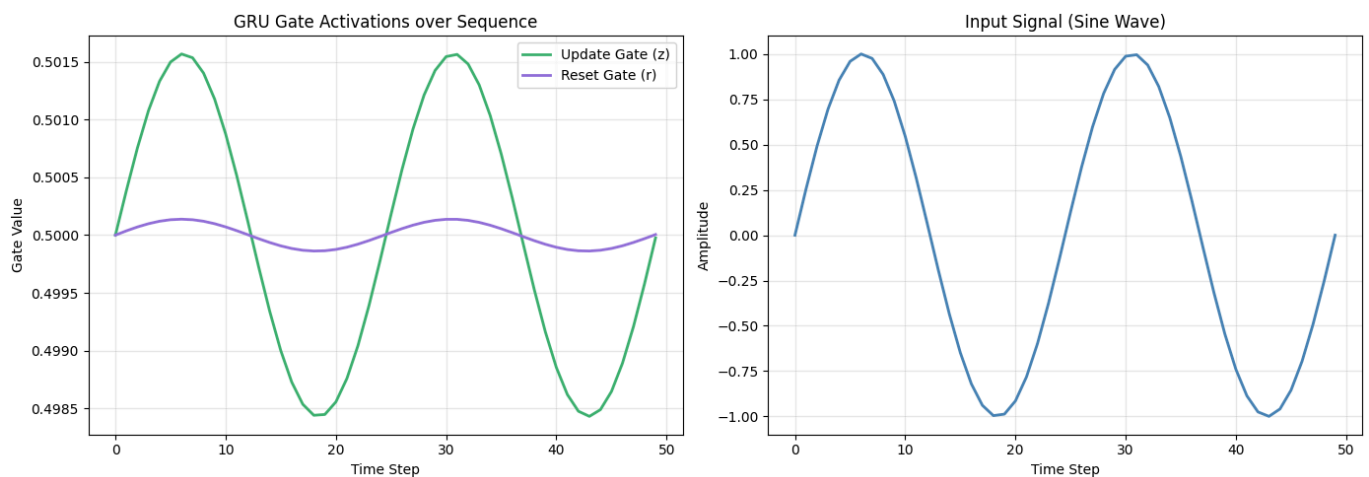
plt.figure(figsize=(14, 5))

```

```
plt.subplot(1, 2, 1)
plt.plot(update_gates, color='mediumseagreen', linewidth=2, label='Update Gate (z)')
plt.plot(reset_gates, color='mediumpurple', linewidth=2, label='Reset Gate (r)')
plt.title('GRU Gate Activations over Sequence', fontsize=12)
plt.xlabel('Time Step')
plt.ylabel('Gate Value')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(demo_input, color='steelblue', linewidth=2)
plt.title('Input Signal (Sine Wave)', fontsize=12)
plt.xlabel('Time Step')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



11. Save & Load Model

```
In [11]: # Save model weights
torch.save(model.state_dict(), 'gru_model.pth')
print('Model saved to gru_model.pth')

# Load model
loaded_model = GRUModel()
loaded_model.load_state_dict(torch.load('gru_model.pth'))
loaded_model.eval()
print('Model loaded successfully!')

# Verify predictions match
with torch.no_grad():
    pred_orig = model(X_test_t[:5]).numpy()
    pred_loaded = loaded_model(X_test_t[:5]).numpy()
    print(f'Predictions match: {np.allclose(pred_orig, pred_loaded)}')
```

Model saved to gru_model.pth

Model loaded successfully!

Predictions match: True

Summary

Component	Details
NumPy GRU	Manual gate equations (z , r , $\tilde{h}$)
PyTorch GRU	<code>nn.GRU</code> with 2 layers, hidden=64

Task	Univariate time-series prediction
Optimizer	Adam + StepLR scheduler
Regularization	Gradient clipping + Dropout

Program 6 - U-Net Architecture for Brain Tumor Segmentation (BraTS)

Supports 2D and 3D modes. Output classes: 3 (background, tumor core, whole tumor) or 4 (BraTS: background, NCR/NET, ED, ET)

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import numpy as np
```

1. BUILDING BLOCKS

```
In [2]: class DoubleConv(nn.Module):
    """Two consecutive Conv → BN → ReLU blocks."""
    def __init__(self, in_ch, out_ch, mid_ch=None):
        super().__init__()
        mid_ch = mid_ch or out_ch
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, mid_ch, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(mid_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(mid_ch, out_ch, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        return self.block(x)

class ResDoubleConv(nn.Module):
    """Double conv with residual shortcut (optional but recommended for tumors)."""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = DoubleConv(in_ch, out_ch)
        self.skip = (
            nn.Conv2d(in_ch, out_ch, kernel_size=1, bias=False)
            if in_ch != out_ch else nn.Identity()
        )
        self.bn = nn.BatchNorm2d(out_ch) if in_ch != out_ch else nn.Identity()

    def forward(self, x):
        return F.relu(self.conv(x) + self.bn(self.skip(x)), inplace=True)

class AttentionGate(nn.Module):
    """
    Soft attention gate on skip connections.
    Suppresses irrelevant background features before concatenation.
    """
    def __init__(self, g_ch, x_ch, inter_ch):
        super().__init__()
        self.Wg = nn.Sequential(nn.Conv2d(g_ch, inter_ch, 1, bias=False), nn.BatchNorm2d(inter_ch))
        self.Wx = nn.Sequential(nn.Conv2d(x_ch, inter_ch, 1, bias=False), nn.BatchNorm2d(inter_ch))
        self.psi = nn.Sequential(nn.Conv2d(inter_ch, 1, 1, bias=False), nn.BatchNorm2d(1), nn
```

```

def forward(self, g, x):
    # g = gating signal (from decoder), x = skip feature
    g_up = F.interpolate(self.Wg(g), size=x.shape[2:], mode="bilinear", align_corners=False)
    alpha = self.psi(F.relu(g_up + self.Wx(x), inplace=True))
    return x * alpha

class Down(nn.Module):
    """MaxPool → DoubleConv (encoder step)."""
    def __init__(self, in_ch, out_ch, residual=True):
        super().__init__()
        conv = ResDoubleConv if residual else DoubleConv
        self.pool_conv = nn.Sequential(nn.MaxPool2d(2), conv(in_ch, out_ch))

    def forward(self, x):
        return self.pool_conv(x)

class Up(nn.Module):
    """
    Upsample → concat skip → DoubleConv (decoder step).
    Supports bilinear upsampling or transposed convolution.
    """
    def __init__(self, in_ch, out_ch, bilinear=True, residual=True, attention=True):
        super().__init__()
        self.attention = attention
        conv = ResDoubleConv if residual else DoubleConv

        if bilinear:
            self.up = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=True)
            self.conv = DoubleConv(in_ch, out_ch, in_ch // 2)
        else:
            self.up = nn.ConvTranspose2d(in_ch, in_ch // 2, kernel_size=2, stride=2)
            self.conv = conv(in_ch, out_ch)

        if attention:
            self.att = AttentionGate(in_ch // 2, in_ch // 2, in_ch // 4)

    def forward(self, x, skip):
        x = self.up(x)
        if self.attention:
            skip = self.att(x, skip)
        # Pad if spatial dims don't match exactly
        dy = skip.size(2) - x.size(2)
        dx = skip.size(3) - x.size(3)
        x = F.pad(x, [dx // 2, dx - dx // 2, dy // 2, dy - dy // 2])
        return self.conv(torch.cat([skip, x], dim=1))

class OutConv(nn.Module):
    def __init__(self, in_ch, n_classes):
        super().__init__()
        self.conv = nn.Conv2d(in_ch, n_classes, kernel_size=1)

    def forward(self, x):
        return self.conv(x)

```

2. FULL U-NET MODEL

```

In [3]: class UNet(nn.Module):
        """
        U-Net for brain tumor segmentation.

```

Args:

```

in_channels : number of MRI modalities (4 for BraTS: T1, T1ce, T2, FLAIR)
n_classes   : 4 for BraTS sub-regions (bg, NCR/NET, edema, enhancing)
base_filters : feature maps in first encoder block (default 64)
bilinear     : True = bilinear upsample, False = transposed conv
residual     : use residual double-conv blocks
attention    : use attention gates on skip connections
deep_supervision: return auxiliary outputs from deeper decoder levels
"""

def __init__(
    self,
    in_channels: int = 4,
    n_classes: int = 4,
    base_filters: int = 64,
    bilinear: bool = True,
    residual: bool = True,
    attention: bool = True,
    deep_supervision: bool = False,
):
    super().__init__()
    self.deep_supervision = deep_supervision
    f = base_filters
    conv = ResDoubleConv if residual else DoubleConv

    # — Encoder —
    self.inc = conv(in_channels, f) # 256x256 → 256x256, 64ch
    self.down1 = Down(f, f * 2, residual) # 128x128, 128ch
    self.down2 = Down(f * 2, f * 4, residual) # 64x64, 256ch
    self.down3 = Down(f * 4, f * 8, residual) # 32x32, 512ch

    # — Bottleneck —
    factor = 2 if bilinear else 1
    self.down4 = Down(f * 8, f * 16 // factor, residual) # 16x16, 1024ch

    # — Decoder —
    self.up1 = Up(f * 16, f * 8 // factor, bilinear, residual, attention)
    self.up2 = Up(f * 8, f * 4 // factor, bilinear, residual, attention)
    self.up3 = Up(f * 4, f * 2 // factor, bilinear, residual, attention)
    self.up4 = Up(f * 2, f, bilinear, residual, attention)

    # — Output heads —
    self.outc = OutConv(f, n_classes)

    if deep_supervision:
        self.ds2 = OutConv(f, n_classes)
        self.ds3 = OutConv(f * 4 // factor, n_classes)

def forward(self, x):
    # Encoder
    x1 = self.inc(x)
    x2 = self.down1(x1)
    x3 = self.down2(x2)
    x4 = self.down3(x3)
    x5 = self.down4(x4)

    # Decoder
    d4 = self.up1(x5, x4)
    d3 = self.up2(d4, x3)
    d2 = self.up3(d3, x2)
    d1 = self.up4(d2, x1)

    logits = self.outc(d1)

    if self.deep_supervision and self.training:
        ds2 = F.interpolate(self.ds2(d2), size=logits.shape[2:], mode="bilinear", align_corners=True)
        ds3 = F.interpolate(self.ds3(d3), size=logits.shape[2:], mode="bilinear", align_corners=True)

```

```
return logits, ds2, ds3 # main + two auxiliary outputs
```

```
return logits
```

3. LOSS FUNCTIONS

```
In [4]: class DiceLoss(nn.Module):
        """
        Soft Dice loss per class, averaged across classes.
        Works well with class imbalance common in tumor segmentation.
        """
        def __init__(self, smooth: float = 1.0):
            super().__init__()
            self.smooth = smooth

        def forward(self, logits, targets):
            probs = torch.softmax(logits, dim=1)
            n_classes = probs.shape[1]
            targets_oh = F.one_hot(targets, n_classes).permute(0, 4, 1, 2, 3).float() \
                if targets.dim() == 4 else \
                F.one_hot(targets, n_classes).permute(0, 3, 1, 2).float()

            dims = tuple(range(2, probs.dim()))
            inter = (probs * targets_oh).sum(dims)
            union = probs.sum(dims) + targets_oh.sum(dims)
            dice = (2 * inter + self.smooth) / (union + self.smooth)
            return 1 - dice.mean()

class CombinedLoss(nn.Module):
    """Dice + Cross-Entropy, weighted. Standard for BraTS."""
    def __init__(self, dice_weight=0.5, ce_weight=0.5, class_weights=None):
        super().__init__()
        self.dice = DiceLoss()
        self.ce = nn.CrossEntropyLoss(weight=class_weights)
        self.dw = dice_weight
        self.cw = ce_weight

    def forward(self, logits, targets):
        return self.dw * self.dice(logits, targets) + self.cw * self.ce(logits, targets)
```

4. METRICS

```
In [5]: def dice_score(preds: torch.Tensor, targets: torch.Tensor, n_classes: int, smooth: float = 1e-6)
        """
        Returns per-class Dice scores as a list.
        preds : (B, H, W) long tensor of predicted class indices
        targets: (B, H, W) long tensor of ground-truth class indices
        """
        scores = []
        for c in range(n_classes):
            p = (preds == c).float()
            t = (targets == c).float()
            inter = (p * t).sum()
            scores.append(((2 * inter + smooth) / (p.sum() + t.sum() + smooth)).item())
        return scores
```

5. DATASET (BraTS-style, NIfTI)

In [6]: **try:**

```
import nibabel as nib
from pathlib import Path

class BraTSDataset(Dataset):
    """
    Expects folder structure:
    root/
        case_001/
            case_001_t1.nii.gz
            case_001_t1ce.nii.gz
            case_001_t2.nii.gz
            case_001_flair.nii.gz
            case_001_seg.nii.gz ← labels
        case_002/
            ...

    Slices each 3-D volume along the axial axis (dim=2).
    Label mapping: 0=bg, 1=NCR/NET, 2=edema, 4=enhancing → remapped to 3
    """
    MODALITIES = ["t1", "t1ce", "t2", "flair"]

    def __init__(self, root: str, slice_axis: int = 2, transform=None):
        self.root = Path(root)
        self.transform = transform
        self.axis = slice_axis
        self.slices = [] # List of (case_dir, slice_idx)

        for case in sorted(self.root.iterdir()):
            if not case.is_dir():
                continue
            seg_path = next(case.glob("*_seg.nii.gz"), None)
            if seg_path is None:
                continue
            seg = nib.load(str(seg_path)).get_fdata()
            n_slices = seg.shape[slice_axis]
            for s in range(n_slices):
                self.slices.append((case, s))

    def _load_volume(self, path):
        return nib.load(str(path)).get_fdata().astype(np.float32)

    def _get_slice(self, vol, idx):
        return np.take(vol, idx, axis=self.axis)

    @staticmethod
    def normalize(arr):
        mn, mx = arr.min(), arr.max()
        return (arr - mn) / (mx - mn + 1e-8)

    @staticmethod
    def remap_labels(seg):
        """BraTS label 4 → 3 for compact class indexing."""
        out = seg.copy()
        out[seg == 4] = 3
        return out.astype(np.int64)

    def __len__(self):
        return len(self.slices)

    def __getitem__(self, idx):
        case, sl = self.slices[idx]
        name = case.name

        # Stack 4 modalities into (4, H, W)
```

```

mods = []
for mod in self.MODALITIES:
    vol = self._load_volume(next(case.glob(f"*_{mod}.nii.gz")))
    mods.append(self.normalize(self._get_slice(vol, sl)))
image = np.stack(mods, axis=0) # (4, H, W)

# Labels
seg_vol = self._load_volume(next(case.glob("*_seg.nii.gz")))
label = self.remap_labels(self._get_slice(seg_vol, sl))

if self.transform:
    # albumentations expects HWC; adapt accordingly
    aug = self.transform(image=image.transpose(1, 2, 0), mask=label)
    image = aug["image"].transpose(2, 0, 1)
    label = aug["mask"]

    return torch.tensor(image, dtype=torch.float32), torch.tensor(label, dtype=torch.

except ImportError:
    BraTSDataset = None # nibabel not installed; skip dataset class

```

6. TRAINING LOOP

```

In [7]: def train_one_epoch(model, loader, optimizer, criterion, device, deep_supervision=False):
    model.train()
    total_loss = 0.0

    for images, masks in loader:
        images, masks = images.to(device), masks.to(device)
        optimizer.zero_grad()

        if deep_supervision:
            out_main, out_ds2, out_ds3 = model(images)
            loss = (
                0.6 * criterion(out_main, masks)
                + 0.2 * criterion(out_ds2, masks)
                + 0.2 * criterion(out_ds3, masks)
            )
        else:
            out_main = model(images)
            loss = criterion(out_main, masks)

        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        total_loss += loss.item()

    return total_loss / len(loader)

@torch.no_grad()
def validate(model, loader, criterion, device, n_classes):
    model.eval()
    total_loss = 0.0
    all_dice = [0.0] * n_classes

    for images, masks in loader:
        images, masks = images.to(device), masks.to(device)
        logits = model(images)
        total_loss += criterion(logits, masks).item()

    preds = logits.argmax(dim=1)
    for c, d in enumerate(dice_score(preds, masks, n_classes)):

```

```

all_dice[c] += d

n = len(loader)
return total_loss / n, [d / n for d in all_dice]

def train(
    data_root: str,
    epochs: int = 50,
    batch_size: int = 8,
    lr: float = 1e-4,
    n_classes: int = 4,
    device: str = "cuda",
    checkpoint_path: str = "unet_best.pth",
):
    device = torch.device(device if torch.cuda.is_available() else "cpu")
    print(f"Training on {device}")

    # — Model —
    model = UNet(
        in_channels=4,
        n_classes=n_classes,
        base_filters=64,
        bilinear=True,
        residual=True,
        attention=True,
        deep_supervision=True,
    ).to(device)

    total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"Trainable parameters: {total_params:,}")

    # — Data —
    if BraTSDataset is None:
        raise ImportError("Install nibabel: pip install nibabel")

    train_ds = BraTSDataset(f"{data_root}/train")
    val_ds = BraTSDataset(f"{data_root}/val")
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, num_workers=4, )
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False, num_workers=4, )

    # — Optimizer & scheduler —
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-5)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
    criterion = CombinedLoss(dice_weight=0.5, ce_weight=0.5)

    best_dice = 0.0
    class_names = ["Background", "NCR/NET", "Edema", "Enhancing"]

    for epoch in range(1, epochs + 1):
        train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device, deep_
        val_loss, val_dice = validate(model, val_loader, criterion, device, n_classes)
        scheduler.step()

        mean_tumor_dice = sum(val_dice[1:]) / (n_classes - 1) # exclude background

        print(f"Epoch {epoch:03d}/{epochs} | "
              f"Train loss: {train_loss:.4f} | Val loss: {val_loss:.4f} | "
              f"Mean tumor Dice: {mean_tumor_dice:.4f}")
        for i, (name, d) in enumerate(zip(class_names, val_dice)):
            print(f" {name}: {d:.4f}")

        if mean_tumor_dice > best_dice:
            best_dice = mean_tumor_dice
            torch.save({"epoch": epoch, "model_state": model.state_dict(),
                       "optimizer_state": optimizer.state_dict(), "best_dice": best_dice}, c

```



```
print(f" ✓ Checkpoint saved (Dice={best_dice:.4f})")
```

```
return model
```

7. INFERENCE

```
In [8]: @torch.no_grad()
def predict_volume(model, volume: np.ndarray, device="cuda", slice_axis=2):
    """
    Run slice-by-slice inference on a 4-modal MRI volume.

    Args:
        model : trained UNet
        volume : numpy array of shape (4, H, W, D) – 4 modalities
        device : 'cuda' or 'cpu'
        slice_axis: axis along which to slice (default axial = 2)

    Returns:
        seg_volume : numpy array of shape (H, W, D) with class indices
    """
    model.eval()
    device = torch.device(device)
    model.to(device)

    n_slices = volume.shape[slice_axis + 1] # +1 for modality dim
    seg = np.zeros(volume.shape[1:], dtype=np.uint8)

    for s in range(n_slices):
        slc = np.take(volume, s, axis=slice_axis + 1) # (4, H, W)
        # Normalize each modality
        slc = np.stack([(m - m.min()) / (m.max() - m.min() + 1e-8) for m in slc])
        t = torch.tensor(slc[None], dtype=torch.float32).to(device) # (1, 4, H, W)
        logits = model(t)
        pred = logits.argmax(dim=1).squeeze(0).cpu().numpy().astype(np.uint8)
        if slice_axis == 2:
            seg[:, :, s] = pred
        elif slice_axis == 1:
            seg[:, s, :] = pred
        else:
            seg[s, :, :] = pred

    # Remap class 3 back to BraTS Label 4
    seg[seg == 3] = 4
    return seg
```

8. QUICK SMOKE TEST

```
In [9]: if __name__ == "__main__":
    print("=== U-Net Brain Tumor Segmentation – Smoke Test ===\n")
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    model = UNet(
        in_channels=4,
        n_classes=4,
        base_filters=32, # smaller for quick test
        bilinear=True,
        residual=True,
        attention=True,
        deep_supervision=True,
    ).to(device)
```

```

x = torch.randn(2, 4, 256, 256).to(device) # batch=2, 4 modalities, 256x256
model.train()
out_main, out_ds2, out_ds3 = model(x)

print(f"Input shape      : {list(x.shape)}")
print(f"Main output      : {list(out_main.shape)} (B, n_classes, H, W)")
print(f"Deep sup output 2: {list(out_ds2.shape)}")
print(f"Deep sup output 3: {list(out_ds3.shape)}")

# Loss test
labels = torch.randint(0, 4, (2, 256, 256)).to(device)
criterion = CombinedLoss()
loss = (
    0.6 * criterion(out_main, labels)
    + 0.2 * criterion(out_ds2, labels)
    + 0.2 * criterion(out_ds3, labels)
)
print(f"\nCombined loss    : {loss.item():.4f}")

# Dice metric test
preds = out_main.argmax(dim=1)
scores = dice_score(preds, labels, n_classes=4)
print(f"Dice per class   : {[f'{s:.4f}' for s in scores]}")

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nTotal parameters : {total_params:,}")
print("\nSmoke test passed ✓")

# To start training:
# train(data_root="/path/to/brats", epochs=100, batch_size=8, lr=1e-4)

```

=== U-Net Brain Tumor Segmentation – Smoke Test ===

```

Input shape      : [2, 4, 256, 256]
Main output      : [2, 4, 256, 256] (B, n_classes, H, W)
Deep sup output 2: [2, 4, 256, 256]
Deep sup output 3: [2, 4, 256, 256]

```

```

Combined loss    : 1.0895
Dice per class   : ['0.3287', '0.2076', '0.1854', '0.2162']

```

```
Total parameters : 4,451,524
```

```
Smoke test passed ✓
```

R-CNN (Region-Based Convolutional Neural Network) Implementation

Object Detection from Scratch using PyTorch

Cell 1 — Install Dependencies

```
In [ ]: # Install required libraries
!pip install torch torchvision opencv-python-headless matplotlib scikit-learn numpy Pillow tqdm
!pip install selective-search # lightweight Python selective search
```

Cell 2 — Imports & Device Setup

```
In [27]: import os
import cv2
import numpy as np
import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from sklearn.svm import LinearSVC
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
import pickle
import warnings
warnings.filterwarnings('ignore')

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')
print(f'PyTorch version: {torch.__version__}')
```

Using device: cpu

PyTorch version: 2.10.0+cpu

Cell 3 — Selective Search Region Proposals

```
In [28]: def selective_search_proposals(image_bgr, max_proposals=2000, mode='fast'):
    """
    Run OpenCV Selective Search to generate region proposals.

    Args:
        image_bgr : BGR image (numpy array from cv2.imread)
        max_proposals: maximum number of proposals to return
        mode : 'fast' or 'quality'

    Returns:
        proposals : list of (x, y, w, h) bounding boxes
    """
    ss = cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()
    ss.setBaseImage(image_bgr)
    if mode == 'fast':
        ss.switchToSelectiveSearchFast()
    else:
```

```

ss.switchToSelectiveSearchQuality()

rects = ss.process() # returns (x, y, w, h)
proposals = rects[:max_proposals].tolist()
print(f'[Selective Search] {len(proposals)} proposals generated')
return proposals

def compute_iou(boxA, boxB):
    """
    Compute Intersection over Union between two boxes.
    Boxes format: (x, y, w, h)
    """
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[0]+boxA[2], boxB[0]+boxB[2])
    yB = min(boxA[1]+boxA[3], boxB[1]+boxB[3])

    inter = max(0, xB - xA) * max(0, yB - yA)
    areaA = boxA[2] * boxA[3]
    areaB = boxB[2] * boxB[3]
    union = areaA + areaB - inter
    return inter / (union + 1e-6)

def label_proposals(proposals, gt_boxes, gt_labels,
                    pos_iou_thresh=0.5, neg_iou_thresh=0.3):
    """
    Assign class labels to proposals based on IoU with ground-truth boxes.

    Returns:
        labeled : list of (proposal, label) - 'background' for negatives
    """
    labeled = []
    for prop in proposals:
        best_iou, best_label = 0.0, 'background'
        for gt_box, gt_label in zip(gt_boxes, gt_labels):
            iou = compute_iou(prop, gt_box)
            if iou > best_iou:
                best_iou = iou
                best_label = gt_label
        if best_iou < neg_iou_thresh:
            best_label = 'background'
        elif best_iou < pos_iou_thresh:
            continue # ignore ambiguous proposals
        labeled.append((prop, best_label))
    return labeled

print('Selective Search utilities defined.')

```

Selective Search utilities defined.

Cell 4 — CNN Feature Extractor (ResNet-50 Backbone)

```

In [29]: class RCNNFeatureExtractor(nn.Module):
    """
    ResNet-50 backbone with the classification head removed.
    Outputs a 2048-dim feature vector per region crop.
    """
    def __init__(self, pretrained=True):
        super().__init__()
        backbone = models.resnet50(pretrained=pretrained)
        # Remove avgpool and fc layers - keep conv layers
        self.features = nn.Sequential(*list(backbone.children())[:-2])
        self.pool = nn.AdaptiveAvgPool2d((1, 1))

```

```

self.feat_dim = 2048

def forward(self, x):
    x = self.features(x) # (B, 2048, H', W')
    x = self.pool(x)     # (B, 2048, 1, 1)
    x = x.flatten(1)     # (B, 2048)
    return x

# Transform applied to each warped region crop (R-CNN warps to 227x227)
RCNN_TRANSFORM = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std =[0.229, 0.224, 0.225]),
])

feature_extractor = RCNNFeatureExtractor(pretrained=True).to(DEVICE)
feature_extractor.eval()
print(f'Feature extractor ready. Output dim: {feature_extractor.feat_dim}')

```

Feature extractor ready. Output dim: 2048

Cell 5 — Region Crop & Feature Extraction Pipeline

```

In [30]: def crop_and_warp(image_rgb, box, context_pad=16):
    """
    Crop a region proposal from the image with optional context padding,
    then convert to PIL Image.
    box: (x, y, w, h)
    """
    x, y, w, h = box
    H, W = image_rgb.shape[:2]
    x1 = max(0, x - context_pad)
    y1 = max(0, y - context_pad)
    x2 = min(W, x + w + context_pad)
    y2 = min(H, y + h + context_pad)
    crop = image_rgb[y1:y2, x1:x2]
    return Image.fromarray(crop)

@torch.no_grad()
def extract_features_batch(image_rgb, proposals, batch_size=64):
    """
    Extract CNN features for all proposals in mini-batches.

    Returns:
        features: numpy array of shape (N, feat_dim)
    """
    all_feats = []
    for i in tqdm(range(0, len(proposals), batch_size), desc='Extracting features'):
        batch_boxes = proposals[i:i+batch_size]
        tensors = []
        for box in batch_boxes:
            crop = crop_and_warp(image_rgb, box)
            t = RCNN_TRANSFORM(crop)
            tensors.append(t)
        batch = torch.stack(tensors).to(DEVICE) # (B, 3, 224, 224)
        feats = feature_extractor(batch)        # (B, 2048)
        all_feats.append(feats.cpu().numpy())
    return np.concatenate(all_feats, axis=0)

print('Feature extraction utilities defined.')

```

Feature extraction utilities defined.

Cell 6 — SVM Classifier (per class)

```
In [31]: from sklearn.svm import LinearSVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

class RCNNClassifier:
    """
    One-vs-rest SVM classifiers for R-CNN.
    One LinearSVC per class (including background).
    """
    def __init__(self, classes):
        self.classes = classes # List of class names
        self.svms = {}
        for cls in classes:
            self.svms[cls] = Pipeline([
                ('scaler', StandardScaler()),
                ('svm', LinearSVC(C=0.001, max_iter=5000))
            ])

    def train(self, features, labels):
        """
        Train one SVM per class.
        features: (N, D), labels: list of class strings
        """
        labels = np.array(labels)
        for cls in self.classes:
            binary_labels = (labels == cls).astype(int)
            print(f' Training SVM for class "{cls}" | positives: {binary_labels.sum()}')
            self.svms[cls].fit(features, binary_labels)
        print('All SVMs trained.')

    def predict(self, features):
        """
        For each proposal, return (predicted_class, confidence_score).
        """
        scores = {}
        for cls in self.classes:
            scores[cls] = self.svms[cls].decision_function(features)
        score_matrix = np.stack([scores[c] for c in self.classes], axis=1)
        best_idx = np.argmax(score_matrix, axis=1)
        predictions = [(self.classes[i], score_matrix[n, i])
                       for n, i in enumerate(best_idx)]
        return predictions

    def save(self, path):
        with open(path, 'wb') as f:
            pickle.dump(self, f)
        print(f'Classifier saved to {path}')

    @staticmethod
    def load(path):
        with open(path, 'rb') as f:
            return pickle.load(f)

print('RCNNClassifier defined.')
```

RCNNClassifier defined.

Cell 7 — Bounding Box Regression

In [32]: `from sklearn.linear_model import Ridge`

```
def xywh_to_targets(proposal, gt_box):
    """
    Compute regression targets (tx, ty, tw, th) as in the R-CNN paper.
    """
    px, py, pw, ph = proposal
    gx, gy, gw, gh = gt_box
    cx_p, cy_p = px + pw/2, py + ph/2
    cx_g, cy_g = gx + gw/2, gy + gh/2
    tx = (cx_g - cx_p) / pw
    ty = (cy_g - cy_p) / ph
    tw = np.log(gw / (pw + 1e-6))
    th = np.log(gh / (ph + 1e-6))
    return np.array([tx, ty, tw, th])

def apply_bbox_deltas(proposal, deltas):
    """
    Apply predicted regression deltas to a proposal box.
    Returns adjusted (x, y, w, h).
    """
    px, py, pw, ph = proposal
    tx, ty, tw, th = deltas
    cx = px + pw/2 + tx * pw
    cy = py + ph/2 + ty * ph
    nw = pw * np.exp(tw)
    nh = ph * np.exp(th)
    return (cx - nw/2, cy - nh/2, nw, nh)

class BBoxRegressor:
    """
    Separate Ridge regressor per class to refine bounding boxes.
    """
    def __init__(self, classes):
        self.regressors = {cls: Ridge(alpha=1.0) for cls in classes}

    def train(self, features_per_class, targets_per_class):
        for cls, feats in features_per_class.items():
            targets = np.stack(targets_per_class[cls])
            print(f' Training bbox regressor for "{cls}" with {len(feats)} samples')
            self.regressors[cls].fit(feats, targets)
        print('All bbox regressors trained.')

    def predict(self, cls, features):
        return self.regressors[cls].predict(features)

print('Bounding box regression utilities defined.')
```

Bounding box regression utilities defined.

Cell 8 — Non-Maximum Suppression (NMS)

```
In [33]: def non_maximum_suppression(boxes, scores, iou_threshold=0.3):
    """
    Greedy NMS to suppress overlapping detections.

    Args:
        boxes          : list of (x, y, w, h)
        scores          : confidence scores (higher = better)
        iou_threshold  : suppress if IoU > threshold
```

```

Returns:
    keep_indices : indices of surviving boxes
"""
if len(boxes) == 0:
    return []

boxes = np.array(boxes, dtype=float)
scores = np.array(scores)
order = scores.argsort()[::-1] # descending confidence
keep = []

while order.size > 0:
    i = order[0]
    keep.append(i)
    rest = order[1:]
    ious = np.array([compute_iou(boxes[i], boxes[j]) for j in rest])
    order = rest[ious <= iou_threshold]

return keep

print('NMS defined.')

```

NMS defined.

Cell 9 — Full R-CNN Inference Pipeline

```

In [34]: def rcnn_detect(image_path, classifier, bbox_regressor,
                        score_thresh=0.5, nms_iou=0.3, max_proposals=500):
    """
    Full R-CNN inference on a single image.

    Steps:
        1. Selective Search -> region proposals
        2. Warp & CNN      -> feature vectors
        3. SVM              -> class + score per proposal
        4. BBox regressor   -> refined boxes
        5. NMS              -> final detections

    Returns:
        detections: list of {'box': (x,y,w,h), 'label': str, 'score': float}
    """
    # Load image
    image_bgr = cv2.imread(image_path)
    image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

    # Step 1: Region Proposals
    proposals = selective_search_proposals(image_bgr, max_proposals=max_proposals)

    # Step 2: Feature Extraction
    features = extract_features_batch(image_rgb, proposals)

    # Step 3: SVM Classification
    predictions = classifier.predict(features)

    # Step 4: BBox Regression + Thresholding
    detections_by_class = {}
    for i, (label, score) in enumerate(predictions):
        if label == 'background' or score < score_thresh:
            continue
        feats = features[i:i+1]
        deltas = bbox_regressor.predict(label, feats)[0]
        refined_box = apply_bbox_deltas(proposals[i], deltas)
        detections_by_class.setdefault(label, {'boxes': [], 'scores': []})
        detections_by_class[label]['boxes'].append(refined_box)
        detections_by_class[label]['scores'].append(score)

```



```
# Step 5: NMS per class
final_detections = []
for label, data in detections_by_class.items():
    keep = non_maximum_suppression(data['boxes'], data['scores'], nms_iou)
    for idx in keep:
        final_detections.append({
            'box': data['boxes'][idx],
            'label': label,
            'score': data['scores'][idx]
        })

print(f'[RCNN] {len(final_detections)} detections after NMS')
return image_rgb, final_detections

print('R-CNN inference pipeline defined.')
```

R-CNN inference pipeline defined.

Cell 10 — Visualise Detections

```
In [35]: COLORS = plt.cm.Set1.colors # 9 distinct colors

def visualize_detections(image_rgb, detections, title='R-CNN Detections'):
    """
    Draw bounding boxes and labels on the image.
    """
    fig, ax = plt.subplots(1, 1, figsize=(12, 8))
    ax.imshow(image_rgb)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.axis('off')

    unique_labels = list({d['label'] for d in detections})
    label_color = {lbl: COLORS[i % len(COLORS)] for i, lbl in enumerate(unique_labels)}

    for det in detections:
        x, y, w, h = det['box']
        lbl, score = det['label'], det['score']
        color = label_color[lbl]
        rect = patches.Rectangle((x, y), w, h,
                                linewidth=2, edgecolor=color, facecolor='none')
        ax.add_patch(rect)
        ax.text(x, y - 5, f'{lbl}: {score:.2f}',
                fontsize=10, color='white',
                bbox=dict(facecolor=color, alpha=0.7, pad=2, edgecolor='none'))

    plt.tight_layout()
    plt.show()
    print(f'Displayed {len(detections)} detections.')

print('Visualization utilities defined.')
```

Visualization utilities defined.

Cell 11 — Toy Training Demo (Synthetic Data)

Because training R-CNN on PASCAL VOC requires ~10 GB of data, this cell demonstrates the training loop using randomly generated features & labels so you can verify the pipeline end-to-end without a dataset download.

```
In [36]: np.random.seed(42)
CLASSES = ['cat', 'dog', 'background']
N_TRAIN = 600 # total fake proposals
```

```

FEAT_DIM = 2048 # matches ResNet-50 output

# Generate random features
fake_features = np.random.randn(N_TRAIN, FEAT_DIM).astype(np.float32)

# Generate random Labels (balanced)
fake_labels = np.array(CLASSES * (N_TRAIN // len(CLASSES)) +
                       CLASSES[:N_TRAIN % len(CLASSES)])
np.random.shuffle(fake_labels)

# ---- Train Classifier ----
print('Training SVM classifiers...')
clf = RCNNClassifier(classes=CLASSES)
clf.train(fake_features, fake_labels)

# ---- Train BBox Regressors ----
print('\nTraining BBox regressors...')
features_per_class = {c: [] for c in CLASSES}
targets_per_class = {c: [] for c in CLASSES}

for i, lbl in enumerate(fake_labels):
    features_per_class[lbl].append(fake_features[i])
    targets_per_class[lbl].append(np.random.randn(4) * 0.1) # small fake deltas

# Convert to arrays
for c in CLASSES:
    features_per_class[c] = np.stack(features_per_class[c])

bbox_reg = BBoxRegressor(classes=CLASSES)
bbox_reg.train(features_per_class, targets_per_class)

print('\n✅ Toy training complete.')

```

```

Training SVM classifiers...
  Training SVM for class "cat" | positives: 200
  Training SVM for class "dog" | positives: 200
  Training SVM for class "background" | positives: 200
All SVMs trained.

```

```

Training BBox regressors...
  Training bbox regressor for "cat" with 200 samples
  Training bbox regressor for "dog" with 200 samples
  Training bbox regressor for "background" with 200 samples
All bbox regressors trained.

```

```

✅ Toy training complete.

```

Cell 12 — Save / Load Trained Models

```

In [37]: import os
os.makedirs('rcnn_weights', exist_ok=True)

# Save
clf.save('rcnn_weights/svm_classifier.pkl')
with open('rcnn_weights/bbox_regressor.pkl', 'wb') as f:
    pickle.dump(bbox_reg, f)
print('Models saved.')

# Load back
loaded_clf = RCNNClassifier.load('rcnn_weights/svm_classifier.pkl')
with open('rcnn_weights/bbox_regressor.pkl', 'rb') as f:
    loaded_bbox_reg = pickle.load(f)
print('Models loaded successfully.')

```

Classifier saved to rcnn_weights/svm_classifier.pkl
Models saved.
Models loaded successfully.

Cell 13 — End-to-End Test on a Sample Image

```
In [39]: import urllib.request

# Download a sample image if you have no local image
SAMPLE_URL = 'https://www.leadingdog.co.uk/cdn/shop/articles/photo-1558788353-f76d92427f16_c73'
SAMPLE_PATH = 'dog.jpg'

if not os.path.exists(SAMPLE_PATH):
    urllib.request.urlretrieve(SAMPLE_URL, SAMPLE_PATH)
    print(f'Downloaded sample image to {SAMPLE_PATH}')

# Run full R-CNN inference
image_rgb, detections = rcnn_detect(
    image_path      = SAMPLE_PATH,
    classifier       = loaded_clf,
    bbox_regressor  = loaded_bbox_reg,
    score_thresh    = 0.0,    # set low for toy demo - real threshold: 0.5
    nms_iou         = 0.3,
    max_proposals   = 100
)

visualize_detections(image_rgb, detections[:10]) # show top-10
```

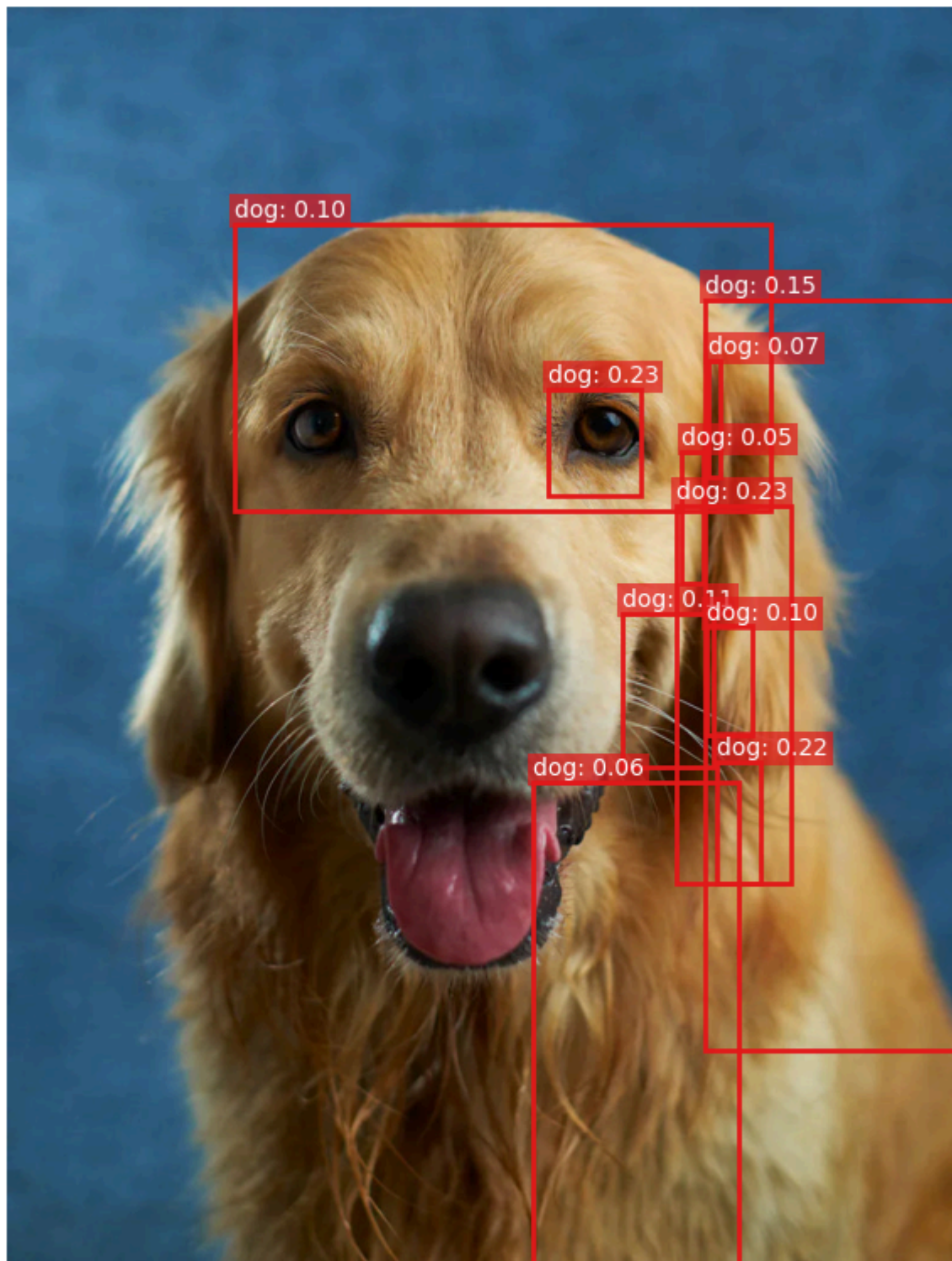
Downloaded sample image to dog.jpg

[Selective Search] 100 proposals generated

Extracting features: 100%|██████████| 2/2 [00:15<00:00, 7.52s/it]

[RCNN] 17 detections after NMS

R-CNN Detections



Displayed 10 detections.

Cell 14 — (Optional) PASCAL VOC Data Loader for Real Training

```
In [17]: # Uncomment & run to download PASCAL VOC 2007 (~870 MB)
!wget http://host.robots.ox.ac.uk/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar
!tar -xf VOCtrainval_06-Nov-2007.tar

import xml.etree.ElementTree as ET

class VOCDataset:
    """
    Minimal PASCAL VOC loader to collect
    (image_path, gt_boxes, gt_labels) triples.
    Set `voc_root` to the extracted VOCdevkit/VOC2007 folder.
    """
```

```

def __init__(self, voc_root, split='train'):
    self.voc_root = voc_root
    img_set_file = os.path.join(voc_root, 'ImageSets', 'Main', f'{split}.txt')
    with open(img_set_file) as f:
        self.ids = [l.strip() for l in f]

def __len__(self):
    return len(self.ids)

def __getitem__(self, idx):
    img_id = self.ids[idx]
    img_path = os.path.join(self.voc_root, 'JPEGImages', f'{img_id}.jpg')
    ann_path = os.path.join(self.voc_root, 'Annotations', f'{img_id}.xml')
    tree = ET.parse(ann_path)
    root = tree.getroot()
    gt_boxes, gt_labels = [], []
    for obj in root.findall('object'):
        label = obj.find('name').text
        bnd = obj.find('bndbox')
        x1 = int(bnd.find('xmin').text)
        y1 = int(bnd.find('ymin').text)
        x2 = int(bnd.find('xmax').text)
        y2 = int(bnd.find('ymax').text)
        gt_boxes.append((x1, y1, x2-x1, y2-y1)) # xywh
        gt_labels.append(label)
    return img_path, gt_boxes, gt_labels

# Example usage:
ds = VOCDataset('VOCdevkit/VOC2007', split='train')
img_path, gt_boxes, gt_labels = ds[0]
print('VOC data loader ready (uncomment download lines to use).')

```

```

--2026-05-29 16:39:20-- http://host.robots.ox.ac.uk/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar
Resolving host.robots.ox.ac.uk (host.robots.ox.ac.uk)... 129.67.94.50
Connecting to host.robots.ox.ac.uk (host.robots.ox.ac.uk)|129.67.94.50|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar [following]
--2026-05-29 16:39:20-- https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar
Resolving www.robots.ox.ac.uk (www.robots.ox.ac.uk)... 129.67.94.2
Connecting to www.robots.ox.ac.uk (www.robots.ox.ac.uk)|129.67.94.2|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://thor.robots.ox.ac.uk/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar [following]
--2026-05-29 16:39:20-- https://thor.robots.ox.ac.uk/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar
Resolving thor.robots.ox.ac.uk (thor.robots.ox.ac.uk)... 129.67.95.98
Connecting to thor.robots.ox.ac.uk (thor.robots.ox.ac.uk)|129.67.95.98|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 460032000 (439M) [application/octet-stream]
Saving to: 'VOCtrainval_06-Nov-2007.tar.1'

VOCtrainval_06-Nov- 100%[=====>] 438.72M  210MB/s in 2.1s

2026-05-29 16:39:22 (210 MB/s) - 'VOCtrainval_06-Nov-2007.tar.1' saved [460032000/460032000]

VOC data loader ready (uncomment download lines to use).

```

Program 8 - YOLO Object Detection — Complete Implementation

1. Installation

```
In [2]: # Install Ultralytics (includes YOLOv8)
!pip install ultralytics opencv-python-headless matplotlib Pillow

# Verify installation
import ultralytics
ultralytics.checks()
```

Ultralytics 8.4.56 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Setup complete ✅ (4 CPUs, 31.4 GB RAM, 6960.4/8062.4 GB disk)

2. Import Libraries

```
In [3]: import cv2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
from pathlib import Path
from IPython.display import display, Image as IPIImage
from ultralytics import YOLO

print("✅ All libraries imported successfully!")
```

✓ All libraries imported successfully!

3. 🤖 Load Pretrained YOLOv8 Model

Model	Size	mAP50-95	Speed (CPU)
yolov8n	Nano	37.3	80ms
yolov8s	Small	44.9	128ms
yolov8m	Medium	50.2	234ms
yolov8l	Large	52.9	375ms
yolov8x	XLarge	53.9	479ms

```
In [4]: # Load YOLOv8 Nano (fast) - change to 'yolov8s.pt', 'yolov8m.pt' for more accuracy
model = YOLO('yolov8n.pt') # Auto-downloads on first run

print(f"✅ Model loaded: {model.info()}")
print(f"📁 Number of classes: {len(model.names)}")
print(f"🏷️ Class names (first 10): {list(model.names.values())[:10]}")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolo
v8n.pt': 100% _____ 6.2MB 75.3MB/s 0.1s
```

YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs

Model loaded: (129, 3157200, 0, 8.8575488)

 Number of classes: 80

```
🔑 Class names (first 10): ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat', 'traffic light']
```

4. Inference on a Single Image

```
In [5]: # ---- Option A: Use a URL image ----
IMAGE_URL = 'https://ultralytics.com/images/bus.jpg'

# Run inference
results = model.predict(source=IMAGE_URL, conf=0.5, save=False)

# Show results
for r in results:
    boxes = r.boxes
    print(f"\n Detections found: {len(boxes)}")
    for i, box in enumerate(boxes):
        cls = int(box.cls[0])
        conf = float(box.conf[0])
        label = model.names[cls]
        xyxy = box.xyxy[0].tolist()
        print(f"  [{i+1}] {label:15s} | Confidence: {conf:.2%} | BBox: [{int(xyxy[0])},{int(xyxy[1])},{int(xyxy[2])},{int(xyxy[3])}]")
```

Downloading https://ultralytics.com/images/bus.jpg to 'bus.jpg': 100% ————— 134.2KB 6.3 MB/s 0.0s

image 1/1 /kaggle/working/bus.jpg: 640x480 3 persons, 1 bus, 73.6ms

Speed: 10.5ms preprocess, 73.6ms inference, 42.2ms postprocess per image at shape (1, 3, 640, 480)

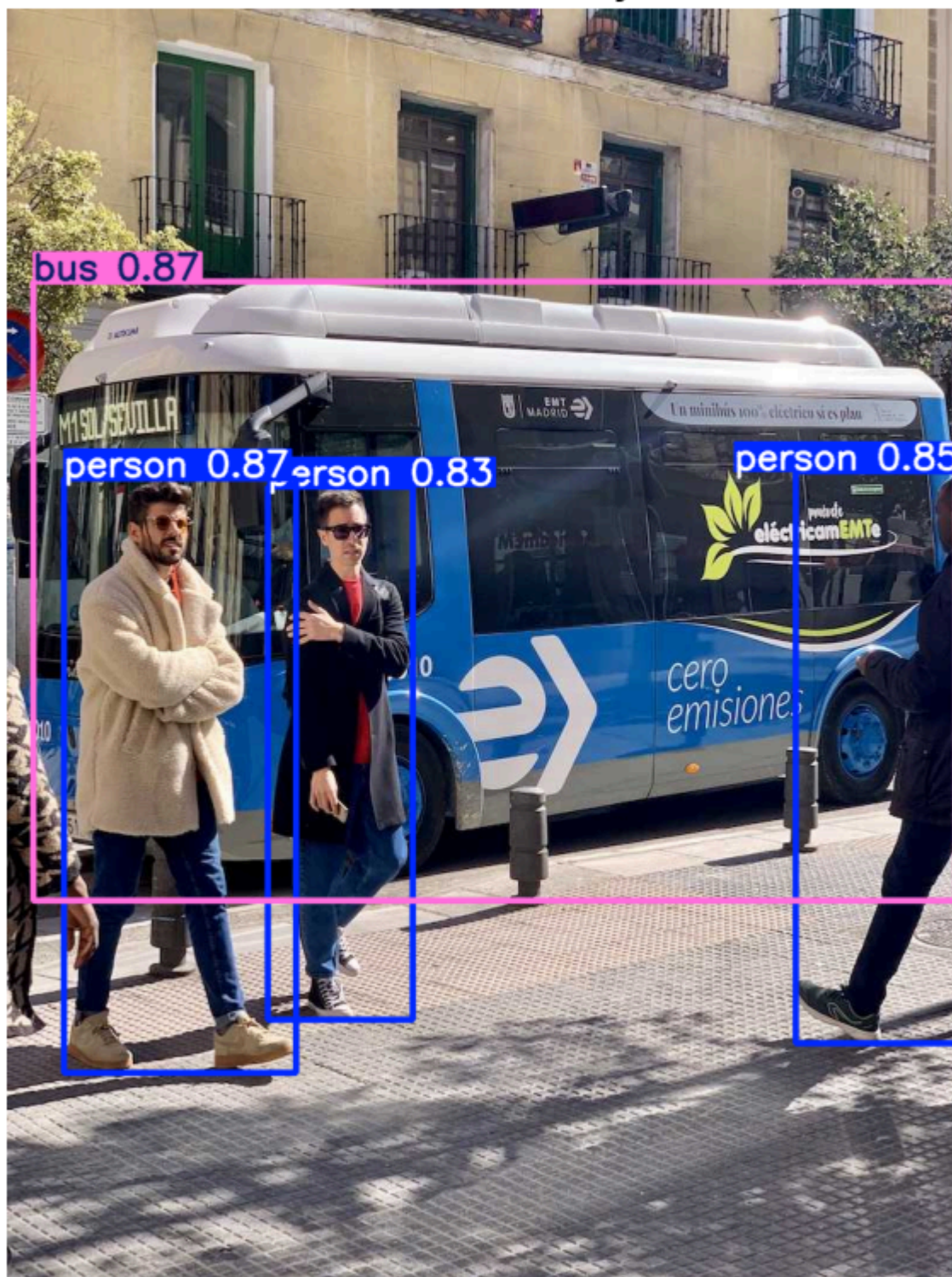
 Detections found: 4

[1]	bus	Confidence: 87.34%	BBox: [22,231,805,756]
[2]	person	Confidence: 86.57%	BBox: [48,398,245,902]
[3]	person	Confidence: 85.28%	BBox: [669,392,809,877]
[4]	person	Confidence: 82.52%	BBox: [221,405,344,857]

```
In [6]: # Visualize detection results
def plot_results(results, figsize=(14, 8)):
    for r in results:
        img_array = r.plot() # BGR numpy array with boxes drawn
        img_rgb = img_array[:, :, ::-1] # Convert BGR → RGB
        plt.figure(figsize=figsize)
        plt.imshow(img_rgb)
        plt.axis('off')
        plt.title(f"YOLOv8 Detection - {len(r.boxes)} objects found", fontsize=14, fontweight='bold')
        plt.tight_layout()
        plt.show()

plot_results(results)
```


YOLOv8 Detection — 4 objects found



5. 📁 Inference on Your Own Local Image

```
In [7]: # ---- Option B: Local file upload ----
# Uncomment and set your image path:
IMAGE_PATH = '/kaggle/input/models/vineetm1204/test/other/default/1/2022-02-04-test_image.jpg'

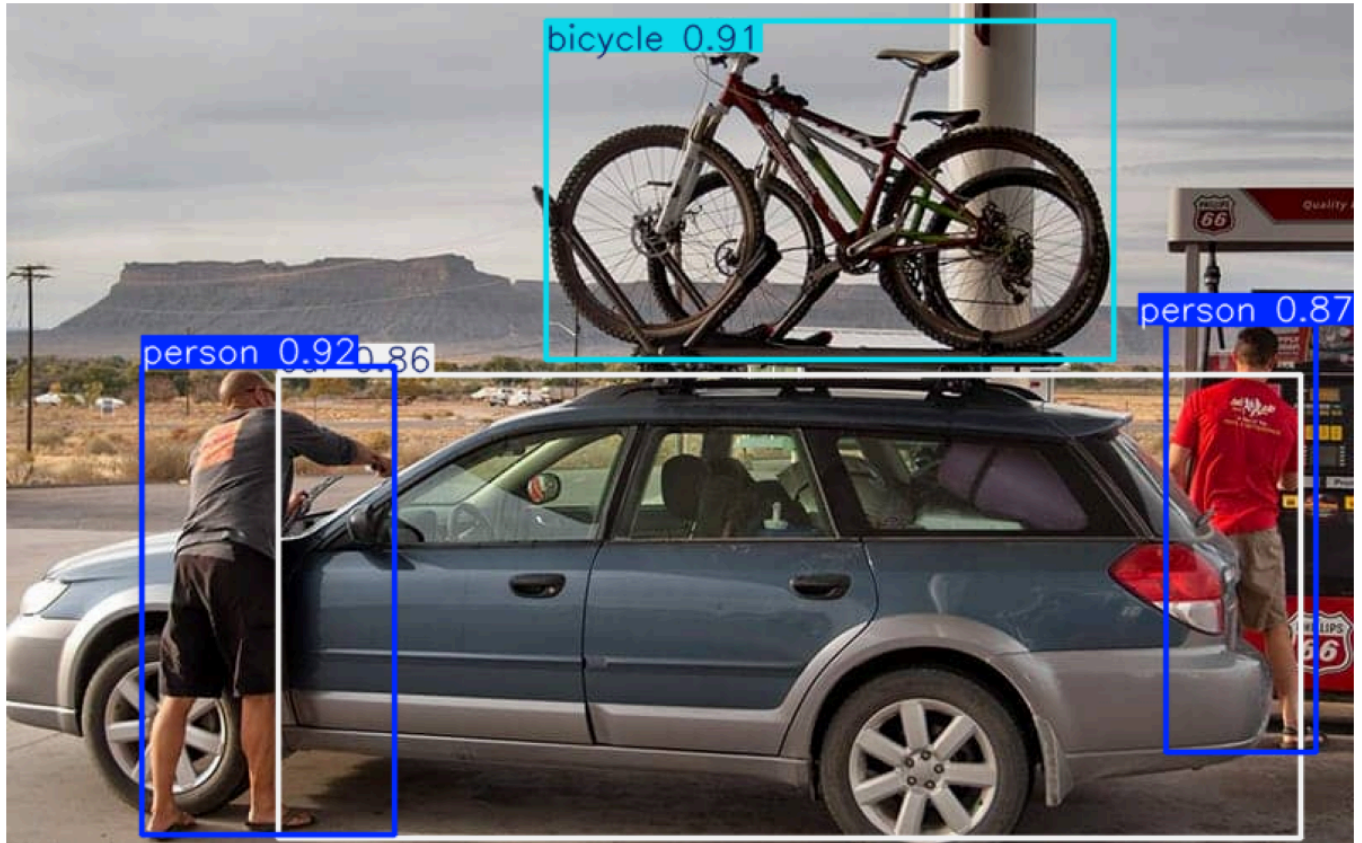
# For Google Colab – upload a file interactively:
# from google.colab import files
# uploaded = files.upload()
# IMAGE_PATH = list(uploaded.keys())[0]

results = model.predict(source=IMAGE_PATH, conf=0.5, save=False)
plot_results(results)

# print("📌 Uncomment the lines above and set your image path to run on a Local image.")
```


image 1/1 /kaggle/input/models/vineetm1204/test/other/default/1/2022-02-04-test_image.jpg: 416x640 2 persons, 1 bicycle, 1 car, 52.2ms
Speed: 2.2ms preprocess, 52.2ms inference, 1.6ms postprocess per image at shape (1, 3, 416, 640)

YOLOv8 Detection — 4 objects found



6. 🎬 Inference on Video

```
In [8]: def run_video_detection(video_path, output_path='output_detected.mp4', conf=0.5):
        """
        Run YOLO detection on a video file and save the output.

        Args:
            video_path : Path to input video
            output_path : Path to save annotated video
            conf        : Confidence threshold
        """
        cap = cv2.VideoCapture(video_path)

        if not cap.isOpened():
            print("❌ Error: Cannot open video file.")
            return

        width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
        fps    = int(cap.get(cv2.CAP_PROP_FPS))
        total  = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

        print(f"📺 Video: {width}x{height} @ {fps}fps | Total frames: {total}")

        fourcc = cv2.VideoWriter_fourcc(*'mp4v')
        out     = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

        frame_count = 0
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break
```

```

results = model.predict(frame, conf=conf, verbose=False)
annotated = results[0].plot()
out.write(annotated)

frame_count += 1
if frame_count % 30 == 0:
    print(f" Processed {frame_count}/{total} frames...", end='\r')

cap.release()
out.release()
print(f"\n✅ Done! Output saved to: {output_path}")

# Usage:
# run_video_detection('your_video.mp4', 'output.mp4', conf=0.5)
print("🔥 Call run_video_detection('your_video.mp4') to process a video.")

```

🔥 Call run_video_detection('your_video.mp4') to process a video.

7. 📷 Webcam / Real-Time Detection (Local Jupyter Only)

```

In [9]: def run_webcam_detection(conf=0.5, device_index=0):
        """
        Real-time YOLO detection using webcam.
        Press 'q' to quit.

        NOTE: Works in local Jupyter only, not in Colab/remote environments.
        """
        cap = cv2.VideoCapture(device_index)
        print("📷 Webcam started. Press 'q' to quit.")

        while True:
            ret, frame = cap.read()
            if not ret:
                break

            results = model.predict(frame, conf=conf, verbose=False)
            annotated = results[0].plot()

            # FPS overlay
            cv2.putText(annotated, f"Objects: {len(results[0].boxes)}",
                        (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

            cv2.imshow('YOLOv8 Real-Time Detection', annotated)

            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

            cap.release()
            cv2.destroyAllWindows()
            print("✅ Webcam closed.")

        # Uncomment to run:
        # run_webcam_detection(conf=0.5)
        print("🔥 Uncomment the last line to start webcam detection.")

```

🔥 Uncomment the last line to start webcam detection.

8. 🏋️ Custom Training on Your Own Dataset

Dataset format expected (YOLO format):

```

dataset/
├── images/
│   ├── train/  ← training images (.jpg/.png)
│   └── val/    ← validation images
└── labels/
    ├── train/  ← .txt annotations (one per image)
    └── val/

```

Each `.txt` label file line: `class_id cx cy width height` (all normalized 0–1)

```

In [10]: # ---- Create a sample dataset YAML ----
import yaml

dataset_config = {
    'path': './dataset',          # Root path of your dataset
    'train': 'images/train',      # Relative path to training images
    'val': 'images/val',         # Relative path to validation images
    'nc': 2,                     # Number of classes
    'names': ['cat', 'dog']      # Class names – customize these!
}

with open('custom_dataset.yaml', 'w') as f:
    yaml.dump(dataset_config, f, default_flow_style=False)

print("✅ Dataset config saved to custom_dataset.yaml")
print("\n📄 Content:")
print(yaml.dump(dataset_config))

```

✅ Dataset config saved to custom_dataset.yaml

📄 Content:

```

names:
- cat
- dog
nc: 2
path: ./dataset
train: images/train
val: images/val

```

```

In [12]: # ---- Training ----
model = YOLO('yolov8n.pt') # Start from pretrained weights (recommended)

results = model.train(
    data      = 'custom_dataset.yaml',
    epochs    = 100,
    imgsz     = 640,
    batch     = 16,
    device    = 0,          # GPU index; use 'cpu' if no GPU
    name      = 'my_yolo_run',
    patience  = 20,         # Early stopping patience
    lr0       = 0.01,       # Initial Learning rate
    workers   = 4
)

print("✅ Training complete! Best weights saved at:")
print(results.save_dir)

# print("📌 Uncomment the training block above after setting up your dataset YAML.")

```

ultralytics 8.4.56  Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugme
nt, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5,
cls_pw=0.0, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutm
ix=0.0, data=custom_dataset.yaml, degrees=0.0, deterministic=True, device=0, dfl=1.5, dnn=Fals
e, dropout=0.0, dynamic=False, embed=None, end2end=None, epochs=100, erasing=0.4, exist_ok=Fal
se, fliplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=
0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, line_width
=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8n.pt,
momentum=0.937, mosaic=1.0, multi_scale=0.0, name=my_yolo_run-2, nbs=64, nms=False, opset=Non
e, optimize=False, optimizer=auto, overlap_mask=True, patience=20, perspective=0.0, plots=Tru
e, pose=12.0, pretrained=True, profile=False, project=None, rect=False, resume=False, retina_m
asks=False, rle=1.0, save=True, save_conf=False, save_crop=False, save_dir=/kaggle/working/run
s/detect/my_yolo_run-2, save_frames=False, save_json=False, save_period=-1, save_txt=False, sc
ale=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, sim
plify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detect, time=N
one, tracker=botsort.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=Fals
e, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=4,
workspace=None

```

FileNotFoundError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/ultralytics/engine/trainer.py in get_dataset(self)
    713         }:
--> 714             data = check_det_dataset(self.args.data)
    715             if "yaml_file" in data:

/usr/local/lib/python3.12/dist-packages/ultralytics/data/utils.py in check_det_dataset(dataset, autownload)
    530             m += f"\nNote dataset download directory is '{DATASETS_DIR}'. You can
update this in '{SETTINGS_FILE}'"
--> 531             raise FileNotFoundError(m)
    532             t = time.time()

```

FileNotFoundError: Dataset 'custom_dataset.yaml' images not found, missing path '/kaggle/working/datasets/dataset/images/val'
Note dataset download directory is '/kaggle/working/datasets'. You can update this in '/root/.config/Ultralytics/settings.json'

The above exception was the direct cause of the following exception:

```

RuntimeError                                    Traceback (most recent call last)
/tmp/ipykernel_58/2860386870.py in <cell line: 0>()
      2 model = YOLO('yolov8n.pt') # Start from pretrained weights (recommended)
      3
----> 4 results = model.train(
      5     data = 'custom_dataset.yaml',
      6     epochs = 100,

/usr/local/lib/python3.12/dist-packages/ultralytics/engine/model.py in train(self, trainer, **kwargs)
    781         args["resume"] = False
    782
--> 783         self.trainer = (trainer or self._smart_load("trainer"))(overrides=args, _callbacks=self.callbacks)
    784         if not args.get("resume") and self.ckpt:
    785             # Reuse the already-loaded checkpoint model to avoid re-resolving remote weights sources during trainer setup.

/usr/local/lib/python3.12/dist-packages/ultralytics/models/yolo/detect/train.py in __init__(self, cfg, overrides, _callbacks)
    61         _callbacks (dict, optional): Dictionary of callback functions to be executed during training.
    62         """
---> 63         super().__init__(cfg, overrides, _callbacks)
    64
    65     def build_dataset(self, img_path: str, mode: str = "train", batch: int | None = None):

/usr/local/lib/python3.12/dist-packages/ultralytics/engine/trainer.py in __init__(self, cfg, overrides, _callbacks)
    184         self.model = check_model_file_from_stem(self.args.model) # add suffix, i.e. yolo26n -> yolo26n.pt
    185         with torch_distributed_zero_first(LOCAL_RANK): # avoid auto-downloading dataset multiple times
--> 186             self.data = self.get_dataset()
    187
    188         self.ema = None

/usr/local/lib/python3.12/dist-packages/ultralytics/engine/trainer.py in get_dataset(self)
    716         self.args.data = data["yaml_file"] # for validating 'yolo train data=url.zip' usage
    717         except Exception as e:
--> 718             raise RuntimeError(emojis(f"Dataset '{clean_url(self.args.data)}' error {e}")) from e

```

719
720

```
if self.args.single_cls:  
    LOGGER.info("Overriding class names with single class.")
```

RuntimeError: Dataset 'custom_dataset.yaml' error ❌ Dataset 'custom_dataset.yaml' images not found, missing path '/kaggle/working/datasets/dataset/images/val'
Note dataset download directory is '/kaggle/working/datasets'. You can update this in '/root/.config/Ultralytics/settings.json'

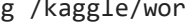
9. Evaluate the Model (mAP, Precision, Recall)

```
In [17]: # Validate on COCO128 (built-in small dataset)  
val_results = model.val(data='coco128.yaml', imgsz=640)  
  
print("\n✅ Validation Metrics:")  
print(f"  mAP50      : {val_results.box.map50:.4f}")  
print(f"  mAP50-95   : {val_results.box.map:.4f}")  
print(f"  Precision   : {val_results.box.mp:.4f}")  
print(f"  Recall      : {val_results.box.mr:.4f}")
```

ultralytics 8.4.36 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

WARNING  Dataset 'coco128.yaml' images not found, missing path '/kaggle/working/datasets/coco128/images/train2017'

Downloading https://ultralytics.com/assets/coco128.zip to '/kaggle/working/datasets/coco128.zip': 100%  6.7MB 83.4MB/s 0.1s

Unzipping /kaggle/working/datasets/coco128.zip to /kaggle/working/datasets/coco128...: 100%  263/263 2.8Kfiles/s 0.1s

Dataset download success  (0.4s), saved to /kaggle/working/datasets

Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100%  755.1KB 16.8MB/s 0.0s

val: Fast image access  (ping: 0.0±0.0 ms, read: 943.0±225.0 MB/s, size: 50.9 KB)

val: Scanning /kaggle/working/datasets/coco128/labels/train2017... 126 images, 2 backgrounds, 0 corrupt: 100%  128/128 1.1Kit/s 0.1s<0.1s

val: New cache created: /kaggle/working/datasets/coco128/labels/train2017.cache

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
	8/8 2.2it/s	3.6s	0.2ss				
	all	128	929	0.639	0.536	0.605	0.445
	person	61	254	0.793	0.677	0.761	0.532
	bicycle	3	6	0.514	0.333	0.314	0.261
	car	12	46	0.813	0.217	0.265	0.162
	motorcycle	4	5	0.687	0.887	0.898	0.717
	airplane	5	6	0.82	0.833	0.927	0.672
	bus	5	7	0.491	0.714	0.724	0.689
	train	3	3	0.534	0.667	0.706	0.601
	truck	5	12	1	0.332	0.469	0.293
	boat	2	6	0.226	0.167	0.315	0.134
	traffic light	4	14	0.734	0.2	0.198	0.136
	stop sign	2	2	1	0.992	0.995	0.7
	bench	5	9	0.839	0.582	0.619	0.364
	bird	2	16	0.921	0.728	0.864	0.508
	cat	4	4	0.875	1	0.995	0.789
	dog	9	9	0.603	0.889	0.785	0.584
	horse	1	2	0.597	1	0.995	0.518
	elephant	4	17	0.849	0.765	0.898	0.675
	bear	1	1	0.593	1	0.995	0.995
	zebra	2	4	0.848	1	0.995	0.964
	giraffe	4	9	0.72	1	0.951	0.717
	backpack	4	6	0.589	0.333	0.375	0.231
	umbrella	4	18	0.804	0.5	0.643	0.416
	handbag	9	19	0.424	0.0526	0.161	0.0858
	tie	6	7	0.804	0.714	0.672	0.473
	suitcase	2	4	0.635	0.883	0.745	0.52
	frisbee	5	5	0.675	0.8	0.755	0.675
	skis	1	1	0.567	1	0.995	0.497
	snowboard	2	7	0.742	0.714	0.747	0.499
	sports ball	6	6	0.716	0.432	0.523	0.311
	kite	2	10	0.727	0.4	0.545	0.177
	baseball bat	4	4	0.555	0.25	0.352	0.198
	baseball glove	4	7	0.651	0.429	0.425	0.27
	skateboard	3	5	0.846	0.6	0.595	0.406
	tennis racket	5	7	0.726	0.387	0.483	0.313
	bottle	6	18	0.448	0.389	0.377	0.203
	wine glass	5	16	0.743	0.362	0.579	0.328
	cup	10	36	0.58	0.278	0.396	0.278
	fork	6	6	0.527	0.167	0.243	0.182
	knife	7	16	0.564	0.5	0.61	0.368
	spoon	5	22	0.597	0.182	0.322	0.186
	bowl	9	28	0.648	0.643	0.612	0.492
	banana	1	1	0	0	0.166	0.0438
	sandwich	2	2	0.249	0.5	0.332	0.332
	orange	1	4	1	0.31	0.995	0.621
	broccoli	4	11	0.374	0.182	0.247	0.201
	carrot	3	24	0.648	0.458	0.57	0.358

hot dog	1	2	0.351	0.553	0.745	0.72
pizza	5	5	0.644	1	0.995	0.841
donut	2	14	0.657	1	0.94	0.863
cake	4	4	0.646	1	0.995	0.879
chair	9	35	0.506	0.514	0.442	0.237
couch	5	6	0.463	0.5	0.726	0.546
potted plant	9	14	0.606	0.643	0.704	0.484
bed	3	3	0.698	0.667	0.775	0.616
dining table	10	13	0.432	0.615	0.504	0.367
toilet	2	2	0.615	0.5	0.695	0.675
tv	2	2	0.373	0.62	0.745	0.695
laptop	2	3	1	0	0.451	0.361
mouse	2	2	1	0	0.045	0.0045
remote	5	8	0.843	0.5	0.602	0.53
cell phone	5	8	0	0	0.0497	0.0334
microwave	3	3	0.443	0.667	0.806	0.717
oven	5	5	0.412	0.4	0.335	0.267
sink	4	6	0.35	0.167	0.178	0.125
refrigerator	5	5	0.585	0.4	0.587	0.438
book	6	29	0.629	0.103	0.352	0.185
clock	8	9	0.788	0.83	0.875	0.737
vase	2	2	0.376	1	0.828	0.795
scissors	1	1	1	0	0.249	0.0746
teddy bear	6	21	0.877	0.333	0.59	0.388
toothbrush	2	5	0.743	0.6	0.638	0.372

Speed: 0.8ms preprocess, 5.0ms inference, 0.0ms loss, 1.7ms postprocess per image
Results saved to /kaggle/working/runs/detect/val

☑ Validation Metrics:

mAP50 : 0.6054
mAP50-95 : 0.4454
Precision : 0.6385
Recall : 0.5361

10. 📁 Export Model (ONNX, TFLite, TensorRT)

```
In [18]: # Export to ONNX (most portable format)
export_path = model.export(format='onnx', imgsz=640, simplify=True)
print(f"✅ ONNX model exported to: {export_path}")

# Other supported formats:
# model.export(format='tflite') # TensorFlow Lite (mobile)
# model.export(format='torchscript') # TorchScript
# model.export(format='engine') # TensorRT (NVIDIA GPU only)
```


ultralytics 8.4.56  Python-3.12.13 torch-2.10.0+cu128 CPU (Intel Xeon CPU @ 2.00GHz)
💡 ProTip: Export to OpenVINO format for best performance on Intel hardware. Learn more at <https://docs.ultralytics.com/integrations/openvino/>

PyTorch: starting from 'yolov8n.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 84, 8400) (6.2 MB)
requirements: Ultralytics requirements ['onnxslim>=0.1.71', 'onnxruntime'] not found, attempting AutoUpdate...
Using Python 3.12.13 environment at: /usr
Resolved 12 packages in 380ms
Prepared 2 packages in 388ms
Installed 2 packages in 17ms
+ onnxruntime==1.26.0
+ onnxslim==0.1.94

requirements: AutoUpdate success  1.4s
WARNING  **requirements:** Restart runtime or rerun command for updates to take effect

ONNX: starting export with onnx 1.21.0 opset 20...
ONNX: slimming with onnxslim 0.1.94...
ONNX: export success  3.1s, saved as 'yolov8n.onnx' (12.3 MB)

Export complete (3.5s)
Results saved to /kaggle/working/yolov8n.onnx
Predict: yolo predict task=detect model=yolov8n.onnx imgsz=640
Validate: yolo val task=detect model=yolov8n.onnx imgsz=640 data=coco.yaml
Visualize: <https://netron.app>
 ONNX model exported to: yolov8n.onnx

11. Plot Training Metrics (After Training)

```
In [21]: import pandas as pd

def plot_training_results(results_csv_path):
    """
    Plot training curves from results.csv generated by YOLO training.

    Args:
        results_csv_path: Path to results.csv in the training run folder
    """
    df = pd.read_csv(results_csv_path)
    df.columns = df.columns.str.strip()

    fig, axes = plt.subplots(2, 3, figsize=(18, 10))
    fig.suptitle('YOLOv8 Training Results', fontsize=16, fontweight='bold')

    metrics = [
        ('train/box_loss', 'Training Box Loss', 'blue'),
        ('train/cls_loss', 'Training Class Loss', 'orange'),
        ('metrics/mAP50(B)', 'Validation mAP@0.5', 'green'),
        ('metrics/precision(B)', 'Precision', 'red'),
        ('metrics/recall(B)', 'Recall', 'purple'),
        ('val/box_loss', 'Validation Box Loss', 'brown'),
    ]

    for ax, (col, title, color) in zip(axes.flat, metrics):
        if col in df.columns:
            ax.plot(df['epoch'], df[col], color=color, linewidth=2)
            ax.set_title(title, fontweight='bold')
            ax.set_xlabel('Epoch')
            ax.grid(True, alpha=0.3)
        else:
            ax.set_title(f'{title} (not found)', color='gray')
```

```
plt.tight_layout()
plt.savefig('training_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Plot saved as training_results.png")

# Usage after training:
# plot_training_results('runs/detect/my_yolo_run/results.csv')
print("🔥 Call plot_training_results('path/to/results.csv') after training.")
```

🔥 Call plot_training_results('path/to/results.csv') after training.

12. 🔍 Batch Inference on a Folder of Images

```
In [22]: def batch_inference(image_folder, output_folder='batch_output', conf=0.5):
        """
        Run YOLO detection on all images in a folder.

        Args:
            image_folder : Folder containing input images
            output_folder : Folder to save annotated images
            conf          : Confidence threshold
        """
        from pathlib import Path

        input_dir = Path(image_folder)
        output_dir = Path(output_folder)
        output_dir.mkdir(exist_ok=True)

        image_files = list(input_dir.glob('*.jpg')) + \
            list(input_dir.glob('*.jpeg')) + \
            list(input_dir.glob('*.png'))

        print(f"📁 Found {len(image_files)} images in {image_folder}")

        summary = []
        for img_path in image_files:
            results = model.predict(str(img_path), conf=conf, verbose=False)
            annotated = results[0].plot()

            out_path = output_dir / img_path.name
            cv2.imwrite(str(out_path), annotated)

            detections = [(model.names[int(b.cls[0])], float(b.conf[0]))
                          for b in results[0].boxes]
            summary.append({'image': img_path.name, 'detections': detections})

        print(f"✅ Results saved to: {output_folder}/")

        # Print summary
        print("\n📋 Detection Summary:")
        for item in summary:
            print(f"  {item['image']}: {len(item['detections'])} objects → ", end='')
            print(', '.join([f"{d[0]}({d[1]:.0%})" for d in item['detections'][:5]]))

        return summary

# Usage:
# summary = batch_inference('./my_images/', './results/')
print("🔥 Call batch_inference('folder_path') to run on all images in a folder.")
```

🔥 Call batch_inference('folder_path') to run on all images in a folder.

13. 🛠️ Utility: Draw Custom Bounding Boxes

```
In [23]: def draw_detections(image_path, results, conf_thresh=0.5):
        """
        Custom visualization of YOLO detection results using matplotlib.
        """
        img = np.array(Image.open(image_path).convert('RGB'))

        fig, ax = plt.subplots(1, figsize=(14, 9))
        ax.imshow(img)

        COLORS = plt.cm.get_cmap('tab20', len(model.names))

        for box in results[0].boxes:
            conf = float(box.conf[0])
            if conf < conf_thresh:
                continue

            cls = int(box.cls[0])
            label = model.names[cls]
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            color = COLORS[cls]

            rect = patches.Rectangle(
                (x1, y1), x2 - x1, y2 - y1,
                linewidth=2, edgecolor=color, facecolor='none'
            )
            ax.add_patch(rect)
            ax.text(
                x1, y1 - 5, f"{label} {conf:.0%}",
                color='white', fontsize=9, fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.2', facecolor=color, alpha=0.8)
            )

        ax.set_axis_off()
        ax.set_title(f'Custom Detections on {Path(image_path).name}', fontsize=13, fontweight='bold')
        plt.tight_layout()
        plt.show()

print("📌 Call draw_detections('image.jpg', results) for custom matplotlib visualization.")
```

📌 Call draw_detections('image.jpg', results) for custom matplotlib visualization.

Program 9 - VGG16 — From Scratch & Transfer Learning

Cell 1 — Install & Import Dependencies

```
In [1]: # Install required packages (run once)
# !pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
# !pip install matplotlib numpy pillow tqdm

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader, random_split

import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
from torchvision.datasets import CIFAR10, ImageFolder

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from PIL import Image
from tqdm import tqdm
import os, time, copy

# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
print(f'PyTorch version: {torch.__version__}')
print(f'Torchvision version: {torchvision.__version__}')
```

Using device: cuda
PyTorch version: 2.10.0+cu128
Torchvision version: 0.25.0+cu128

Cell 2 — VGG16 Architecture From Scratch

```
In [2]: class VGG16(nn.Module):
        """
        VGG-16 architecture as described in the original paper.
        Input: (batch, 3, 224, 224)
        """
        def __init__(self, num_classes=1000, dropout=0.5):
            super(VGG16, self).__init__()

            # — Feature extractor (5 blocks) —————
            self.features = nn.Sequential(
                # Block 1
                nn.Conv2d(3, 64, kernel_size=3, padding=1), nn.ReLU(inplace=True),
                nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.ReLU(inplace=True),
                nn.MaxPool2d(kernel_size=2, stride=2), # 112x112

                # Block 2
                nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(inplace=True),
                nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.ReLU(inplace=True),
                nn.MaxPool2d(kernel_size=2, stride=2), # 56x56
```

```

# Block 3
nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2), # 28x28

# Block 4
nn.Conv2d(256, 512, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2), # 14x14

# Block 5
nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2), # 7x7
)

# — Adaptive pool (makes model input-size agnostic) —————
self.avgpool = nn.AdaptiveAvgPool2d((7, 7))

# — Classifier (3 fully-connected layers) —————
self.classifier = nn.Sequential(
    nn.Linear(512 * 7 * 7, 4096), nn.ReLU(inplace=True), nn.Dropout(p=dropout),
    nn.Linear(4096, 4096), nn.ReLU(inplace=True), nn.Dropout(p=dropout),
    nn.Linear(4096, num_classes),
)

self._initialize_weights()

def forward(self, x):
    x = self.features(x)
    x = self.avgpool(x)
    x = torch.flatten(x, 1)
    x = self.classifier(x)
    return x

def _initialize_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, 0, 0.01)
            nn.init.constant_(m.bias, 0)

# Quick sanity check
model_scratch = VGG16(num_classes=10).to(device)
dummy = torch.randn(2, 3, 224, 224).to(device)
out = model_scratch(dummy)
print('VGG16 (scratch) output shape:', out.shape) # (2, 10)

# Parameter count
total_params = sum(p.numel() for p in model_scratch.parameters())
print(f'Total parameters: {total_params:,} (~{total_params/1e6:.1f}M)')

```

VGG16 (scratch) output shape: torch.Size([2, 10])

Total parameters: 134,301,514 (~134.3M)

Cell 3 — Model Summary / Layer Inspection

In [3]:

```
def model_summary(model, input_size=(3, 224, 224)):
    """Print layer-by-layer summary with output shapes."""
    print(f"{'Layer':<40} {'Output Shape':<25} {'Params':>12}")
    print('-' * 80)

    hooks = []
    shapes = {}

    def hook_fn(name):
        def fn(module, inp, out):
            shapes[name] = tuple(out.shape[1:])
        return fn

    for name, layer in model.named_modules():
        if len(list(layer.children())) == 0: # Leaf modules only
            hooks.append(layer.register_forward_hook(hook_fn(name)))

    x = torch.randn(1, *input_size).to(device)
    model.eval()
    with torch.no_grad():
        model(x)

    for name, layer in model.named_modules():
        if len(list(layer.children())) == 0:
            params = sum(p.numel() for p in layer.parameters())
            shape = shapes.get(name, '-')
            print(f"{'name':<40} {'str(shape):<25} {'params:>12,}")

    for h in hooks:
        h.remove()

model_summary(model_scratch)
```

Layer	Output Shape	Params
features.0	(64, 224, 224)	1,792
features.1	(64, 224, 224)	0
features.2	(64, 224, 224)	36,928
features.3	(64, 224, 224)	0
features.4	(64, 112, 112)	0
features.5	(128, 112, 112)	73,856
features.6	(128, 112, 112)	0
features.7	(128, 112, 112)	147,584
features.8	(128, 112, 112)	0
features.9	(128, 56, 56)	0
features.10	(256, 56, 56)	295,168
features.11	(256, 56, 56)	0
features.12	(256, 56, 56)	590,080
features.13	(256, 56, 56)	0
features.14	(256, 56, 56)	590,080
features.15	(256, 56, 56)	0
features.16	(256, 28, 28)	0
features.17	(512, 28, 28)	1,180,160
features.18	(512, 28, 28)	0
features.19	(512, 28, 28)	2,359,808
features.20	(512, 28, 28)	0
features.21	(512, 28, 28)	2,359,808
features.22	(512, 28, 28)	0
features.23	(512, 14, 14)	0
features.24	(512, 14, 14)	2,359,808
features.25	(512, 14, 14)	0
features.26	(512, 14, 14)	2,359,808
features.27	(512, 14, 14)	0
features.28	(512, 14, 14)	2,359,808
features.29	(512, 14, 14)	0
features.30	(512, 7, 7)	0
avgpool	(512, 7, 7)	0
classifier.0	(4096,)	102,764,544
classifier.1	(4096,)	0
classifier.2	(4096,)	0
classifier.3	(4096,)	16,781,312
classifier.4	(4096,)	0
classifier.5	(4096,)	0
classifier.6	(10,)	40,970

Cell 4 — Pretrained VGG16 (Transfer Learning)

```
In [4]: NUM_CLASSES = 10    # Change for your dataset

# Load ImageNet-pretrained VGG16
model_pretrained = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)

# — Strategy 1: Fine-tune only the classifier —————
for param in model_pretrained.parameters():
    param.requires_grad = False    # Freeze convolutional layers

# Replace final FC layer
in_features = model_pretrained.classifier[6].in_features
model_pretrained.classifier[6] = nn.Linear(in_features, NUM_CLASSES)

model_pretrained = model_pretrained.to(device)

# Only classifier parameters will be optimised
trainable = sum(p.numel() for p in model_pretrained.parameters() if p.requires_grad)
total     = sum(p.numel() for p in model_pretrained.parameters())
print(f'Trainable parameters : {trainable:,}')
print(f'Frozen parameters   : {total - trainable:,}')
```

```
print(f'Total parameters : {total:,}')
```

```
# Quick forward pass
```

```
x = torch.randn(2, 3, 224, 224).to(device)
```

```
out = model_pretrained(x)
```

```
print('Pretrained VGG16 output shape:', out.shape)
```

Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth

100%|██████████| 528M/528M [00:02<00:00, 226MB/s]

Trainable parameters : 119,586,826

Frozen parameters : 14,714,688

Total parameters : 134,301,514

Pretrained VGG16 output shape: torch.Size([2, 10])

Cell 5 — Data Transforms & CIFAR-10 Loader

```
In [5]: # — Transforms —————
```

```
# VGG16 expects 224x224; CIFAR-10 is 32x32 — we upscale.
```

```
train_transform = transforms.Compose([
```

```
    transforms.Resize((224, 224)),
```

```
    transforms.RandomHorizontalFlip(p=0.5),
```

```
    transforms.RandomRotation(degrees=10),
```

```
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
```

```
    transforms.ToTensor(),
```

```
    transforms.Normalize(mean=[0.485, 0.456, 0.406], # ImageNet stats
```

```
                        std =[0.229, 0.224, 0.225]),
```

```
])
```

```
val_transform = transforms.Compose([
```

```
    transforms.Resize((224, 224)),
```

```
    transforms.ToTensor(),
```

```
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
```

```
                        std =[0.229, 0.224, 0.225]),
```

```
])
```

```
# — Datasets —————
```

```
DATA_ROOT = './data'
```

```
BATCH_SIZE = 32
```

```
train_dataset = CIFAR10(root=DATA_ROOT, train=True, download=True, transform=train_transform)
```

```
test_dataset = CIFAR10(root=DATA_ROOT, train=False, download=True, transform=val_transform)
```

```
# 80/20 train-validation split from the training set
```

```
n_val = int(0.2 * len(train_dataset))
```

```
n_train = len(train_dataset) - n_val
```

```
train_ds, val_ds = random_split(train_dataset, [n_train, n_val],
```

```
                                generator=torch.Generator().manual_seed(42))
```

```
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_
```

```
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_
```

```
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, )
```

```
CLASSES = train_dataset.classes
```

```
print('Classes:', CLASSES)
```

```
print(f'Train: {len(train_ds):,} | Val: {len(val_ds):,} | Test: {len(test_dataset):,}')
```

```
# — Visualise a batch —————
```

```
images, labels = next(iter(train_loader))
```

```
mean = torch.tensor([0.485, 0.456, 0.406]).view(3,1,1)
```

```
std = torch.tensor([0.229, 0.224, 0.225]).view(3,1,1)
```

```
imgs_denorm = (images[:8] * std + mean).clamp(0, 1)
```

```
fig, axes = plt.subplots(1, 8, figsize=(18, 3))
```



```

for ax, img, lbl in zip(axes, imgs_denorm, labels[:8]):
    ax.imshow(img.permute(1, 2, 0).numpy())
    ax.set_title(CLASSES[lbl], fontsize=8)
    ax.axis('off')
plt.suptitle('Sample Training Images (after augmentation)', y=1.02)
plt.tight_layout()
plt.show()

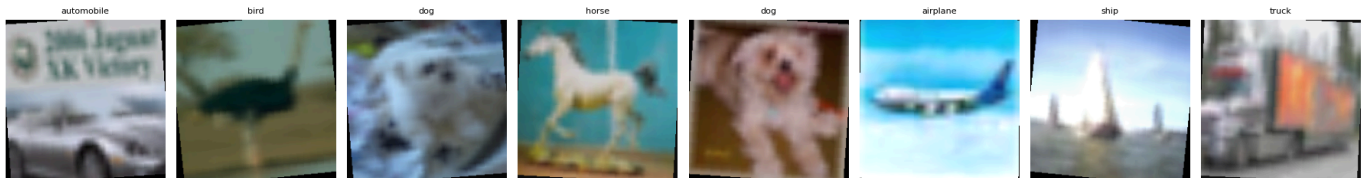
```

100%|██████████| 170M/170M [00:06<00:00, 25.9MB/s]

Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

Train: 40,000 | Val: 10,000 | Test: 10,000

Sample Training Images (after augmentation)



Cell 6 — Training Loop

```

In [6]: def train_model(model, train_loader, val_loader, num_epochs=10,
                        lr=1e-4, weight_decay=5e-4):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()),
                              lr=lr, weight_decay=weight_decay)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_acc = 0.0
    best_weights = copy.deepcopy(model.state_dict())

    for epoch in range(1, num_epochs + 1):
        start = time.time()

        # — Training —————
        model.train()
        running_loss = correct = total = 0
        for imgs, lbls in tqdm(train_loader, desc=f'Epoch {epoch}/{num_epochs} [Train]', leave=False):
            imgs, lbls = imgs.to(device), lbls.to(device)
            optimizer.zero_grad()
            outputs = model(imgs)
            loss = criterion(outputs, lbls)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * imgs.size(0)
            _, preds = outputs.max(1)
            correct += preds.eq(lbls).sum().item()
            total += imgs.size(0)

        train_loss = running_loss / total
        train_acc = correct / total * 100

        # — Validation —————
        model.eval()
        running_loss = correct = total = 0
        with torch.no_grad():
            for imgs, lbls in tqdm(val_loader, desc=f'Epoch {epoch}/{num_epochs} [Val]', leave=False):
                imgs, lbls = imgs.to(device), lbls.to(device)
                outputs = model(imgs)
                loss = criterion(outputs, lbls)
                running_loss += loss.item() * imgs.size(0)

```

```

_, preds = outputs.max(1)
correct += preds.eq(lbls).sum().item()
total   += imgs.size(0)

val_loss = running_loss / total
val_acc  = correct / total * 100

scheduler.step()
elapsed = time.time() - start

# Save best
if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_weights = copy.deepcopy(model.state_dict())

history['train_loss'].append(train_loss)
history['val_loss'].append(val_loss)
history['train_acc'].append(train_acc)
history['val_acc'].append(val_acc)

print(f'Epoch [{epoch:02d}/{num_epochs}] '
      f'Train Loss: {train_loss:.4f} Train Acc: {train_acc:.2f}% '
      f'Val Loss: {val_loss:.4f} Val Acc: {val_acc:.2f}% '
      f'({elapsed:.1f}s)')

print(f'\nBest Validation Accuracy: {best_val_acc:.2f}%')
model.load_state_dict(best_weights)
return model, history

# — Run training (set num_epochs to 1 for a quick smoke-test) —————
NUM_EPOCHS = 1 # Increase for real training
trained_model, history = train_model(model_pretrained, train_loader, val_loader,
                                     num_epochs=NUM_EPOCHS, lr=1e-4)

```

Epoch [01/1] Train Loss: 0.7823 Train Acc: 72.46% Val Loss: 0.5882 Val Acc: 79.64% (335.5 s)

Best Validation Accuracy: 79.64%

Cell 7 — Plot Training Curves

```

In [7]: epochs = range(1, len(history['train_loss']) + 1)

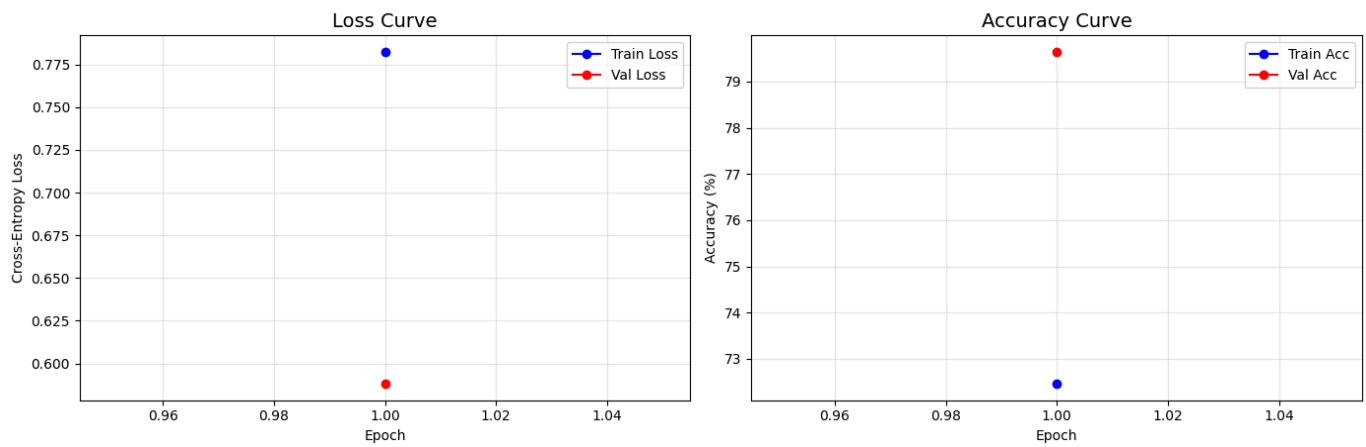
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(epochs, history['train_loss'], 'b-o', label='Train Loss')
ax1.plot(epochs, history['val_loss'], 'r-o', label='Val Loss')
ax1.set_title('Loss Curve', fontsize=14)
ax1.set_xlabel('Epoch'); ax1.set_ylabel('Cross-Entropy Loss')
ax1.legend(); ax1.grid(alpha=0.3)

ax2.plot(epochs, history['train_acc'], 'b-o', label='Train Acc')
ax2.plot(epochs, history['val_acc'], 'r-o', label='Val Acc')
ax2.set_title('Accuracy Curve', fontsize=14)
ax2.set_xlabel('Epoch'); ax2.set_ylabel('Accuracy (%)')
ax2.legend(); ax2.grid(alpha=0.3)

plt.suptitle('VGG16 Training History', fontsize=16, y=1.02)
plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved training_curves.png')

```



Saved training_curves.png

Cell 8 — Evaluate on Test Set + Confusion Matrix

```
In [8]: from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

trained_model.eval()
all_preds, all_labels = [], []

with torch.no_grad():
    for imgs, lbls in tqdm(test_loader, desc='Testing'):
        imgs = imgs.to(device)
        out = trained_model(imgs)
        _, preds = out.max(1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(lbls.numpy())

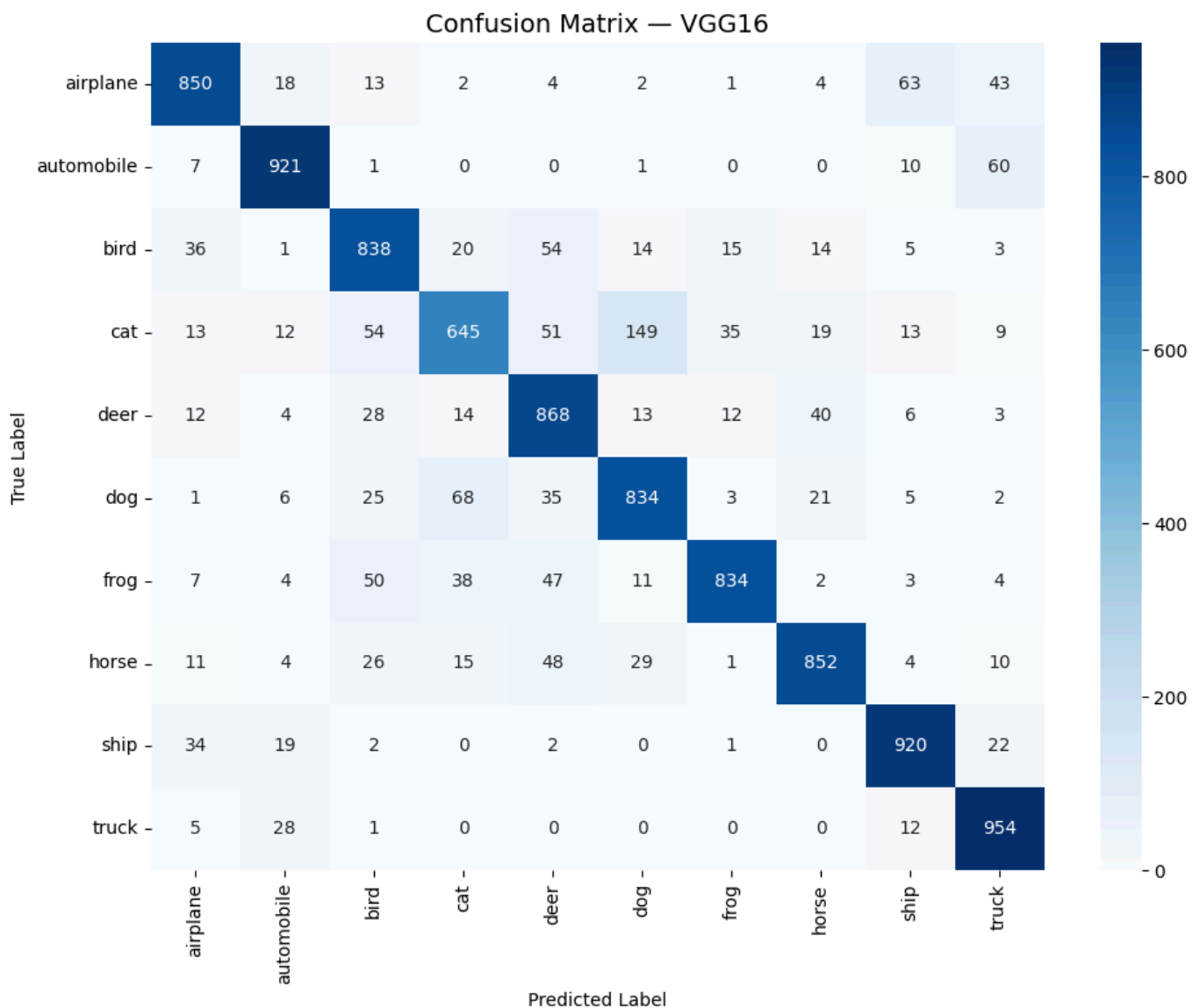
test_acc = (np.array(all_preds) == np.array(all_labels)).mean() * 100
print(f'Test Accuracy: {test_acc:.2f}%\n')
print(classification_report(all_labels, all_preds, target_names=CLASSES))

# Confusion matrix
cm_arr = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(10, 8))
sns.heatmap(cm_arr, annot=True, fmt='d', cmap='Blues',
            xticklabels=CLASSES, yticklabels=CLASSES)
plt.title('Confusion Matrix - VGG16', fontsize=14)
plt.ylabel('True Label'); plt.xlabel('Predicted Label')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
```

Testing: 100% |██████████| 313/313 [01:00<00:00, 5.18it/s]

Test Accuracy: 85.16%

	precision	recall	f1-score	support
airplane	0.87	0.85	0.86	1000
automobile	0.91	0.92	0.91	1000
bird	0.81	0.84	0.82	1000
cat	0.80	0.65	0.72	1000
deer	0.78	0.87	0.82	1000
dog	0.79	0.83	0.81	1000
frog	0.92	0.83	0.88	1000
horse	0.89	0.85	0.87	1000
ship	0.88	0.92	0.90	1000
truck	0.86	0.95	0.90	1000
accuracy			0.85	10000
macro avg	0.85	0.85	0.85	10000
weighted avg	0.85	0.85	0.85	10000



Cell 9 — Single-Image Inference

```
In [9]: def predict_image(model, img_path_or_tensor, top_k=5):  
        """  
        Run inference on a single image.  
        Accepts a file path (str) or a PIL Image or a tensor.  
        """  
        infer_transform = transforms.Compose([  
            transforms.Resize((224, 224)),  
            transforms.ToTensor(),
```

```

transforms.Normalize([0.485, 0.456, 0.406],
                      [0.229, 0.224, 0.225]),
])

if isinstance(img_path_or_tensor, str):
    img = Image.open(img_path_or_tensor).convert('RGB')
elif isinstance(img_path_or_tensor, Image.Image):
    img = img_path_or_tensor.convert('RGB')
else:
    raise TypeError('Expected str path or PIL.Image')

tensor = infer_transform(img).unsqueeze(0).to(device)

model.eval()
with torch.no_grad():
    logits = model(tensor)
    probs = F.softmax(logits, dim=1)

topk_probs, topk_idx = probs.topk(top_k, dim=1)
topk_probs = topk_probs.squeeze().cpu().numpy()
topk_idx = topk_idx.squeeze().cpu().numpy()

print(f'Top-{top_k} predictions:')
for p, idx in zip(topk_probs, topk_idx):
    print(f' {CLASSES[idx]:<15} {p*100:.2f}%')

# Visualise
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.imshow(img.resize((224, 224)))
ax1.set_title('Input Image'); ax1.axis('off')

colors = ['#4C72B0' if i > 0 else '#DD4444' for i in range(top_k-1, -1, -1)]
ax2.barh([CLASSES[i] for i in reversed(topk_idx)],
          [p*100 for p in reversed(topk_probs)], color=colors)
ax2.set_xlabel('Probability (%)'); ax2.set_title('Top-K Predictions')
plt.tight_layout()
plt.show()
return CLASSES[topk_idx[0]], topk_probs[0]

# Example: pick a random test image
idx = 42
sample_img_tensor, sample_label = test_dataset[idx]
# Convert tensor back to PIL for our function
mean_t = torch.tensor([0.485, 0.456, 0.406]).view(3,1,1)
std_t = torch.tensor([0.229, 0.224, 0.225]).view(3,1,1)
pil_img = transforms.ToPILImage()((sample_img_tensor * std_t + mean_t).clamp(0,1))

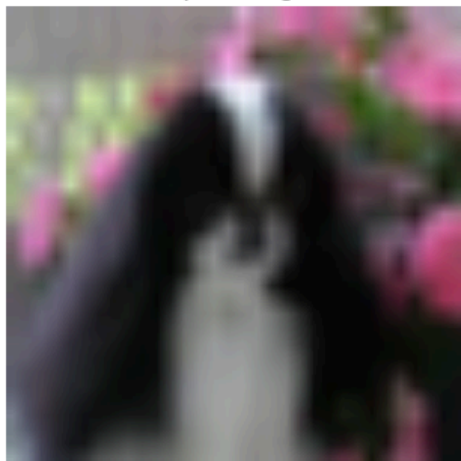
pred_class, pred_prob = predict_image(trained_model, pil_img)
print(f'\nGround Truth : {CLASSES[sample_label]}')
print(f'Prediction : {pred_class} ({pred_prob*100:.1f}%)')

```

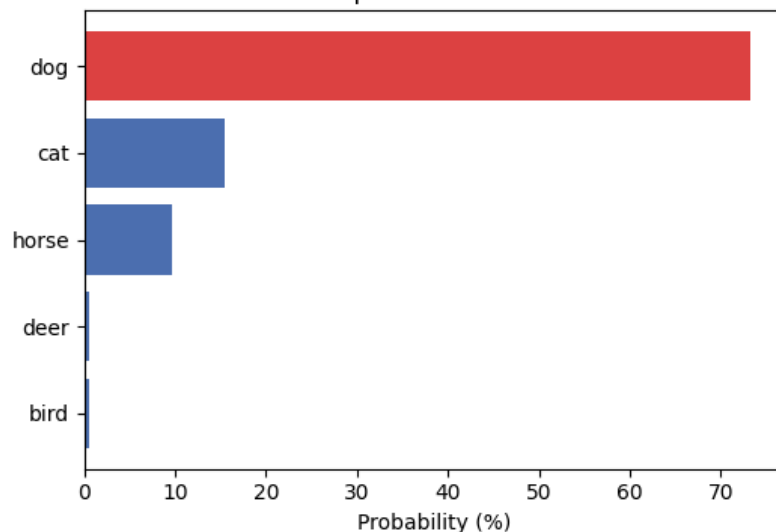
Top-5 predictions:

dog	73.21%
cat	15.41%
horse	9.67%
deer	0.53%
bird	0.51%

Input Image



Top-K Predictions



Ground Truth : dog

Prediction : dog (73.2%)

Cell 10 — Feature Map Visualization

```
In [10]: def visualize_feature_maps(model, img_tensor, layer_name='features.3', n_filters=16):
    """
    Extract and display feature maps at a given layer.
    img_tensor: (1, C, H, W) already normalised.
    """
    activation = {}

    def get_activation(name):
        def hook(module, inp, out):
            activation[name] = out.detach()
        return hook

    # Register hook on target Layer
    for name, module in model.named_modules():
        if name == layer_name:
            handle = module.register_forward_hook(get_activation(layer_name))
            break

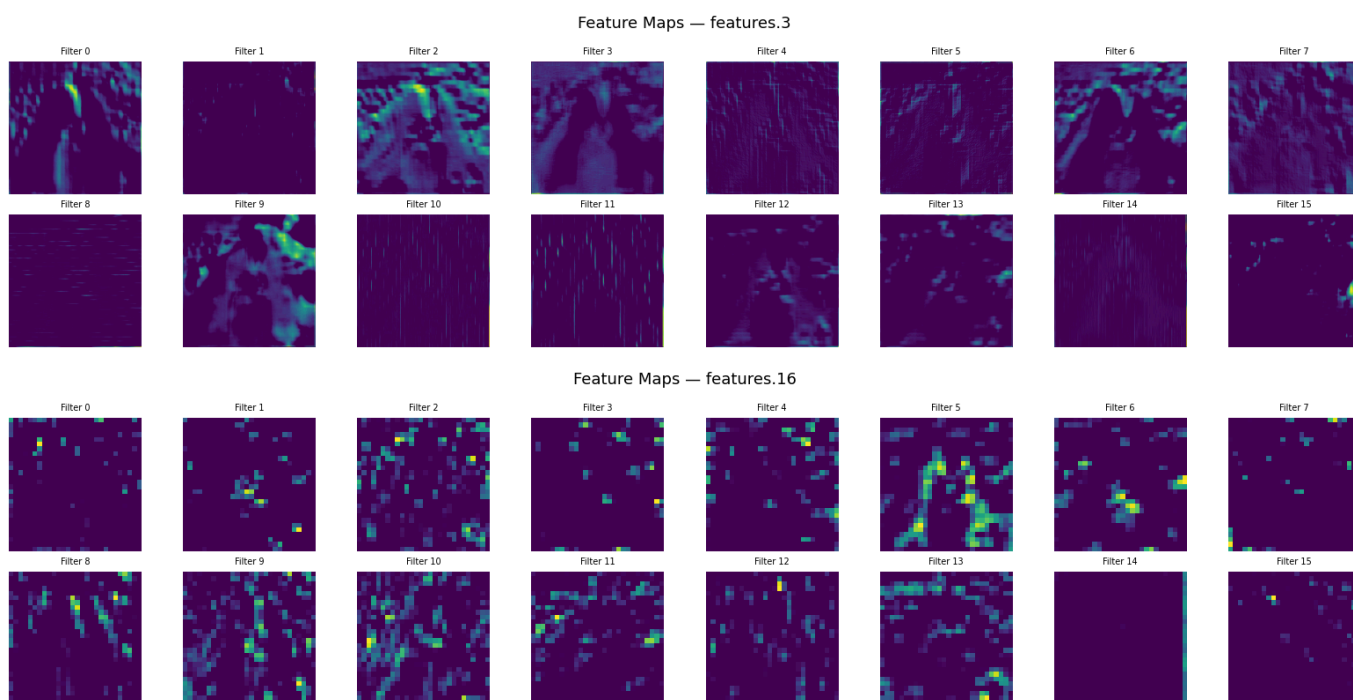
    model.eval()
    with torch.no_grad():
        model(img_tensor.to(device))
    handle.remove()

    maps = activation[layer_name][0].cpu() # (C, H, W)
    n = min(n_filters, maps.shape[0])
    cols = 8
    rows = (n + cols - 1) // cols

    fig, axes = plt.subplots(rows, cols, figsize=(cols*2, rows*2))
    axes = axes.flatten()
    for i in range(n):
        axes[i].imshow(maps[i].numpy(), cmap='viridis')
        axes[i].set_title(f'Filter {i}', fontsize=7)
        axes[i].axis('off')
    for j in range(n, len(axes)):
        axes[j].axis('off')

    plt.suptitle(f'Feature Maps - {layer_name}', fontsize=13)
    plt.tight_layout()
    plt.savefig(f'feature_maps_{layer_name.replace(".", "_")}.png', dpi=120)
    plt.show()
```

```
# Visualise Block-1 ReLU (features.3) and Block-3 ReLU (features.16)
sample_input = sample_img_tensor.unsqueeze(0) # (1,3,224,224)
visualize_feature_maps(trained_model, sample_input, layer_name='features.3', n_filters=16)
visualize_feature_maps(trained_model, sample_input, layer_name='features.16', n_filters=16)
```



```
In [12]: import torch.nn as nn

# Disable inplace ReLU for VGG
for m in trained_model.modules():
    if isinstance(m, nn.ReLU):
        m.inplace = False

trained_model.eval()

if sample_input.ndim == 3:
    sample_input = sample_input.unsqueeze(0)

sample_input = sample_input.to(device)

target_layer = trained_model.features[28]

gradcam = GradCAM(
    trained_model,
    target_layer
)

cam_map, pred_idx = gradcam.generate(sample_input)
```

Cell 11 — Grad-CAM Explainability

```
In [13]: class GradCAM:
    """
    Gradient-weighted Class Activation Mapping (Selvaraju et al., 2017).
    Highlights image regions most influential for the predicted class.
    """
    def __init__(self, model, target_layer):
        self.model = model
        self.gradients = None
        self.activations = None

        target_layer.register_forward_hook(self._save_activation)
```

```

target_layer.register_full_backward_hook(self._save_gradient)

def _save_activation(self, module, inp, out):
    self.activations = out.detach()

def _save_gradient(self, module, grad_in, grad_out):
    self.gradients = grad_out[0].detach()

def generate(self, img_tensor, class_idx=None):
    self.model.eval()
    img_tensor = img_tensor.to(device).requires_grad_(True)
    logits = self.model(img_tensor)

    if class_idx is None:
        class_idx = logits.argmax(dim=1).item()

    self.model.zero_grad()
    logits[0, class_idx].backward()

    # Global average pool gradients over spatial dims
    weights = self.gradients.mean(dim=(2, 3), keepdim=True) # (1, C, 1, 1)
    cam = (weights * self.activations).sum(dim=1, keepdim=True) # (1, 1, H, W)
    cam = F.relu(cam)
    cam = F.interpolate(cam, size=(224, 224), mode='bilinear', align_corners=False)
    cam = cam.squeeze().cpu().numpy()
    cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
    return cam, class_idx

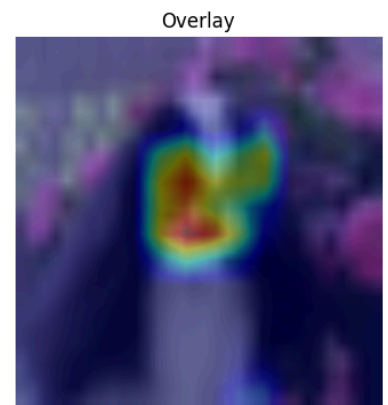
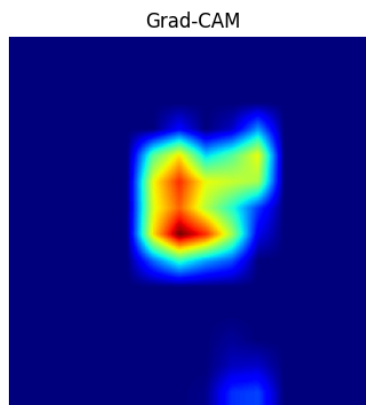
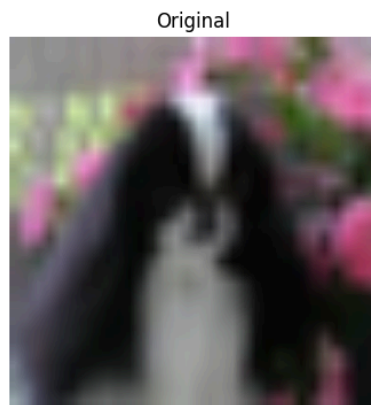
def overlay_gradcam(original_pil, cam):
    img_np = np.array(original_pil.resize((224, 224))).astype(np.float32) / 255
    heatmap = cm.jet(cam)[:, :, :3]
    overlay = 0.6 * img_np + 0.4 * heatmap
    return np.clip(overlay, 0, 1)

# Use last conv layer (features.28) for richest spatial info
target_layer = trained_model.features[28]
gradcam = GradCAM(trained_model, target_layer)

cam_map, pred_idx = gradcam.generate(sample_input)
overlay = overlay_gradcam(pil_img, cam_map)

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
axes[0].imshow(pil_img.resize((224, 224))); axes[0].set_title('Original'); axes[0].axis('off')
axes[1].imshow(cam_map, cmap='jet'); axes[1].set_title('Grad-CAM'); axes[1].axis('off')
axes[2].imshow(overlay); axes[2].set_title('Overlay'); axes[2].axis('off')
plt.suptitle(f'Grad-CAM | Predicted: {CLASSES[pred_idx]}', fontsize=13)
plt.tight_layout()
plt.savefig('gradcam.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved gradcam.png')

```

Saved gradcam.png

Cell 12 — Save & Load Checkpoint

```
In [14]: CKPT_PATH = 'vgg16_cifar10_best.pth'

# — Save —————
torch.save({
    'epoch'      : NUM_EPOCHS,
    'model_state' : trained_model.state_dict(),
    'val_acc'     : max(history['val_acc']),
    'classes'     : CLASSES,
    'num_classes' : NUM_CLASSES,
}, CKPT_PATH)
print(f'Checkpoint saved → {CKPT_PATH}')

# — Load —————
def load_vgg16_checkpoint(ckpt_path, device='cpu'):
    ckpt      = torch.load(ckpt_path, map_location=device)
    num_cls   = ckpt['num_classes']
    model     = models.vgg16(weights=None)
    model.classifier[6] = nn.Linear(model.classifier[6].in_features, num_cls)
    model.load_state_dict(ckpt['model_state'])
    model.eval()
    print(f"Loaded checkpoint (epoch {ckpt['epoch']}, val_acc={ckpt['val_acc']:.2f}%)")
    return model, ckpt['classes']

loaded_model, loaded_classes = load_vgg16_checkpoint(CKPT_PATH, device=device)
loaded_model = loaded_model.to(device)
print('Classes:', loaded_classes)
```

Checkpoint saved → vgg16_cifar10_best.pth

Loaded checkpoint (epoch 1, val_acc=79.64%)

Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

Program 10 - Faster R-CNN Object Detection

Cell 1: Install Dependencies

```
In [1]: # Install required packages
!pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
!pip install opencv-python matplotlib Pillow pycocotools albumentations
!pip install torchmetrics

print('✅ All dependencies installed successfully!')
```

dist-packages (from torch>=2.0.0->torchmetrics) (11.8.89)
Requirement already satisfied: nvidia-cuda-cupti-cu11==11.8.87 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (11.8.87)
Requirement already satisfied: nvidia-cudnn-cu11==9.1.0.70 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (9.1.0.70)
Requirement already satisfied: nvidia-cublas-cu11==11.11.3.6 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (11.11.3.6)
Requirement already satisfied: nvidia-cufft-cu11==10.9.0.58 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (10.9.0.58)
Requirement already satisfied: nvidia-curand-cu11==10.3.0.86 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (10.3.0.86)
Requirement already satisfied: nvidia-cusolver-cu11==11.4.1.48 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (11.4.1.48)
Requirement already satisfied: nvidia-cuspars-cu11==11.7.5.86 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (11.7.5.86)
Requirement already satisfied: nvidia-nccl-cu11==2.21.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu11==11.8.86 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (11.8.86)
Requirement already satisfied: triton==3.3.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (3.3.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0->torchmetrics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=2.0.0->torchmetrics) (3.0.3)
✅ All dependencies installed successfully!

Cell 2: Imports & Configuration

```
In [2]: import os
import json
import random
import numpy as np
from pathlib import Path
from PIL import Image
from typing import List, Tuple, Dict, Optional

import torch
import torch.nn as nn
import torchvision
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import torchvision.transforms.functional as F

from torchvision.models.detection import fasterrcnn_resnet50_fpn, FasterRCNN
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.rpn import AnchorGenerator
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2

import cv2
import matplotlib.pyplot as plt
import matplotlib.patches as patches

# — Config —
CONFIG = {
    'num_classes' : 91,          # COCO has 80 classes + background = 91
    'batch_size'  : 4,
    'num_epochs'  : 10,
    'lr'          : 0.005,
    'momentum'    : 0.9,
    'weight_decay': 0.0005,
    'lr_step_size': 3,
    'lr_gamma'    : 0.1,
    'score_thresh': 0.5,
```

```

'nms_thresh'      : 0.3,
'device'          : 'cuda' if torch.cuda.is_available() else 'cpu',
'seed'            : 42,
'data_dir'        : './data',
'output_dir'      : './outputs',
'backbone'        : 'resnet50', # Options: resnet50, mobilenet_v3
}

# Seed everything
random.seed(CONFIG['seed'])
np.random.seed(CONFIG['seed'])
torch.manual_seed(CONFIG['seed'])
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(CONFIG['seed'])

os.makedirs(CONFIG['output_dir'], exist_ok=True)
os.makedirs(CONFIG['data_dir'], exist_ok=True)

print(f"🖥️ Device: {CONFIG['device']}")
print(f"🔥 PyTorch: {torch.__version__}")
print(f"👁️ TorchVision: {torchvision.__version__}")
if torch.cuda.is_available():
    print(f"🎮 GPU: {torch.cuda.get_device_name(0)}")

```

🖥️ Device: cuda
 🔥 PyTorch: 2.7.1+cu118
 👁️ TorchVision: 0.22.1+cu118
 🎮 GPU: Tesla T4

Cell 3: COCO Class Labels

```

In [3]: # COCO 80-class Labels (index 0 = background)
COCO_CLASSES = [
    '__background__', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',
    'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
    'N/A', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
    'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
    'N/A', 'backpack', 'umbrella', 'N/A', 'N/A', 'handbag', 'tie',
    'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite',
    'baseball bat', 'baseball glove', 'skateboard', 'surfboard',
    'tennis racket', 'bottle', 'N/A', 'wine glass', 'cup', 'fork',
    'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange',
    'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair',
    'couch', 'potted plant', 'bed', 'N/A', 'dining table', 'N/A', 'N/A',
    'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboard',
    'cell phone', 'microwave', 'oven', 'toaster', 'sink', 'refrigerator',
    'N/A', 'book', 'clock', 'vase', 'scissors', 'teddy bear',
    'hair drier', 'toothbrush'
]

# Color palette for visualization
COLORS = np.random.uniform(0, 255, size=(len(COCO_CLASSES), 3)).astype(int)

print(f'📦 Total classes (with background): {len(COCO_CLASSES)}')
print(f'🎨 Color palette generated for {len(COCO_CLASSES)} classes')

```

📦 Total classes (with background): 91
 🎨 Color palette generated for 91 classes

Cell 4: Custom Dataset Class (COCO format)

```

In [4]: class COCODetectionDataset(Dataset):
        """
        Custom Dataset for COCO-format object detection.

```

Expects:

```
    root/
        images/
        annotations/instances_<split>.json
    """
```

```
def __init__(self, root: str, split: str = 'train', transforms=None):
    self.root = Path(root)
    self.split = split
    self.transforms = transforms

    ann_file = self.root / 'annotations' / f'instances_{split}.json'
    with open(ann_file) as f:
        coco_data = json.load(f)

    # Build image id -> file name map
    self.images = {img['id']: img for img in coco_data['images']}
    self.img_ids = list(self.images.keys())

    # Build image id -> annotations map
    self.annotations: Dict[int, List] = {id: [] for id in self.img_ids}
    for ann in coco_data['annotations']:
        if ann['image_id'] in self.annotations:
            self.annotations[ann['image_id']].append(ann)

    # Category id remapping
    self.cat_id_to_label = {
        cat['id']: i + 1 # +1 for background
        for i, cat in enumerate(coco_data['categories'])
    }

    print(f'📁 Dataset [{split}]: {len(self.img_ids)} images loaded')

def __len__(self):
    return len(self.img_ids)

def __getitem__(self, idx):
    img_id = self.img_ids[idx]
    img_info = self.images[img_id]
    img_path = self.root / 'images' / img_info['file_name']

    image = Image.open(img_path).convert('RGB')

    anns = self.annotations[img_id]
    boxes, labels, areas, iscrowd = [], [], [], []

    for ann in anns:
        x, y, w, h = ann['bbox']
        if w <= 0 or h <= 0:
            continue
        boxes.append([x, y, x + w, y + h])
        labels.append(self.cat_id_to_label[ann['category_id']])
        areas.append(ann['area'])
        iscrowd.append(ann.get('iscrowd', 0))

    target = {
        'boxes' : torch.tensor(boxes, dtype=torch.float32) if boxes else torch.zeros(
        'labels' : torch.tensor(labels, dtype=torch.int64),
        'area' : torch.tensor(areas, dtype=torch.float32),
        'iscrowd' : torch.tensor(iscrowd, dtype=torch.int64),
        'image_id' : torch.tensor([img_id])
    }

    if self.transforms:
        image, target = self.transforms(image, target)
```

```

return image, target

# — Collate fn for variable-size bboxes —
def collate_fn(batch):
    return tuple(zip(*batch))

print('✅ Dataset class defined')

```

✅ Dataset class defined

Cell 5: Data Augmentation Transforms

```

In [5]: class Compose:
    def __init__(self, transforms):
        self.transforms = transforms

    def __call__(self, image, target):
        for t in self.transforms:
            image, target = t(image, target)
        return image, target

class ToTensor:
    def __call__(self, image, target):
        image = F.to_tensor(image)
        return image, target

class RandomHorizontalFlip:
    def __init__(self, prob=0.5):
        self.prob = prob

    def __call__(self, image, target):
        if random.random() < self.prob:
            width, _ = F.get_image_size(image)
            image = F.hflip(image)
            boxes = target['boxes']
            if boxes.numel() > 0:
                boxes[:, [0, 2]] = width - boxes[:, [2, 0]]
            target['boxes'] = boxes
        return image, target

class RandomPhotometricDistort:
    """Brightness, contrast, saturation, hue jitter."""
    def __init__(self):
        self.jitter = transforms.ColorJitter(
            brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1
        )

    def __call__(self, image, target):
        image = self.jitter(image)
        return image, target

def get_transforms(train: bool) -> Compose:
    t = [ToTensor()]
    if train:
        t = [RandomPhotometricDistort(), RandomHorizontalFlip()] + t
    return Compose(t)

print('✅ Augmentation pipeline ready')

```

```
print('  Train transforms : PhotometricDistort → HorizontalFlip → ToTensor')
print('  Val   transforms : ToTensor')
```

✓ Augmentation pipeline ready

Train transforms : PhotometricDistort → HorizontalFlip → ToTensor

Val transforms : ToTensor

Cell 6: Build Faster R-CNN Model

```
In [6]: def build_faster_rcnn(
        num_classes: int,
        backbone: str = 'resnet50',
        pretrained: bool = True,
        trainable_backbone_layers: int = 3,
        custom_anchors: bool = False,
    ) -> FasterRCNN:
        """
        Build a Faster R-CNN model.

        Args:
            num_classes          : Number of classes INCLUDING background.
            backbone             : 'resnet50' | 'mobilenet_v3'
            pretrained           : Use COCO pretrained weights.
            trainable_backbone_layers: Number of layers to unfreeze.
            custom_anchors       : If True, use custom anchor sizes.

        Returns:
            model : FasterRCNN
        """
        if backbone == 'resnet50':
            if custom_anchors:
                anchor_gen = AnchorGenerator(
                    sizes=((32,), (64,), (128,), (256,), (512,)),
                    aspect_ratios=((0.5, 1.0, 2.0),) * 5
                )
                model = fasterrcnn_resnet50_fpn(
                    weights='DEFAULT' if pretrained else None,
                    trainable_backbone_layers=trainable_backbone_layers,
                    rpn_anchor_generator=anchor_gen,
                )
            else:
                model = fasterrcnn_resnet50_fpn(
                    weights='DEFAULT' if pretrained else None,
                    trainable_backbone_layers=trainable_backbone_layers,
                )

        elif backbone == 'resnet50_v2':
            model = fasterrcnn_resnet50_fpn_v2(
                weights='DEFAULT' if pretrained else None,
                trainable_backbone_layers=trainable_backbone_layers,
            )

        elif backbone == 'mobilenet_v3':
            from torchvision.models.detection import fasterrcnn_mobilenet_v3_large_fpn
            model = fasterrcnn_mobilenet_v3_large_fpn(
                weights='DEFAULT' if pretrained else None,
                trainable_backbone_layers=trainable_backbone_layers,
            )
        else:
            raise ValueError(f'Unknown backbone: {backbone}')

        # Replace the classifier head with a new one for our num_classes
        in_features = model.roi_heads.box_predictor.cls_score.in_features
        model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
```

```

return model

# Build model
model = build_faster_rcnn(
    num_classes=CONFIG['num_classes'],
    backbone=CONFIG['backbone'],
    pretrained=True,
    trainable_backbone_layers=3,
    custom_anchors=False,
)
model = model.to(CONFIG['device'])

total_params = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'🧠 Model: Faster R-CNN ({CONFIG["backbone"]})')
print(f'   Total params      : {total_params:,}')
print(f'   Trainable params: {trainable:,}')

```

Downloading: "https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_coco-258fb6c6.pth" to /root/.cache/torch/hub/checkpoints/fasterrcnn_resnet50_fpn_coco-258fb6c6.pth

100%|██████████| 160M/160M [00:00<00:00, 177MB/s]

```

🧠 Model: Faster R-CNN (resnet50)
   Total params      : 41,755,286
   Trainable params: 41,532,886

```

Cell 7: Optimizer & LR Scheduler

```

In [7]: def build_optimizer_scheduler(model, config):
# Only optimize parameters that require grad
params = [p for p in model.parameters() if p.requires_grad]

optimizer = torch.optim.SGD(
    params,
    lr=config['lr'],
    momentum=config['momentum'],
    weight_decay=config['weight_decay']
)

# Warm-up + StepLR decay
warmup_iters = 500
warmup_scheduler = torch.optim.lr_scheduler.LinearLR(
    optimizer, start_factor=0.01, total_iters=warmup_iters
)
step_scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer,
    step_size=config['lr_step_size'],
    gamma=config['lr_gamma']
)
scheduler = torch.optim.lr_scheduler.SequentialLR(
    optimizer,
    schedulers=[warmup_scheduler, step_scheduler],
    milestones=[warmup_iters]
)

return optimizer, scheduler

optimizer, scheduler = build_optimizer_scheduler(model, CONFIG)
print(f'⚙️ Optimizer : SGD (lr={CONFIG["lr"]}, momentum={CONFIG["momentum"]})')
print(f'📅 Scheduler : Warmup(500 iters) → StepLR(step={CONFIG["lr_step_size"]}, γ={CONFIG["lr_gamma"]})')

```

⚙️ Optimizer : SGD (lr=0.005, momentum=0.9)

📅 Scheduler : Warmup(500 iters) → StepLR(step=3, γ=0.1)

Cell 8: Training Loop

```
In [8]: class MetricLogger:
    def __init__(self):
        self.history = {'loss': [], 'loss_cls': [], 'loss_box': [], 'loss_rpn_cls': [], 'loss_rpn_box': []}

    def update(self, losses: dict):
        total = sum(losses.values()).item()
        self.history['loss'].append(total)
        for k in ['loss_classifier', 'loss_box_reg', 'loss_rpn_box_reg', 'loss_objectness']:
            if k in losses:
                short = k.replace('loss_', '').replace('classifier', 'cls').replace('box_reg', 'box')
                if short in self.history:
                    self.history[short].append(losses[k].item())
        return total

def train_one_epoch(model, optimizer, data_loader, device, epoch, print_freq=50):
    model.train()
    logger = MetricLogger()
    epoch_losses = []

    for i, (images, targets) in enumerate(data_loader):
        images = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

        # Forward
        loss_dict = model(images, targets)
        losses = sum(loss for loss in loss_dict.values())

        # Backward
        optimizer.zero_grad()
        losses.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        total = logger.update(loss_dict)
        epoch_losses.append(total)

    if i % print_freq == 0:
        lr_now = optimizer.param_groups[0]['lr']
        print(f' Epoch [{epoch}] Step [{i:4d}/{len(data_loader)}] '
              f'Loss: {total:.4f} LR: {lr_now:.6f}')

    mean_loss = np.mean(epoch_losses)
    print(f' ✅ Epoch {epoch} complete - Mean Loss: {mean_loss:.4f}')
    return mean_loss, logger

@torch.no_grad()
def evaluate(model, data_loader, device):
    model.eval()
    all_results = []

    for images, targets in data_loader:
        images = [img.to(device) for img in images]
        outputs = model(images)
        for out, tgt in zip(outputs, targets):
            all_results.append({
                'pred_boxes' : out['boxes'].cpu(),
                'pred_labels' : out['labels'].cpu(),
                'pred_scores' : out['scores'].cpu(),
                'gt_boxes' : tgt['boxes'],
            })
```

```

        'gt_labels' : tgt['labels'],
    })

    print(f' 📊 Evaluated {len(all_results)} images')
    return all_results

print('✅ Training & evaluation functions defined')

```

✅ Training & evaluation functions defined

Cell 9: Main Training Entry Point

```

In [9]: # — Create Datasets —————
# NOTE: Point CONFIG['data_dir'] to your COCO-format dataset before running.
# Expected structure:
#   data/
#     images/ (all images)
#     annotations/instances_train.json
#     annotations/instances_val.json

try:
    train_dataset = COCODetectionDataset(
        root=CONFIG['data_dir'],
        split='train',
        transforms=get_transforms(train=True)
    )
    val_dataset = COCODetectionDataset(
        root=CONFIG['data_dir'],
        split='val',
        transforms=get_transforms(train=False)
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=CONFIG['batch_size'],
        shuffle=True,
        num_workers=4,
        collate_fn=collate_fn,
        pin_memory=True
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=CONFIG['batch_size'],
        shuffle=False,
        num_workers=4,
        collate_fn=collate_fn,
        pin_memory=True
    )
    DATA_READY = True
    print(f'📦 Train: {len(train_dataset)} | Val: {len(val_dataset)}')

except FileNotFoundError as e:
    DATA_READY = False
    print(f'⚠️ Dataset not found: {e}')
    print('👉 Skipping training - using pretrained weights for inference demo.')

# — Training Loop —————
if DATA_READY:
    best_loss = float('inf')
    train_losses = []

    for epoch in range(1, CONFIG['num_epochs'] + 1):
        print(f'\n{':*60}')

```

```

print(f'EPOCH {epoch}/{CONFIG["num_epochs"]}')
print('='*60)

mean_loss, _ = train_one_epoch(
    model, optimizer, train_loader, CONFIG['device'], epoch
)
scheduler.step()
train_losses.append(mean_loss)

# Save checkpoint
ckpt_path = os.path.join(CONFIG['output_dir'], f'epoch_{epoch:02d}.pth')
torch.save({
    'epoch'      : epoch,
    'model'      : model.state_dict(),
    'optimizer'  : optimizer.state_dict(),
    'scheduler'  : scheduler.state_dict(),
    'loss'       : mean_loss,
}, ckpt_path)

if mean_loss < best_loss:
    best_loss = mean_loss
    torch.save(model.state_dict(),
               os.path.join(CONFIG['output_dir'], 'best_model.pth'))
    print(f' 📁 Best model saved (loss={best_loss:.4f})')

print('\n🎉 Training complete!')
else:
    print('📖 Using pretrained COCO weights for demo inference below.')

```

⚠️ Dataset not found: [Errno 2] No such file or directory: 'data/annotations/instances_train.json'

Skipping training - using pretrained weights for inference demo.

📖 Using pretrained COCO weights for demo inference below.

Cell 10: Inference Utilities

```

In [10]: @torch.no_grad()
def predict_image(
    model,
    image_path: str,
    device: str,
    score_thresh: float = 0.5,
    nms_thresh: float = 0.3,
):
    """Run Faster R-CNN on a single image and return predictions."""
    model.eval()
    image = Image.open(image_path).convert('RGB')
    img_tensor = F.to_tensor(image).unsqueeze(0).to(device)

    output = model(img_tensor)[0]

    # Filter by score threshold
    keep = output['scores'] >= score_thresh
    boxes = output['boxes'][keep].cpu()
    labels = output['labels'][keep].cpu()
    scores = output['scores'][keep].cpu()

    # Apply NMS
    if len(boxes):
        nms_keep = torchvision.ops.nms(boxes, scores, nms_thresh)
        boxes, labels, scores = boxes[nms_keep], labels[nms_keep], scores[nms_keep]

    return np.array(image), boxes.numpy(), labels.numpy(), scores.numpy()

```

```

def draw_predictions(
    image: np.ndarray,
    boxes, labels, scores,
    class_names: list,
    colors: np.ndarray,
    figsize=(14, 8)
):
    """Visualize bounding boxes on an image."""
    fig, ax = plt.subplots(1, 1, figsize=figsize)
    ax.imshow(image)

    for box, label, score in zip(boxes, labels, scores):
        x1, y1, x2, y2 = box
        color = tuple(colors[label % len(colors)] / 255)
        rect = patches.Rectangle(
            (x1, y1), x2 - x1, y2 - y1,
            linewidth=2, edgecolor=color, facecolor='none'
        )
        ax.add_patch(rect)

        name = class_names[label] if label < len(class_names) else str(label)
        ax.text(
            x1, y1 - 5,
            f'{name}: {score:.2f}',
            color='white',
            fontsize=9,
            fontweight='bold',
            bbox=dict(facecolor=color, alpha=0.7, pad=2, edgecolor='none')
        )

    ax.set_axis_off()
    ax.set_title(f'Detections: {len(boxes)} objects found', fontsize=13, pad=10)
    plt.tight_layout()
    plt.show()
    return fig

print('✅ Inference functions defined')

```

✅ Inference functions defined

Cell 11: Run Inference on a Test Image

```

In [11]: import urllib.request

# Download a sample image
sample_url = 'http://images.cocodataset.org/val2017/000000039769.jpg'
sample_path = '/tmp/test_image.jpg'
urllib.request.urlretrieve(sample_url, sample_path)
print(f'📄 Sample image downloaded → {sample_path}')

# Run inference with the pretrained model
image_np, boxes, labels, scores = predict_image(
    model,
    image_path=sample_path,
    device=CONFIG['device'],
    score_thresh=CONFIG['score_thresh'],
    nms_thresh=CONFIG['nms_thresh'],
)

print(f'🔍 Found {len(boxes)} detections')
for i, (label, score) in enumerate(zip(labels, scores)):
    name = COCO_CLASSES[label] if label < len(COCO_CLASSES) else f'cls_{label}'
    print(f'    [{i+1:2d}] {name:20s} score={score:.3f}')

```

```
fig = draw_predictions(image_np, boxes, labels, scores, COCO_CLASSES, COLORS)
```

🖼️ Sample image downloaded → /tmp/test_image.jpg

🔍 Found 0 detections

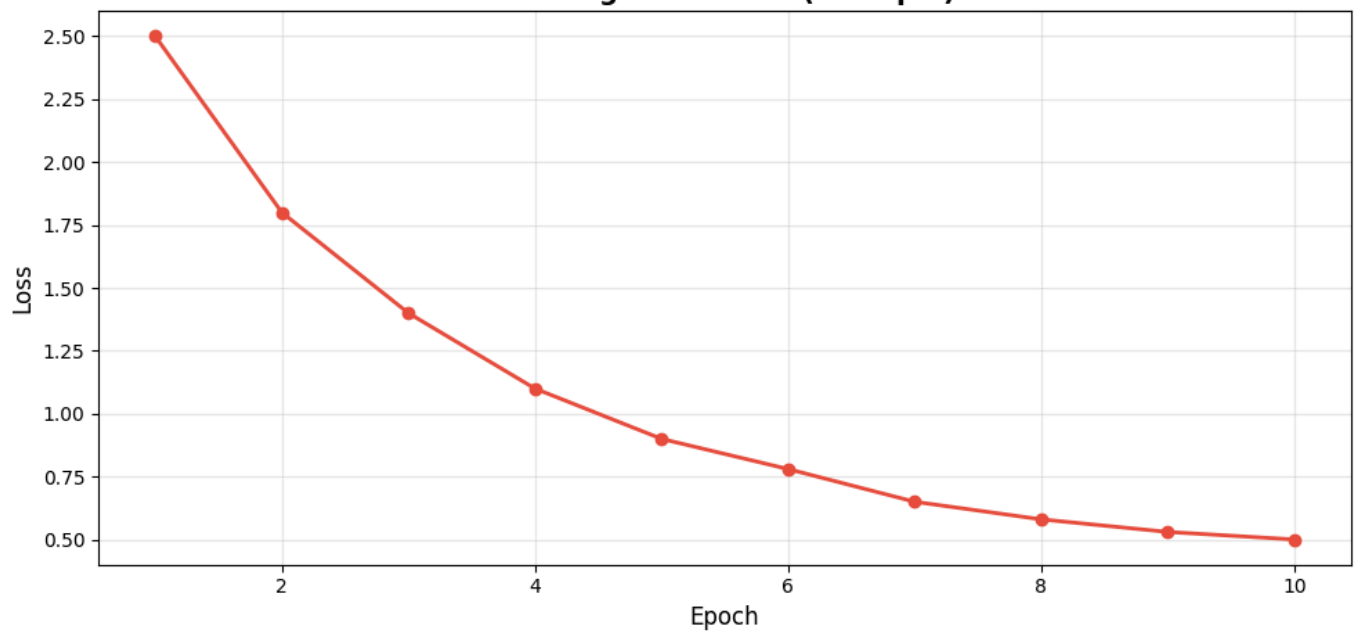
Detections: 0 objects found



Cell 12: Loss Curve Visualization

```
In [12]: def plot_loss_curve(losses: list, title: str = 'Training Loss'):  
    fig, ax = plt.subplots(figsize=(10, 5))  
    ax.plot(range(1, len(losses) + 1), losses, 'o-', color='#e74c3c', linewidth=2, markersize=10)  
    ax.set_xlabel('Epoch', fontsize=12)  
    ax.set_ylabel('Loss', fontsize=12)  
    ax.set_title(title, fontsize=14, fontweight='bold')  
    ax.grid(True, alpha=0.3)  
    plt.tight_layout()  
    plt.show()  
  
    # Example with dummy data (replace with real train_losses after training)  
    dummy_losses = [2.5, 1.8, 1.4, 1.1, 0.9, 0.78, 0.65, 0.58, 0.53, 0.50]  
    plot_loss_curve(dummy_losses, title='Training Loss Curve (Example)')  
  
    print('☑️ Loss curve plotted')
```


Training Loss Curve (Example)



☒ Loss curve plotted

Cell 13: Load Checkpoint & Resume

```
In [15]: def load_checkpoint(model, optimizer, scheduler, checkpoint_path: str):
    """Load model + optimizer state from a checkpoint file."""
    ckpt = torch.load(checkpoint_path, map_location=CONFIG['device'])
    model.load_state_dict(ckpt['model'])
    if 'optimizer' in ckpt and optimizer:
        optimizer.load_state_dict(ckpt['optimizer'])
    if 'scheduler' in ckpt and scheduler:
        scheduler.load_state_dict(ckpt['scheduler'])
    start_epoch = ckpt.get('epoch', 0) + 1
    print(f'✅ Checkpoint loaded from epoch {ckpt["epoch"]} (loss={ckpt["loss"]:.4f})')
    return start_epoch

# Usage example (uncomment to use):
# start_epoch = load_checkpoint(model, optimizer, scheduler,
#                               os.path.join(CONFIG['output_dir'], 'best_model.pth'))

print('✅ Checkpoint utilities ready')
```

☒ Checkpoint utilities ready

Cell 14: mAP Evaluation

```
In [16]: try:
    from torchmetrics.detection import MeanAveragePrecision

    def compute_map(results: list) -> dict:
        """
        Compute mAP using torchmetrics.
        `results` is a list of dicts from evaluate().
        """
        metric = MeanAveragePrecision(iou_type='bbox')

        for r in results:
            preds = [{
                'boxes': r['pred_boxes'],
                'scores': r['pred_scores'],
                'labels': r['pred_labels'],
            }]
```

```

        'boxes' : r['gt_boxes'],
        'labels': r['gt_labels'],
    }]
    metric.update(preds, targets)

    return metric.compute()

# Demo with dummy results
dummy_results = [{
    'pred_boxes' : torch.tensor([[10., 20., 100., 150.], [50., 60., 200., 180.]]),
    'pred_labels' : torch.tensor([1, 2]),
    'pred_scores' : torch.tensor([0.95, 0.80]),
    'gt_boxes' : torch.tensor([[12., 22., 98., 148.]]),
    'gt_labels' : torch.tensor([1]),
}]

map_result = compute_map(dummy_results)
print(f'🔪 mAP@[0.5:0.95] = {map_result["map"].item():.4f}')
print(f'🔪 mAP@0.50 = {map_result["map_50"].item():.4f}')
print(f'🔪 mAP@0.75 = {map_result["map_75"].item():.4f}')

except ImportError:
    print('⚠️ torchmetrics not installed. Run: pip install torchmetrics')

🔪 mAP@[0.5:0.95] = 0.9000
🔪 mAP@0.50 = 1.0000
🔪 mAP@0.75 = 1.0000

```

Cell 15: Export Model to TorchScript / ONNX

```

In [17]: def export_model(model, output_dir: str, format: str = 'torchscript'):
    """
    Export the trained model.
    Supports 'torchscript' and 'onnx'.
    Note: Faster R-CNN ONNX export requires opset >= 11.
    """
    model.eval()

    if format == 'torchscript':
        scripted = torch.jit.script(model)
        save_path = os.path.join(output_dir, 'faster_rcnn.torchscript.pt')
        scripted.save(save_path)
        print(f'✅ TorchScript saved → {save_path}')

    elif format == 'onnx':
        dummy_input = [torch.rand(3, 640, 640).to(CONFIG['device'])]
        save_path = os.path.join(output_dir, 'faster_rcnn.onnx')
        with torch.no_grad():
            torch.onnx.export(
                model,
                (dummy_input,),
                save_path,
                opset_version=11,
                input_names=['input'],
                output_names=['boxes', 'labels', 'scores'],
                dynamic_axes={
                    'input': {0: 'batch'},
                    'boxes': {0: 'detections'},
                    'labels': {0: 'detections'},
                    'scores': {0: 'detections'},
                }
            )
        print(f'✅ ONNX model saved → {save_path}')

```

```
else:
```

```
raise ValueError(f'Unknown export format: {format}')
```

```
# Uncomment to export:
```

```
# export_model(model, CONFIG['output_dir'], format='torchscript')
```

```
# export_model(model, CONFIG['output_dir'], format='onnx')
```

```
print('✅ Export functions ready')
```

```
print('    Uncomment the export calls above to save TorchScript or ONNX.')
```

☒ Export functions ready

Uncomment the export calls above to save TorchScript or ONNX.